

Data Engineering on Google Cloud — Day 1 of 2

Iowa Department of Management, Department of IT · October 5, 2026 · Student slide pack

- Slides follow the class agenda
- Hands-on activities: <https://roittraining.github.io/gcp-demos/bigquery.html>
- Labs are in the Google Skills classroom

Agenda — Monday, October 5 (1 of 2) · morning

- | | | |
|-------------|---|-----|
| 09:00–09:25 | Welcome, audience survey & the reframe — your two-day migration story | |
| 09:25–10:10 | Module 1 — Data Engineering Tasks & Components + GCP platform orientation | |
| 10:10–10:25 | Exploring projects, datasets & tables — star public projects, create your class dataset | DIN |
| 10:25–10:40 | Morning break | |
| 10:40–11:10 | Loading Data into BigQuery — the first lab, a confidence builder | LAB |
| 11:10–11:50 | Module 2 — Data Replication & Migration: killing the nightly extracts | |

Agenda — Monday, October 5 (2 of 2) · afternoon

11:50–13:15	Datastream: PostgreSQL → BigQuery — start the stream, lunch while it replicates, validate the CDC moment (lunch 12:00–13:00)	LAB
13:15–13:45	Module 3 — Extract & Load: the pattern for data that still arrives as files	
13:45–14:30	Loading block — console & CLI load, the autodetect pitfall, CSV vs Parquet race, LOAD DATA SQL (break 14:30–14:45)	DIN
14:45–15:25	Module 4 — Extract, Load & Transform (ELT) + the Dataform walkthrough: your Silver→Gold tool	
15:25–16:05	Create & Execute a SQL Workflow in Dataform — the anchor lab	LAB
16:05–16:30	Modules 5+6 compressed: ETL options & automation — then the scheduled-query closing demo	DEMO

Welcome — who this course is for

Iowa DOM DoIT data team · Oct 5–6, 2026 · 9:00 AM–4:30 PM CT

- ✓ ETL developers living in SQL Server and SSIS— sources, transforms, sinks are your daily work
- ✓ A dedicated M&O team— you will own what gets designed here, so operations questions are welcome
- ✓ Power BI report authors— the last mile matters: your Gold marts feed the reports you already have
- ✓ Near-zero GCP console experience across the room— that is expected, and this course is built for it
- ✓ Senior in data engineering, new to one console— we compress concepts, slow down on where things live

05:00

Introduction to Data Engineering on Google Cloud

Name
Role

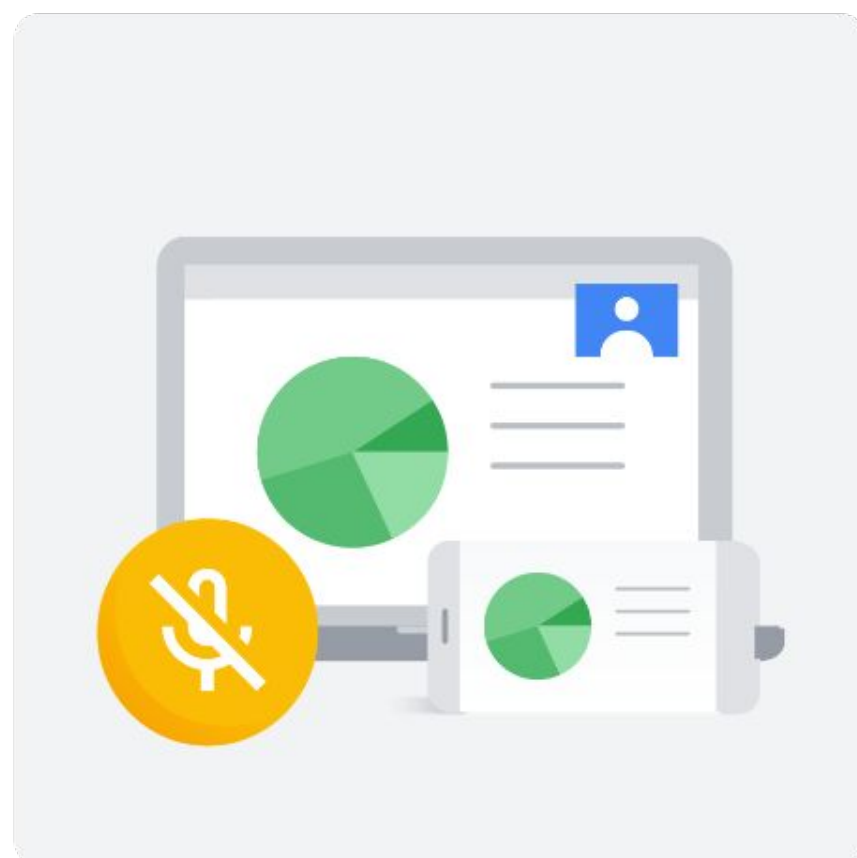
Your instructor — Imran Ahmad, PhD

Google Cloud Authorized Trainer · AWS Authorized Instructor · Microsoft Certified Trainer

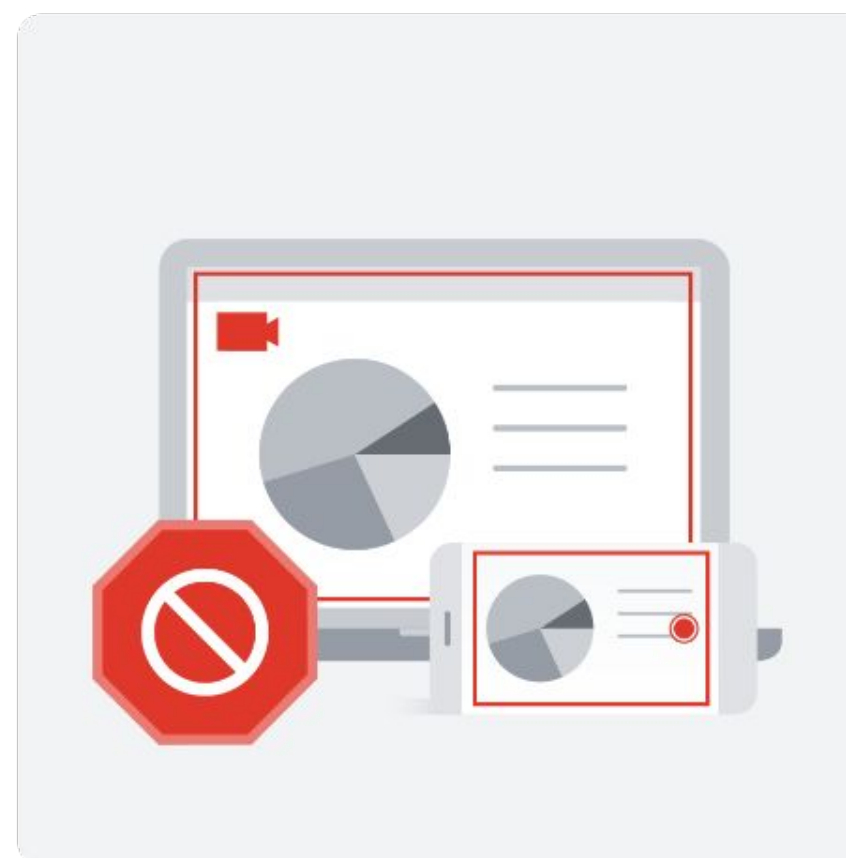
- Large-scale data engineering is the craft — production pipelines over high-volume, multi-source datasets at national scale, across Google Cloud, AWS, and Azure
- PhD in data processing — created a large-scale data-processing algorithm on cloud/grid computing; pioneered techniques applied to data-intensive science, including neutrino-particle experiments
- Taught data engineering and cloud for Citibank, JP Morgan, the United Nations, Deloitte, and the European Commission
- Second act: AI and agents — author of 30 Agents Every AI Engineer Must Build and 50 Algorithms Every Programmer Should Know (Packt)
- Fluent in your world and your target — SQL Server and SSIS to BigQuery; these two days map that craft onto your migration



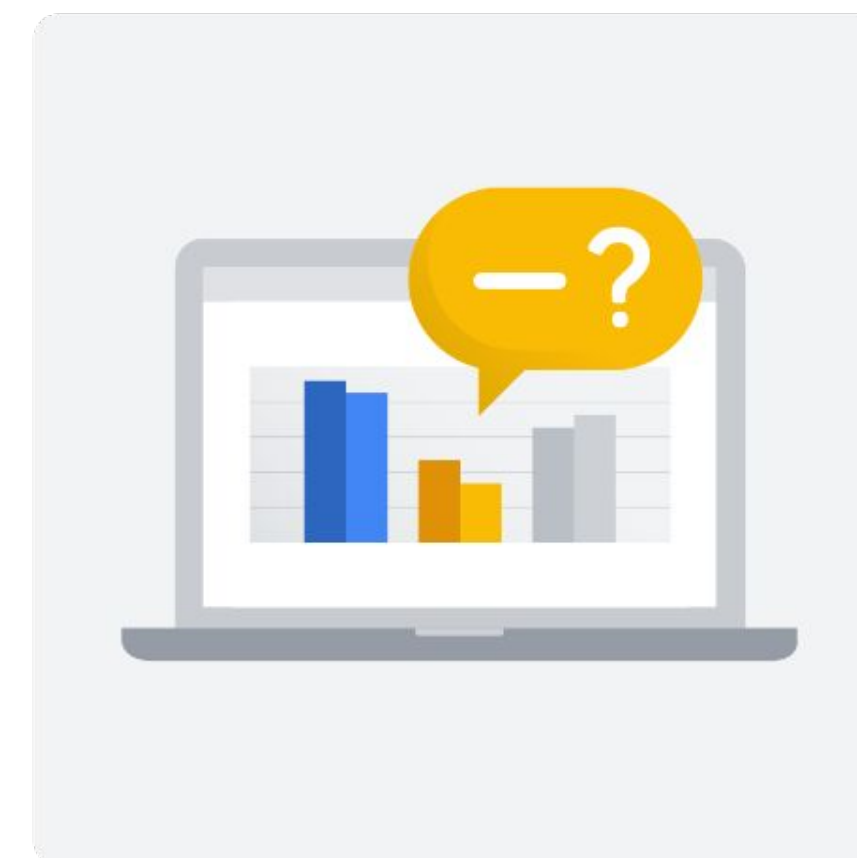
Etiquette



Mute microphone



No recording



Ask questions

The reframe: two days = your migration story

Not a generic Google Cloud tour — every module hangs on the project you are actually running.

- Kill the nightly extracts— Datastream change data capture replicates continuously, no extract window
- Land in BigQuery— serverless, no instance, no patching; your SQL Server DW's replacement
- Bronze / Silver / Gold— the medallion lakehouse from your own architecture doc, taught on Day 2
- Dataform for ELT— SQL workflows and release configs; your SSIS-in-git CI/CD discipline translates
- Power BI last mile— Gold marts feed the reports you already have (Day 2 afternoon)
- Single source of truth— BigQuery becomes the analytics source; SQL Server stays system of record

Every demo, DIN activity and lab in the next two days hangs on one of these steps.

Hands up — a quick survey before we start

Your answers set the depth for the next two days. Calibration, not a quiz.

- ETL developers— who builds and maintains the SQL Server / SSIS packages today?
- M&O team— who keeps it running and will inherit whatever we design here?
- Power BI authors— who builds the reports sitting on top of the marts?
- Who has opened the Google Cloud console— even once, even just to click around?
- Who has written a query in BigQuery, anywhere, ever?

Answer honestly, including 'never opened it' — the room is senior in DE, new to this platform.

Day 1 — Landing your data in BigQuery

09:00–09:25

Welcome, hands-up survey, the reframe (this block)

09:25–10:25

M1 DE tasks & components + console orientation — create your class dataset

DIN

10:40–11:10

Lab: Loading Data into BigQuery — your first win in a new console

LAB

11:10–13:15

M2 killing the nightly extracts — Datastream CDC lab runs across lunch

LAB

13:15–14:30

M3 extract & load + four loading DINs — autodetect pitfalls, CSV vs Parquet

DIN

14:45–16:30

M4 ELT + Dataform demo and anchor lab; M5/M6 compressed; day close

LAB

Day 2 — Lakehouse & medallion, your target model

09:00–09:15

Recap quiz — which GCP tool replaces which piece of your current stack

QUIZ

09:15–10:30

M7 lake vs warehouse vs lakehouse + M8 building a data lakehouse

10:45–12:00

Federated query lab, external tables (Bronze made tangible), M9 modernizing DWs

LAB

13:00–14:35

Iceberg lab + deep-dive; M10 governance + Google's own Bronze/Silver/Gold slides

LAB

14:50–15:20

M11 compressed — BQML direction + the Power BI to BigQuery last mile

DEMO

15:20–16:30

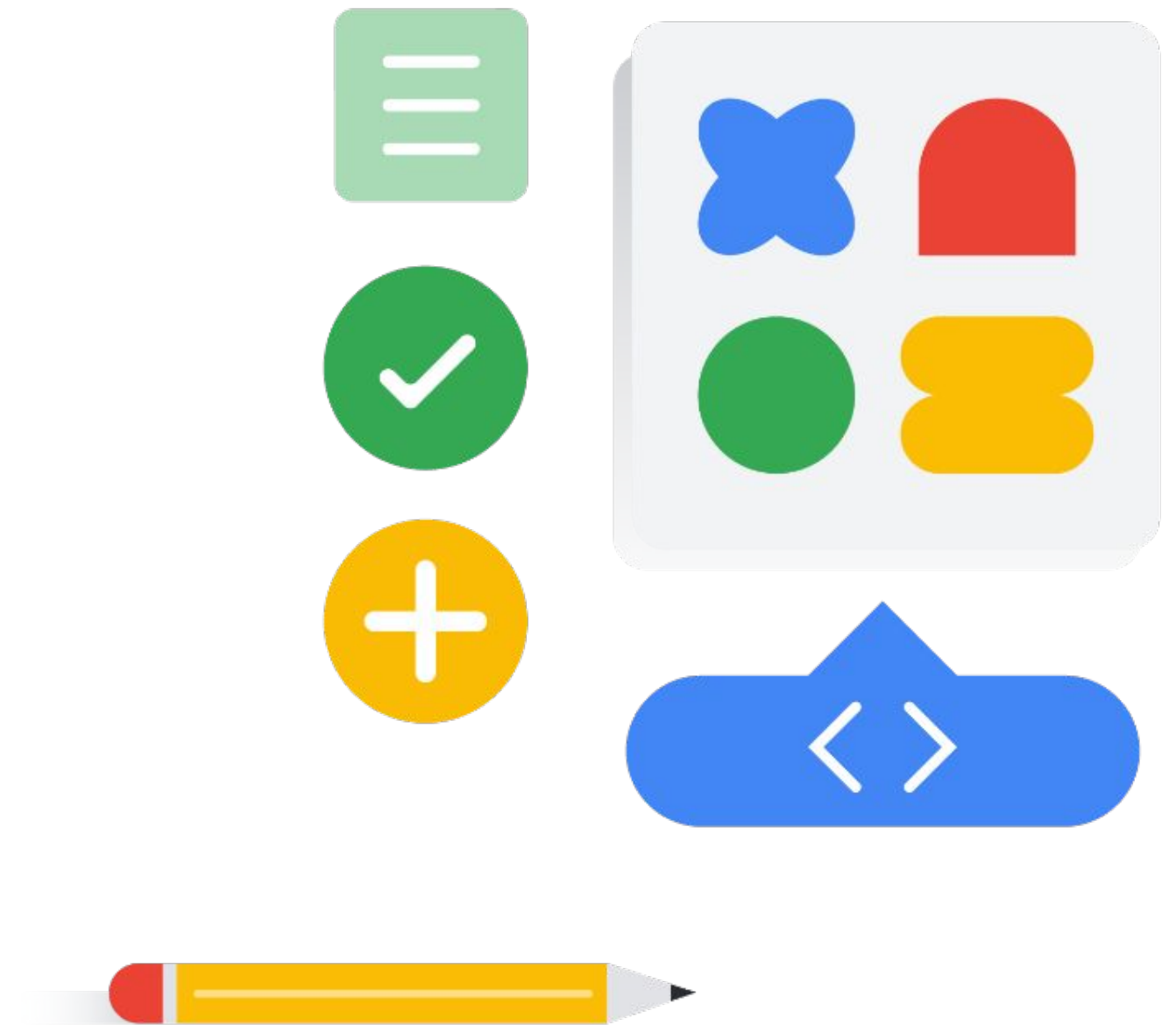
Capstone workshop: map one of your real extract jobs; summary and Q&A

WORKSHOP

Lab environment

For each lab, Google Skills offers:

- ✓ A free set of resources for a fixed amount of time
- ✓ A clean environment with permissions



Open Google Skills

- 1 Open an incognito window (or private and/or anonymous window).
- 2 Go to the URL your instructor provides.
- 3 Sign In with existing account or Join with new account (with email you used to register for the course).
- 4 Launch the course from the Dashboard page.

Access issues

The process to open Google Skills can differ based on credentials used. Please reach out to your instructor if you have any access issues.

Up next

Activities

Classrooms

SCHEDULED COURSE

Introduction to Data
Engineering on Google Cloud

▶ Join

View your labs

Do **NOT** launch a lab until instructed to do so!

Labs	Lecture Notes
	
	
	Lab Currently Disabled
	Lab Currently Disabled

- ← Lab completed
- ← To be completed
- ← Not yet available

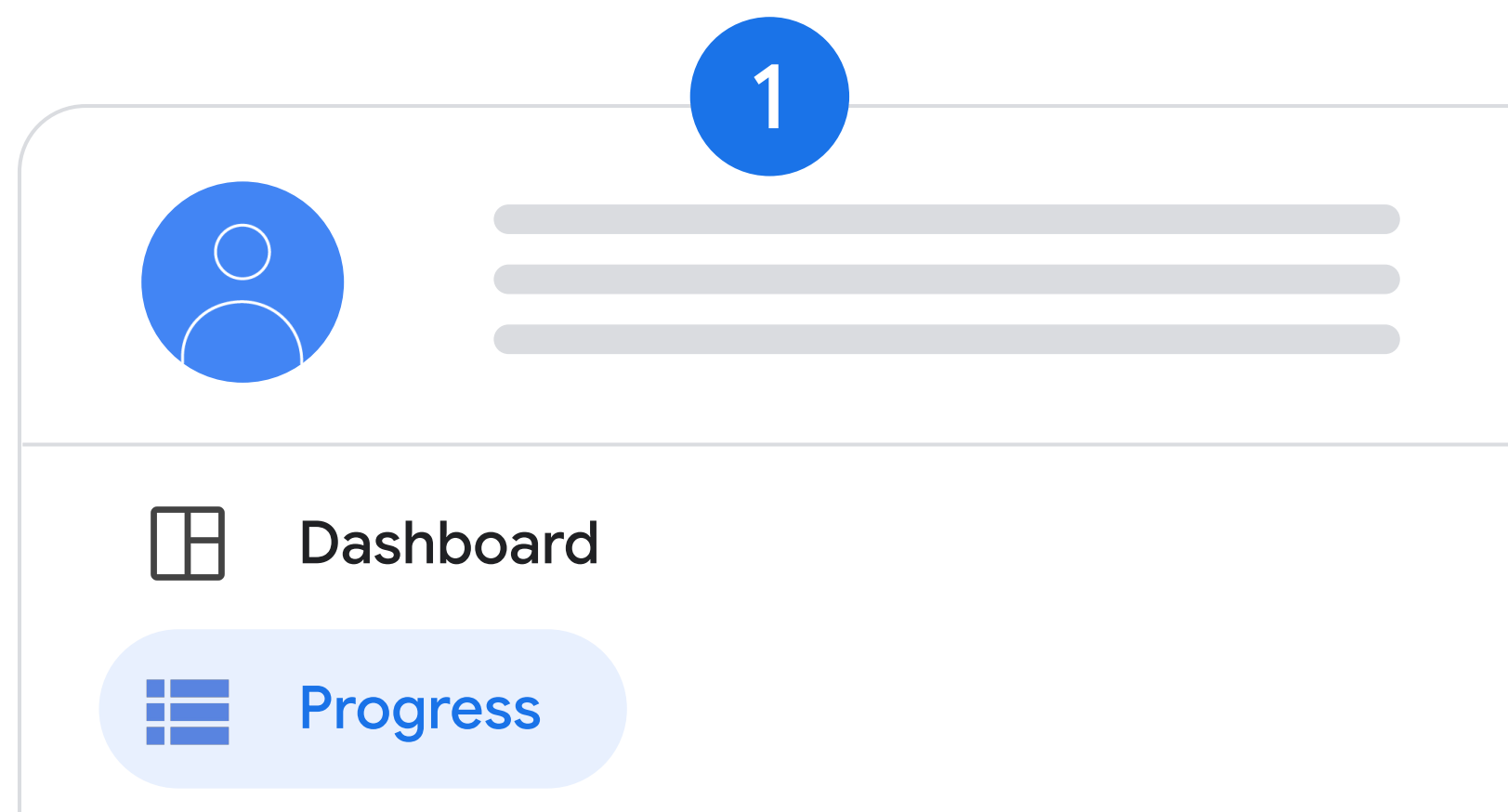
View lecture notes

Labs	Lecture Notes
01	⬇
02	⬇
03	⬇
04	⬇

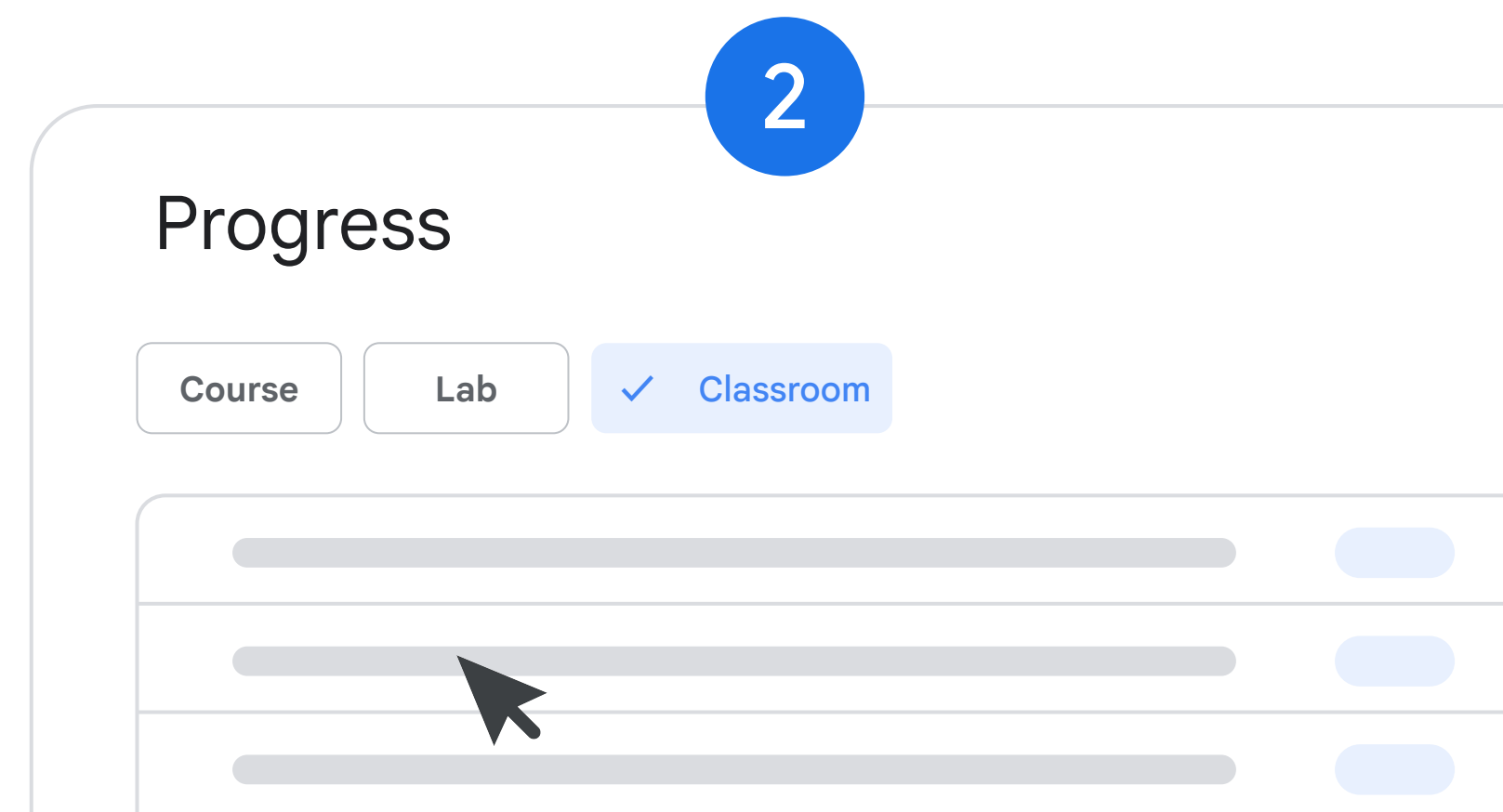
You can download these as PDF files

End of class - Materials

Materials are available for 2 years



Click on your **Profile** and select **Progress**.



Select the class from the **Classroom** list.

Logistics and ground rules

- Google Skills classroom—
<https://www.skills.google/ilt/classrooms/37990>
— all lab steps live there
- Temporary lab projects— each lab gives a throwaway GCP project plus a lab timer; nothing bills to you
- Labs are safe to break— disposable projects exist for experiments; break things freely
- Lunch 12:00–1:00 PM CT— on Day 1 the Datastream lab keeps replicating while we eat
- Breaks— 15 minutes mid-morning and mid-afternoon, both days
- Parking lot— off-track questions get written down and answered at breaks or Day 2 flex time
- Pre-course survey— fill your row if you have not yet: role, 0–3 self-ratings, one extract job you own (link in the chat and the student pack)

Keep two browser tabs ready each morning: the Skills classroom and the Google Cloud console.
Pre-course survey: survey.anum.cloud — student pack: github.anum.cloud

Take these with you — scan any code

Everything from these two days, in your pocket before you leave the room.



Pre-course survey

survey.anum.cloud



Student pack — notes, slides, labs

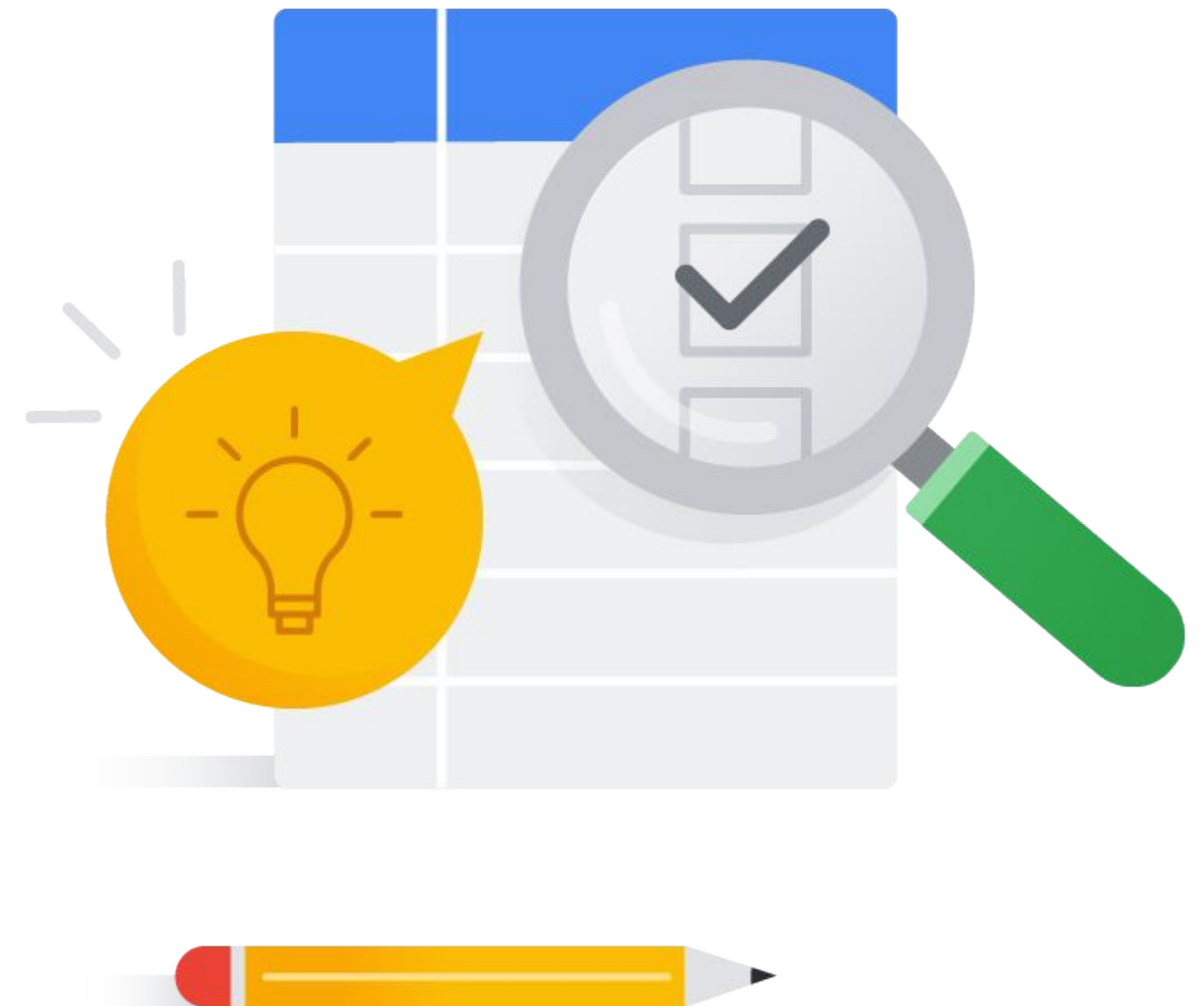
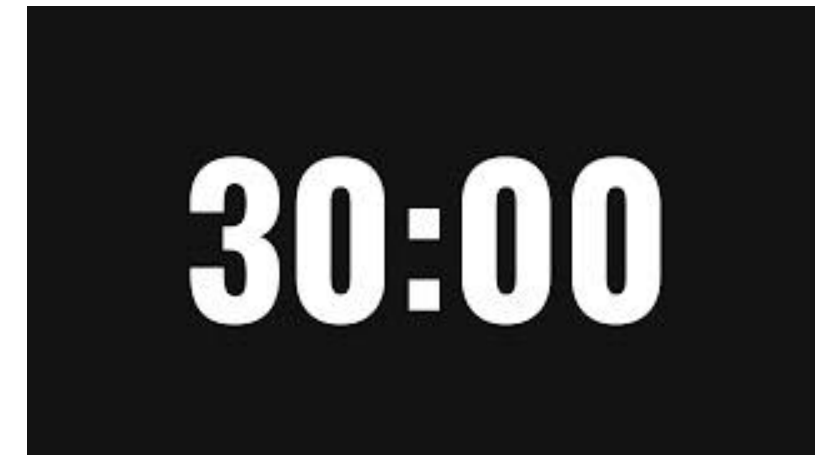
github.anum.cloud

Lab: Loading Data into BigQuery

🕒 30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



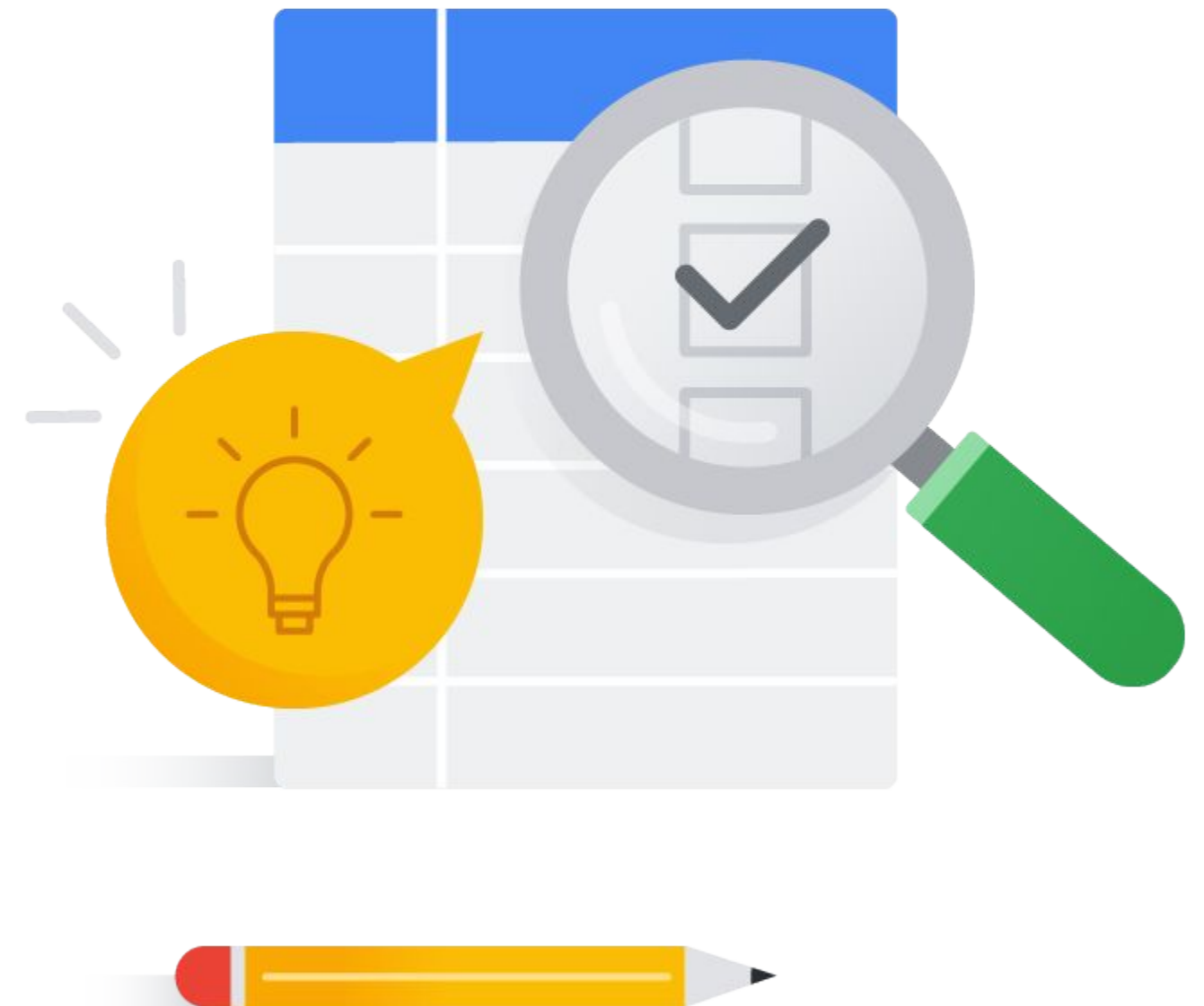
Lab (optional): Datastream: PostgreSQL Replication to BigQuery

45:00

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.

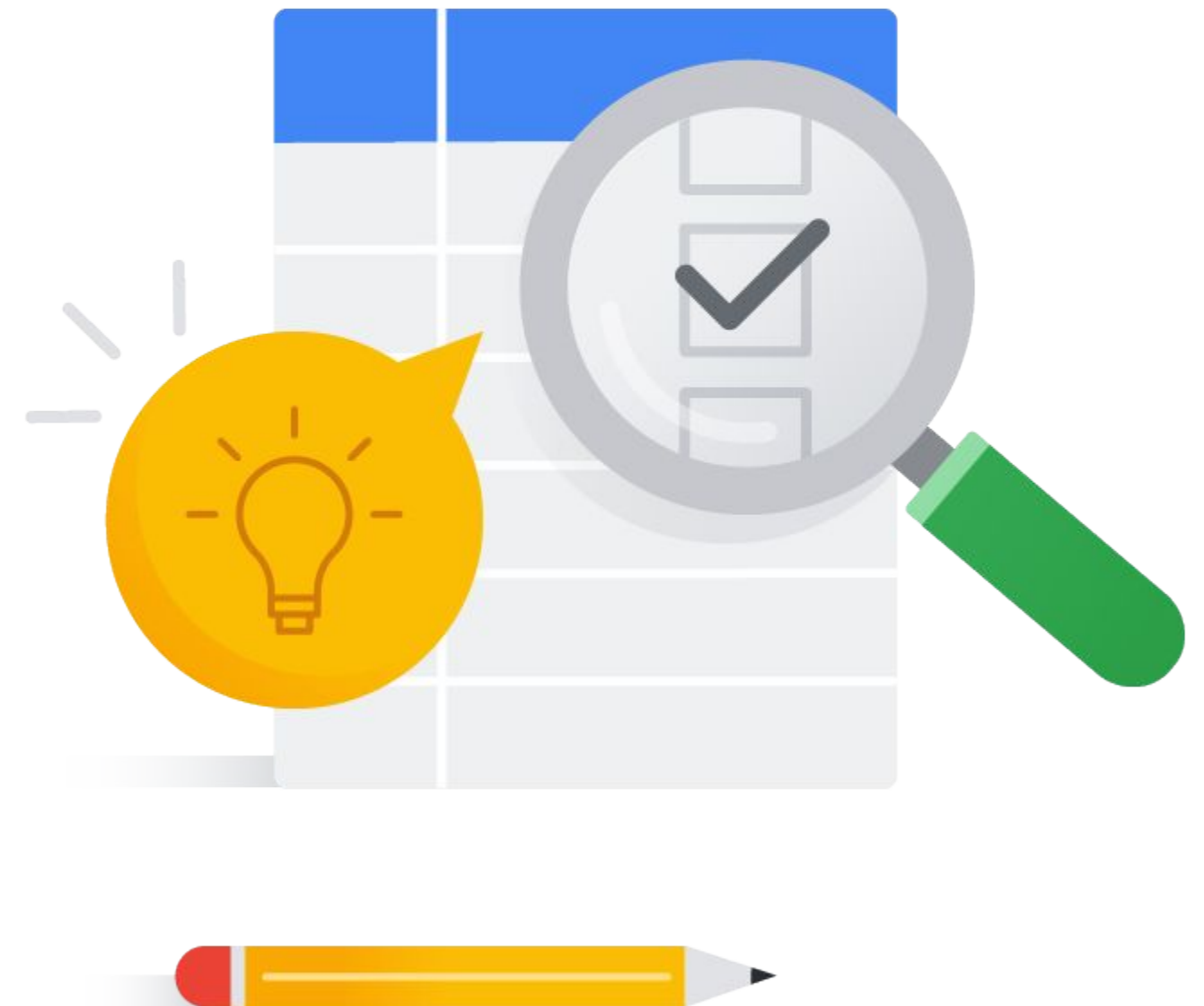
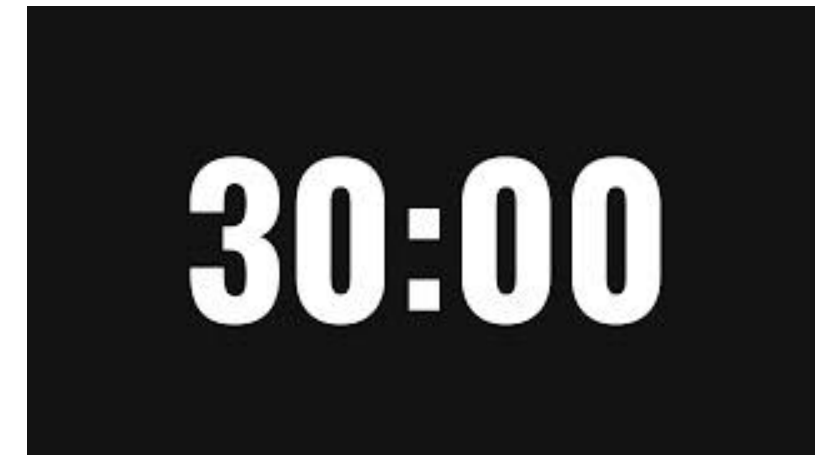


Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Google Cloud

Next at 09:25 — M1: Data Engineering Tasks & Components

- ✓ You have the shape of the two days— kill the extracts, land in BigQuery, medallion, Dataform, Power BI
- ✓ M1 maps your SSIS mental model 1:1— sources, transforms, sinks with Google service names on each box
- ✓ And orients you in the platform— console, projects, IAM, billing, regions
- ✓ At 10:10 you go hands-on— star two projects and create your class dataset (everything after lunch needs it)
- ✓ Tabs ready: Skills classroom + Google Cloud console

WATCH FIRST · 2:27 · GOOGLE CLOUD TECH

What is BigQuery?

- ✓ The two-minute answer before we open the console: a serverless warehouse — no instance to start, no version to patch, storage and compute scaled apart.
- ✓ Watch for: why 'serverless' is not a marketing word — it is the reason you never size anything this week.



Scan to open on YouTube

youtu.be/m8WqxLd1jSc

M1 — What this block gets you

- ✓ Your SSIS mental model, unchanged— sources → transforms → sinks maps 1:1 onto Google Cloud
- ✓ Where every piece lives in GCP— console, projects, IAM, billing, regions
- ✓ Why file format matters more than file location— CSV vs Parquet sets up this afternoon
- ✓ BigQuery as your target warehouse— serverless, no instance, no patching
- ✓ Hands-on: orient in the console, create your class dataset, then your first load lab

This morning, block by block

09:25–10:10 M1 — Data engineering tasks & components, plus GCP platform orientation

10:10–10:25 Console orientation — star the demo projects, create your class dataset

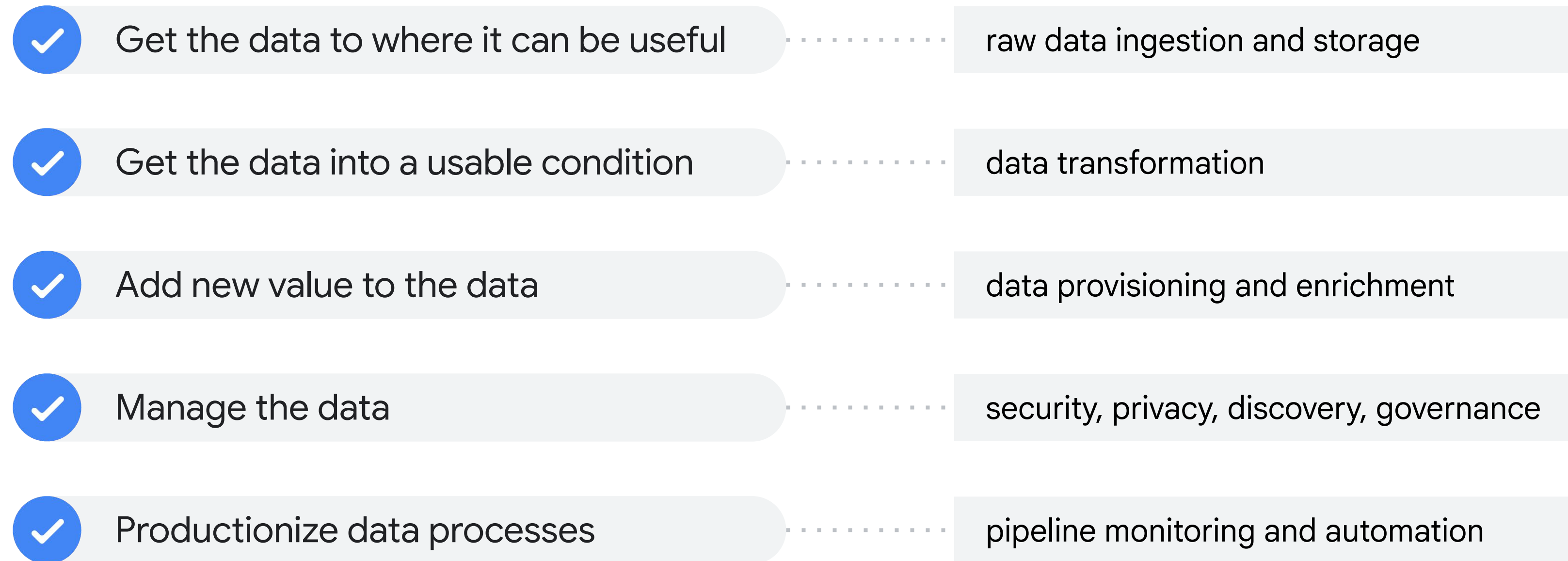
DIN

10:25–10:40 Break

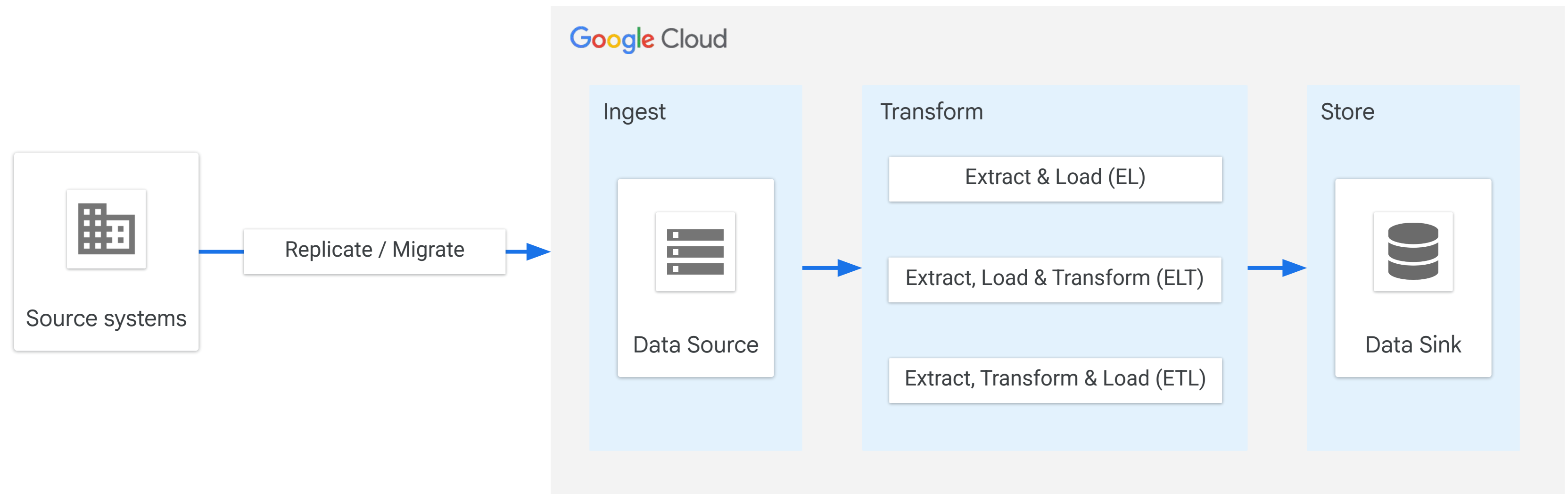
10:40–11:10 Lab: Loading Data into BigQuery — your first win in a new console

LAB

A data engineer builds data pipelines to enable data-driven decisions



Data engineering tasks evolve around ingesting, transforming, and storing data

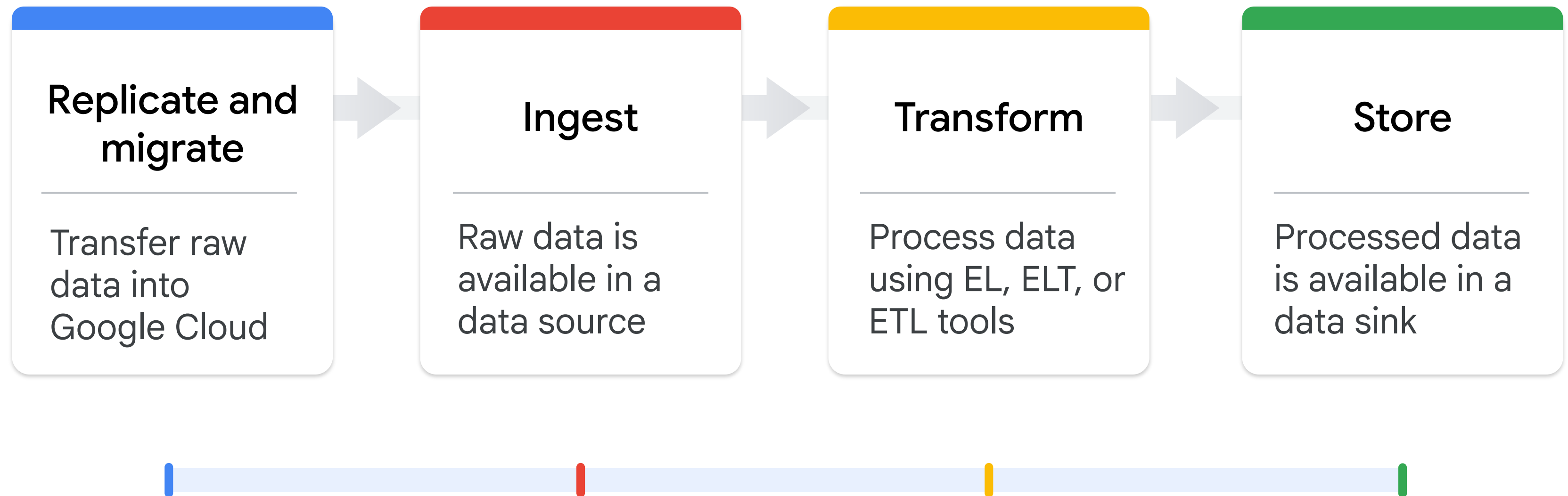


Your SQL Server stack, translated

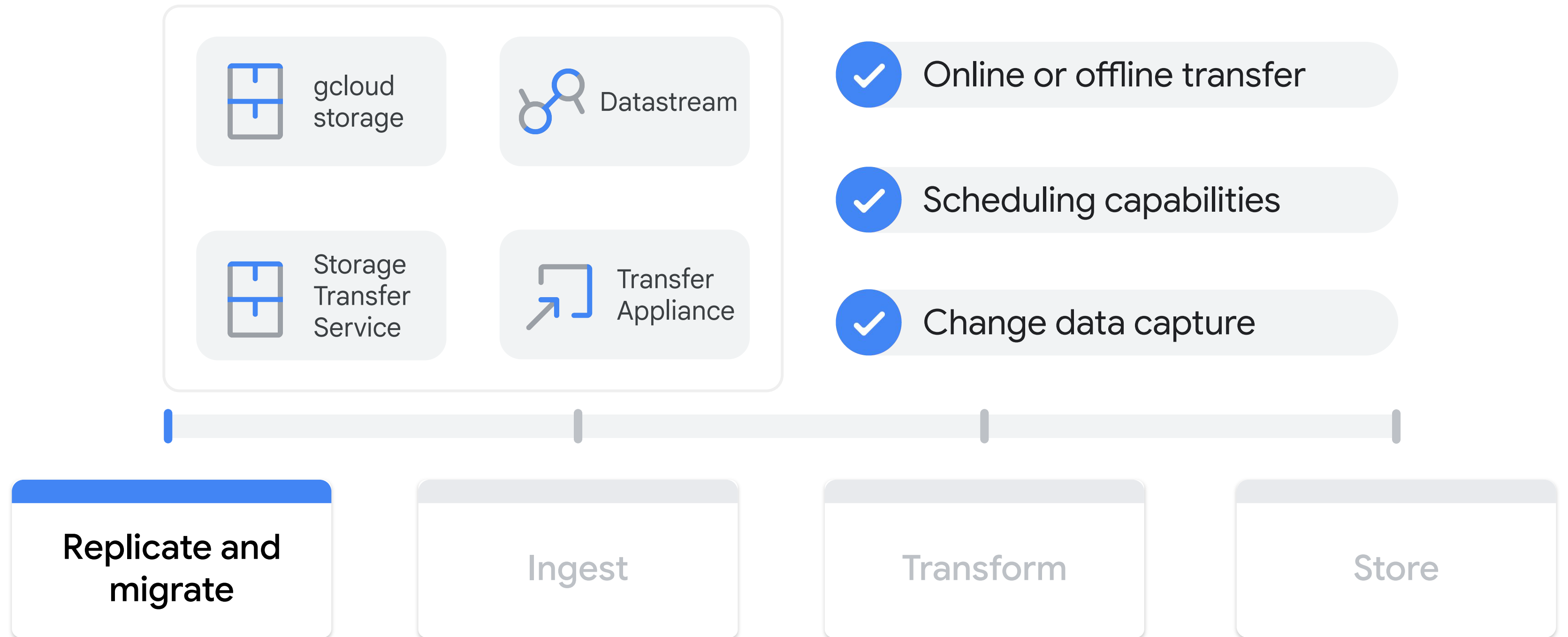
Everything on the next slides is a tool you already own, wearing a Google name.

- SSIS data flow— sources → transforms → sinks, unchanged as a mental model
- SQL Agent jobs— Cloud Scheduler and BigQuery scheduled queries (end of Day 1)
- SSIS packages in git + CI/CD— Dataform repositories and release configs (this afternoon)
- Nightly extract windows— Datastream change data capture, replicating continuously (M2)
- Production SQL Server— stays the system of record while BigQuery becomes the analytics source of truth
- Your data marts → Power BI— Gold marts in BigQuery feed the reports you already have (Day 2)

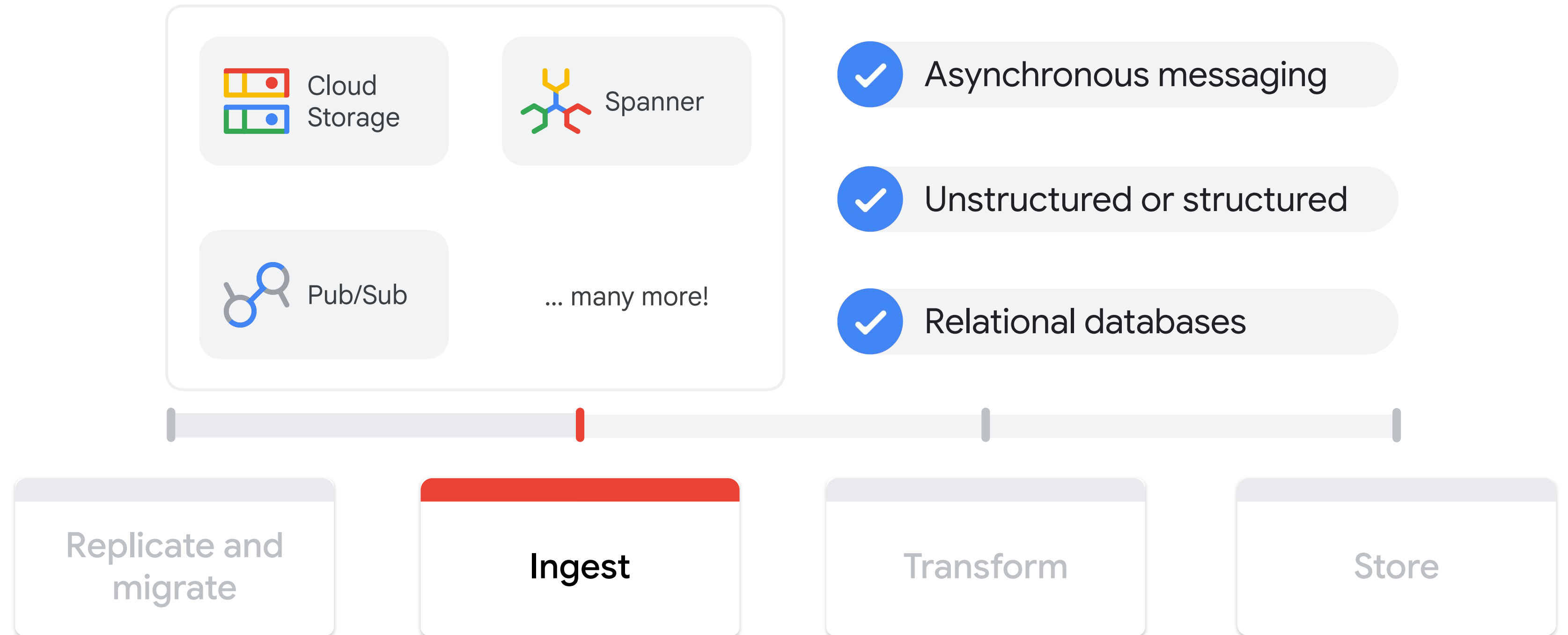
Data engineering tasks evolve around ingesting, transforming, and storing data



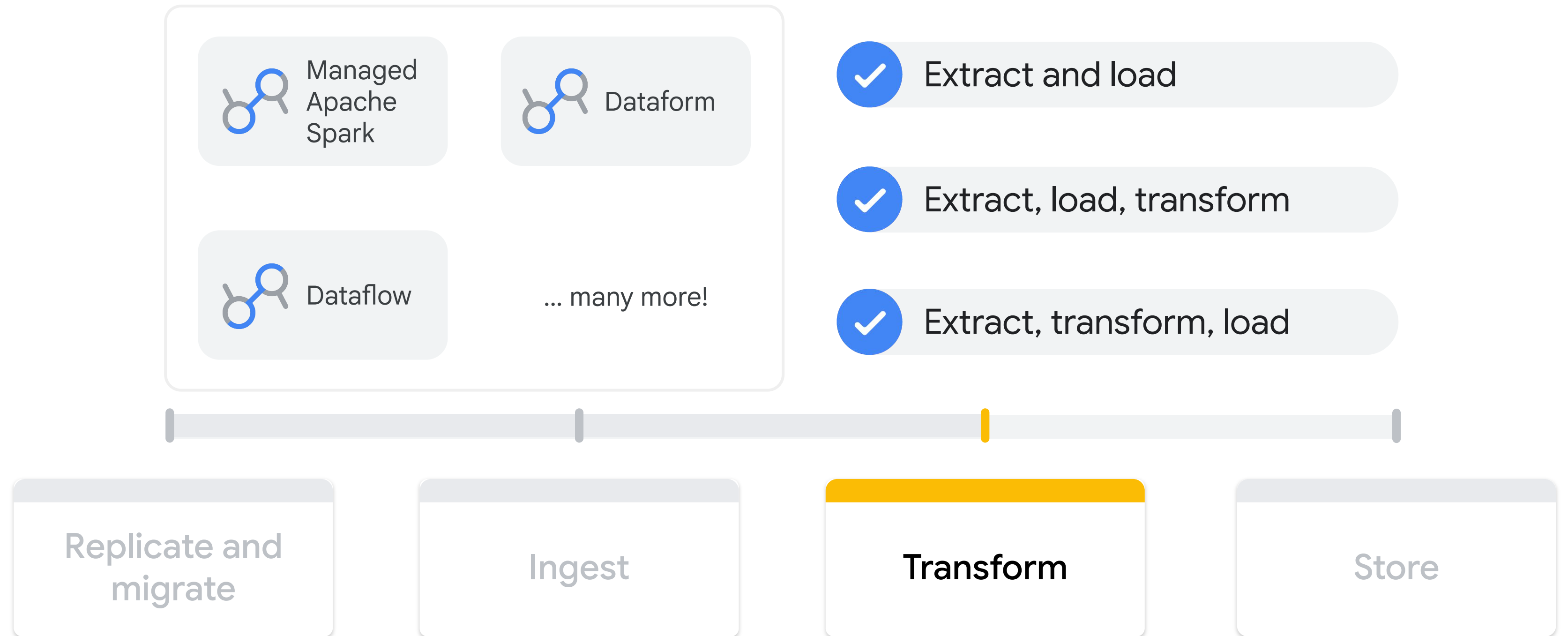
Replication and migration services onboard your data into Google Cloud



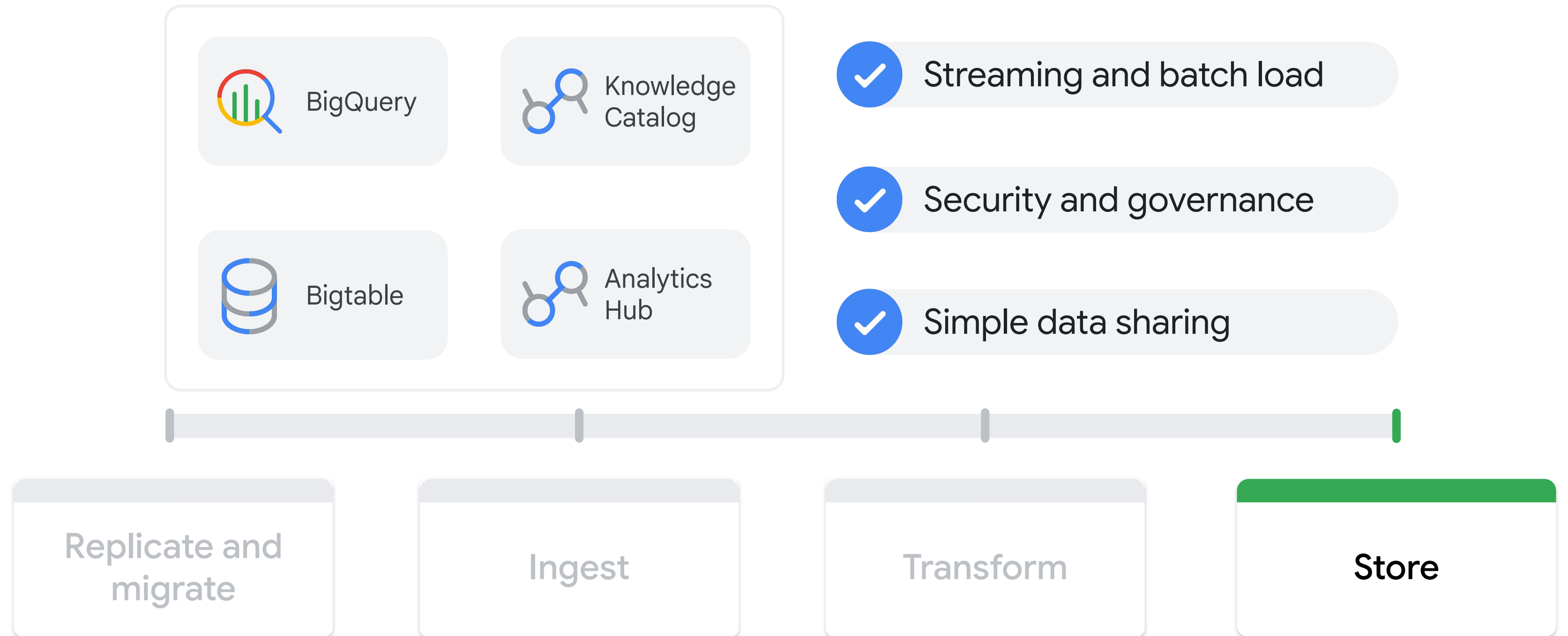
Data sources are the origin point of your raw data on Google Cloud



Transformation services add new value to your data



Data sinks store your processed data on Google Cloud



Data can have different formats

Storage/process data

✓ Data type

✓ Business need

Unstructured



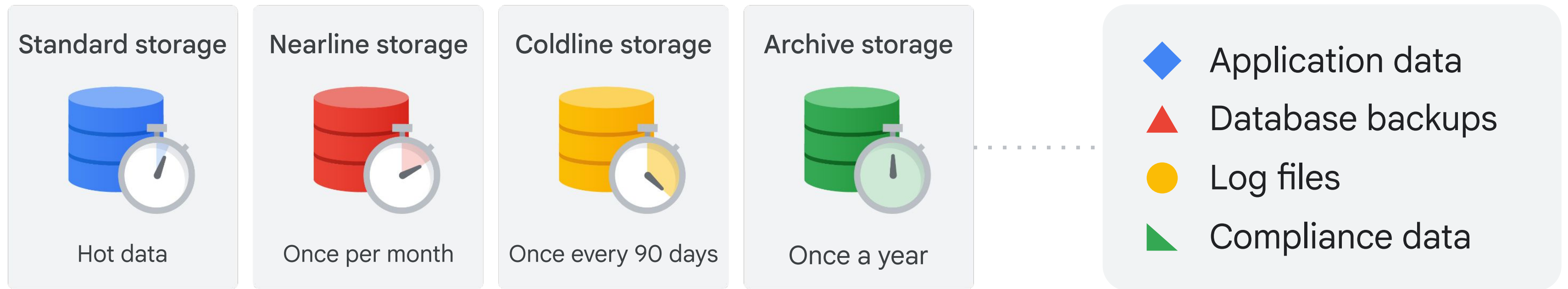
- ◆ Documents
- ▲ Images
- Audio files

Structured



- ◆ Tables
 - └ Rows
 - └ Columns

Cloud Storage holds your unstructured data



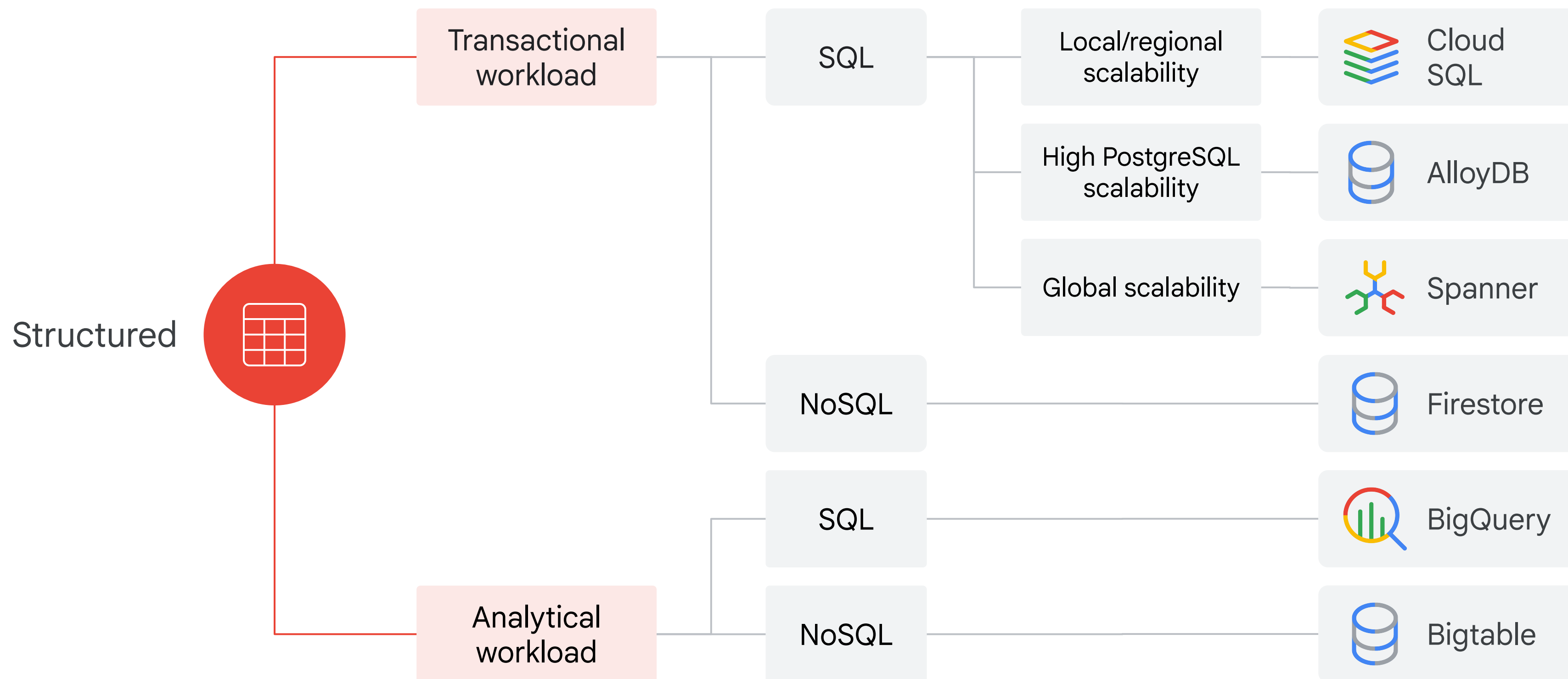
✓ Reliability and scalability

✓ Accessed by HTTP request

✓ Retrieved by object name

✓ Max object size of 5 TB

Options for storing structured data



Data lake versus data warehouse

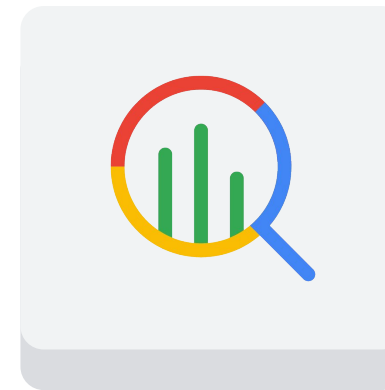
	Data lake	Data warehouse
Data format	Native (unstructured, semi-structured, structured)	Schema (structured or semi-structured)
Data type	Raw	Pre-processed and aggregated from multiple data sources
Purpose	Data science, applications, business decisions	Long-term business analysis
Dependencies	Tools/processes to enable data discovery, governance, security, metadata management	Standalone
Service	 Cloud Storage	 BigQuery

BigQuery is a serverless, fully managed data warehouse

Security on dataset, table, column and row level

Built-in **machine learning** and **geographic information system**

Scalable storage and analytics services

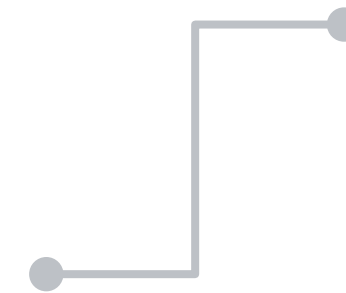


BigQuery

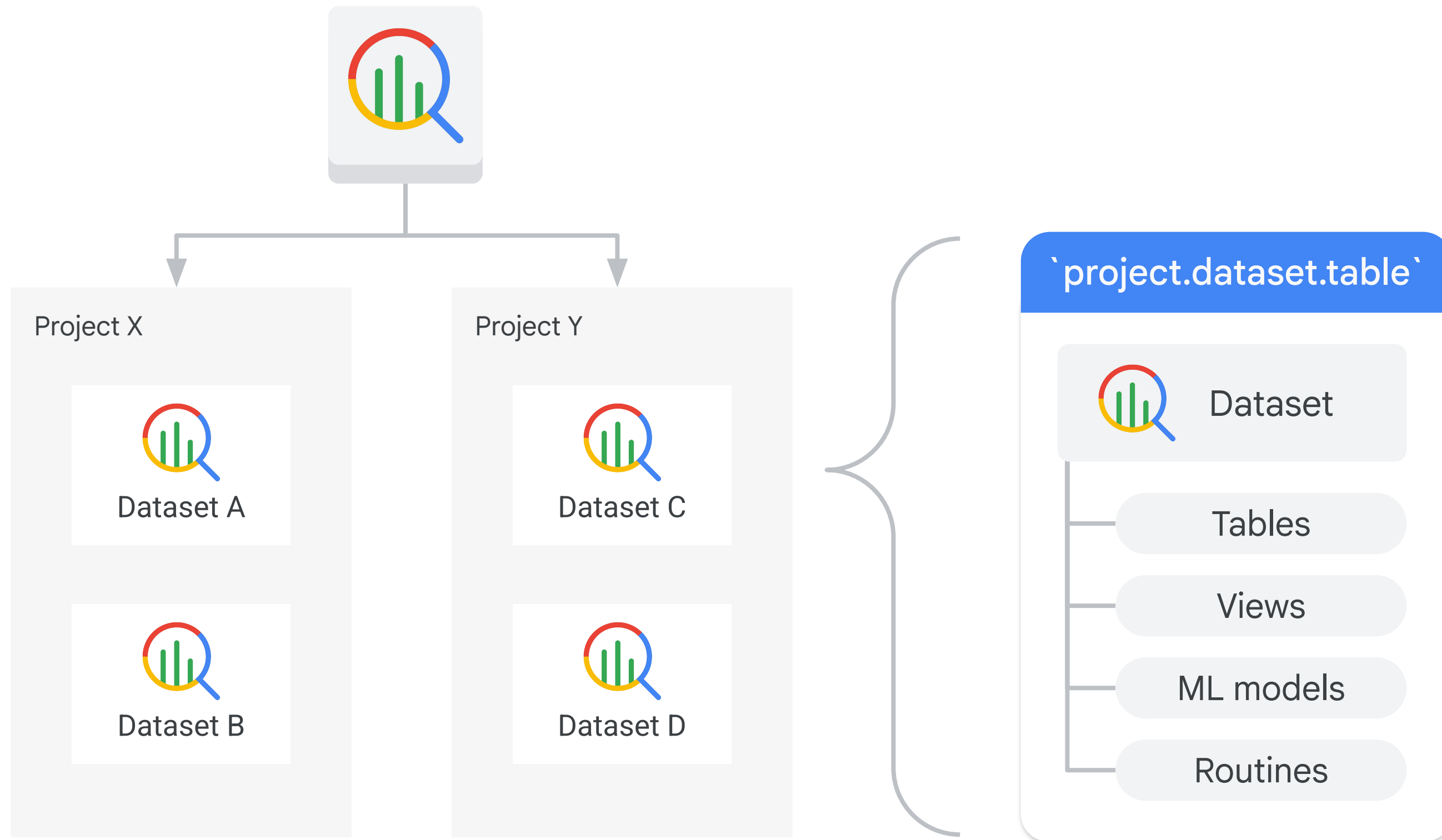
Rich ecosystem for data transformations

Integration with other storage services

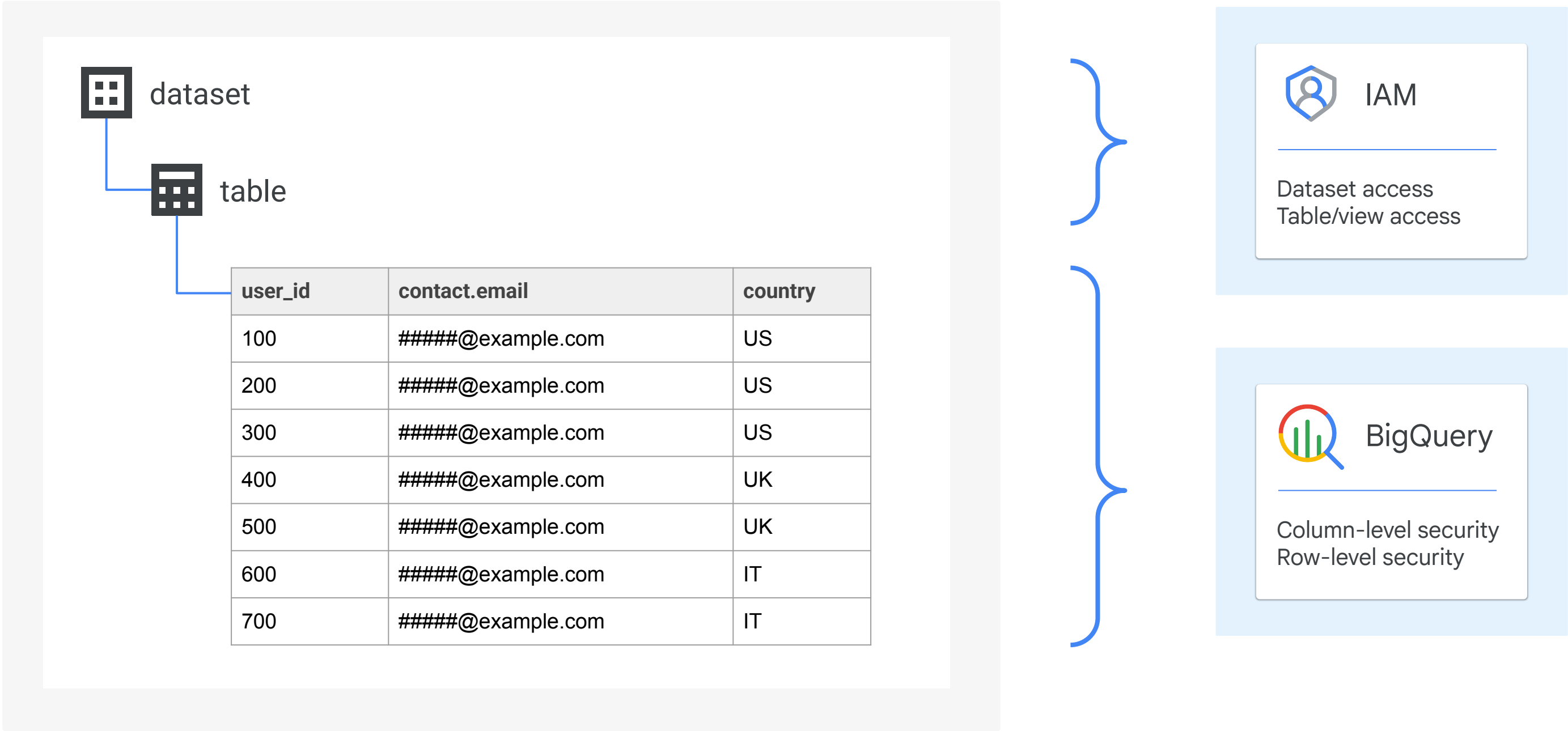
Real-time analytics on streaming data



How BigQuery's resources are organized

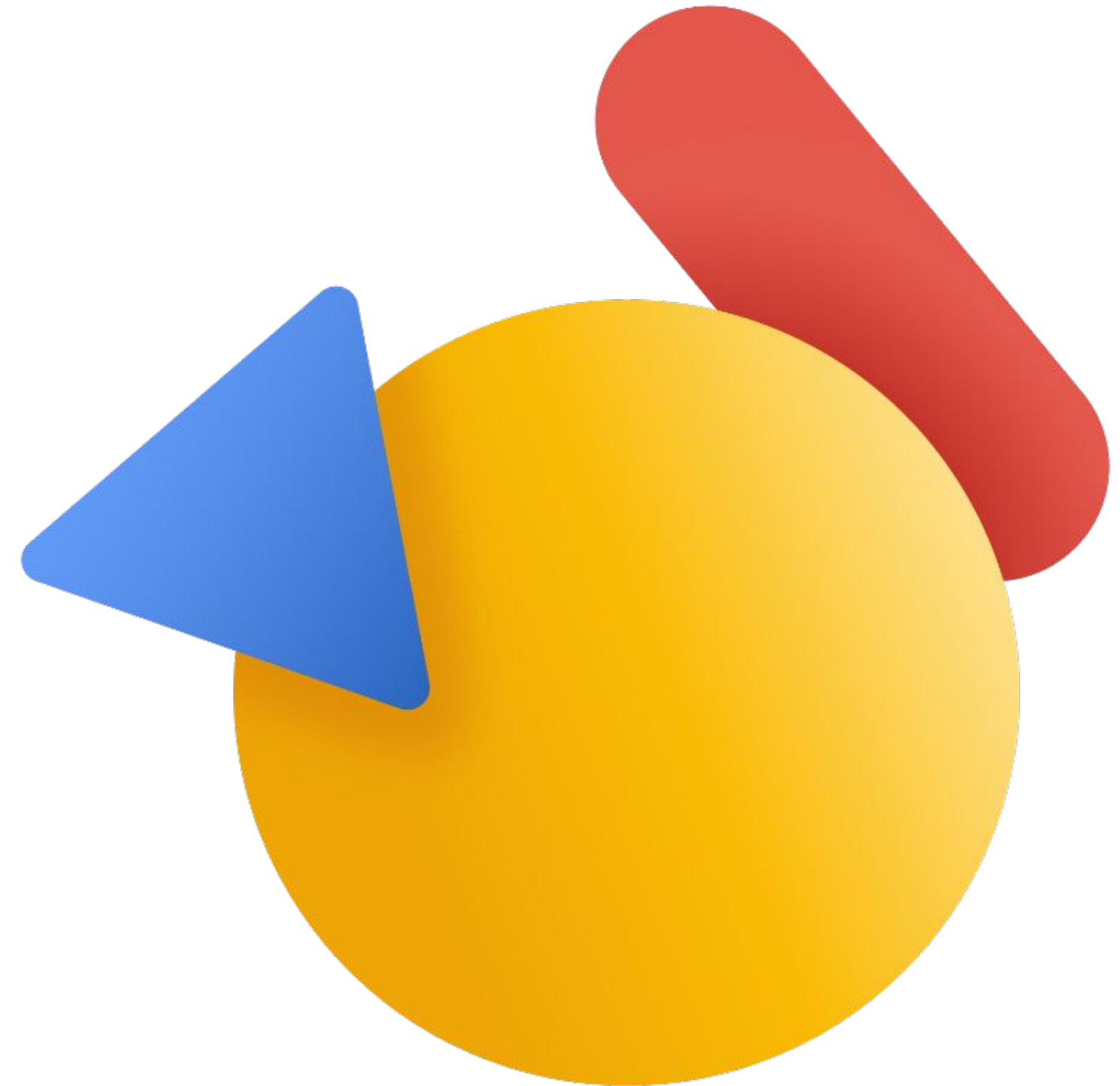


You can secure your BigQuery resources on multiple levels



Demo

Query terabytes of data in seconds



DIN: Exploring projects, datasets and tables

15 minutes · Google Cloud console — your lab project

1. Open the BigQuery console and star the bigquery-public-data and roi-bq-demos projects
2. Create a dataset named class in your lab project — every afternoon loading activity writes into it
3. Browse a public table: preview rows, inspect the schema, find the table details
4. Notice where you are: project → dataset → table, and the region you picked at creation

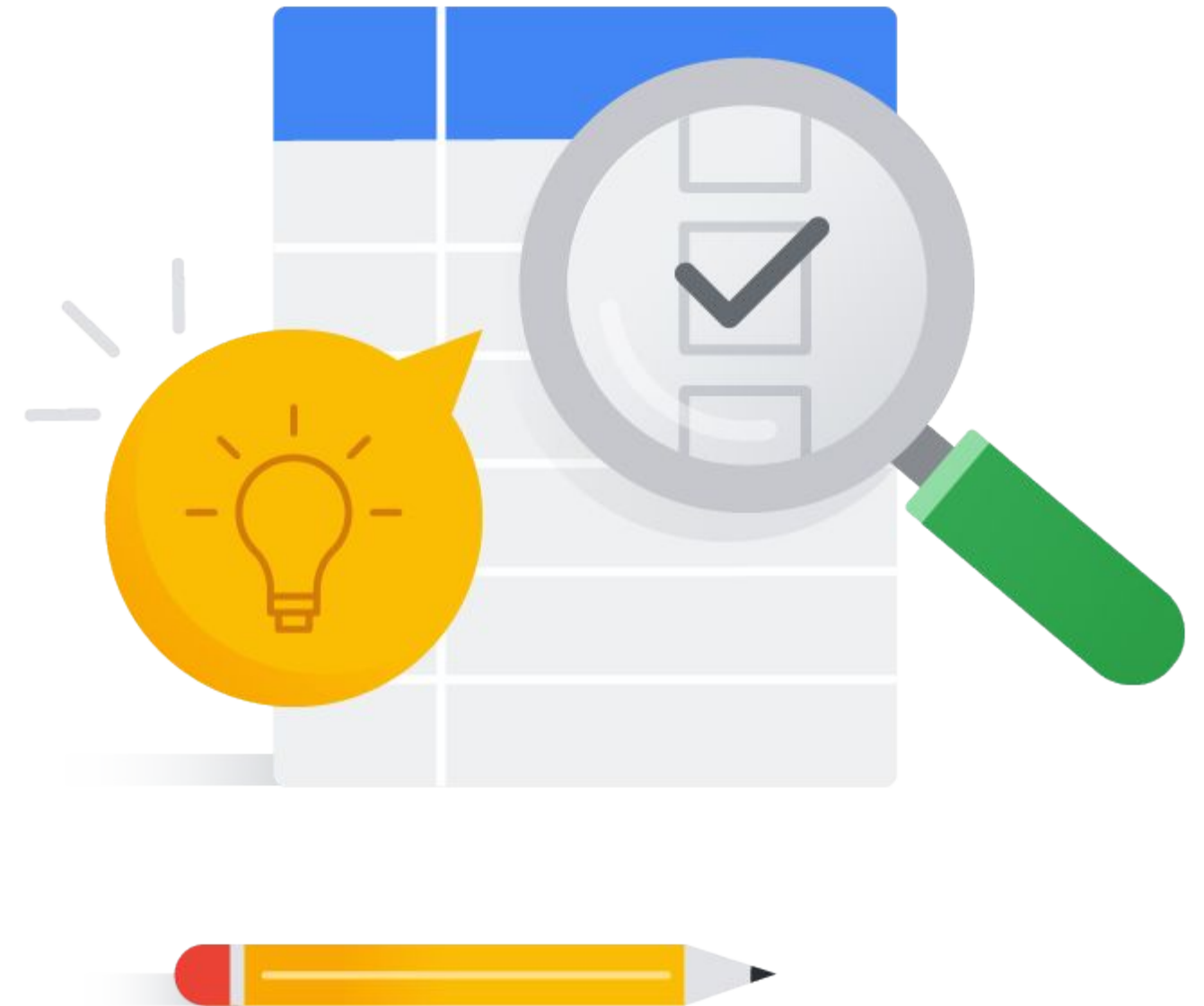
Why it matters — near-zero console experience is the only real gap in this room — this is the 15 minutes that closes it, and the class dataset is required by everything after lunch.

Lab: Loading Data into BigQuery

🕒 30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



M1 — What you now have

- ✓ The pipeline shape you know— ingest, transform, store — with Google service names on each box
- ✓ A mental map of the console: projects, datasets, tables, IAM and billing at the project level
- ✓ Your class dataset created— the landing zone for this afternoon's four loading activities
- ✓ Next: killing the nightly extracts— Datastream and change data capture (M2)

Exploring projects, datasets & tables (console orientation)

15 minutes — 10:10–10:25 · BigQuery console — instructor leads, you follow

1. Open BigQuery from the nav menu and pin it — check the project picker shows your lab project
2. Star three projects: bigquery-samples, bigquery-public-data, roi-bq-demos
3. Expand bigquery-samples → wikipedia_benchmark and read a dataset's metadata
4. Create your own dataset named exactly class (your project's three-dot menu → Create dataset)
5. Open the 100B table — Schema, Details, Preview: 100 billion rows, no query, no cost

Why it matters — project → dataset → table is the same skeleton as instance → database → schema.table — and class is the landing zone for every load this afternoon.

Morning Break

15 minutes · 10:25–10:40

Lab — Loading Data into BigQuery

30 minutes — 10:40–11:10 · Google Skills classroom — Course 1, Module 1 lab

1. Launch the lab from the Skills classroom — lab timer running, lab credentials, right project
2. Load data into BigQuery from various sources through the Cloud console
3. Repeat a load from Cloud Shell with the bq command-line tool
4. Create and load a table with DDL — load logic as code
5. Early? DIN §24: load a CSV from jwd-gcp-demos into your class dataset — note the load time

Why it matters — the simplest version of what your nightly extract jobs do — a file becomes a queryable table. Deliberately easy: a first win in an unfamiliar console.

WATCH FIRST · 2:11 · GOOGLE CLOUD TECH

Replicate MySQL → BigQuery with Datastream

- ✓ A live change-data-capture stream, configured in two minutes: backfill once, then every INSERT, UPDATE, DELETE lands in BigQuery.
- ✓ MySQL in the demo, SQL Server in your world — same log-based capture, same console. Your 11:50 lab is the PostgreSQL twin of exactly this.



Scan to open on YouTube

youtu.be/vkCgqUILeeU

M2 — What this block gets you

- ✓ Replication vs migration— backfill once, then a live change stream; Datastream does both as one operation
- ✓ Change data capture (CDC) as a managed service— every INSERT, UPDATE, DELETE lands in BigQuery
- ✓ SQL Server is a supported Datastream source— your SQL Server → BigQuery path is first-class
- ✓ Log-based capture— no triggers, no polling, no production load (the answer for your M&O team)
- ✓ Hands-on: start a Datastream stream before lunch, validate the replication after it

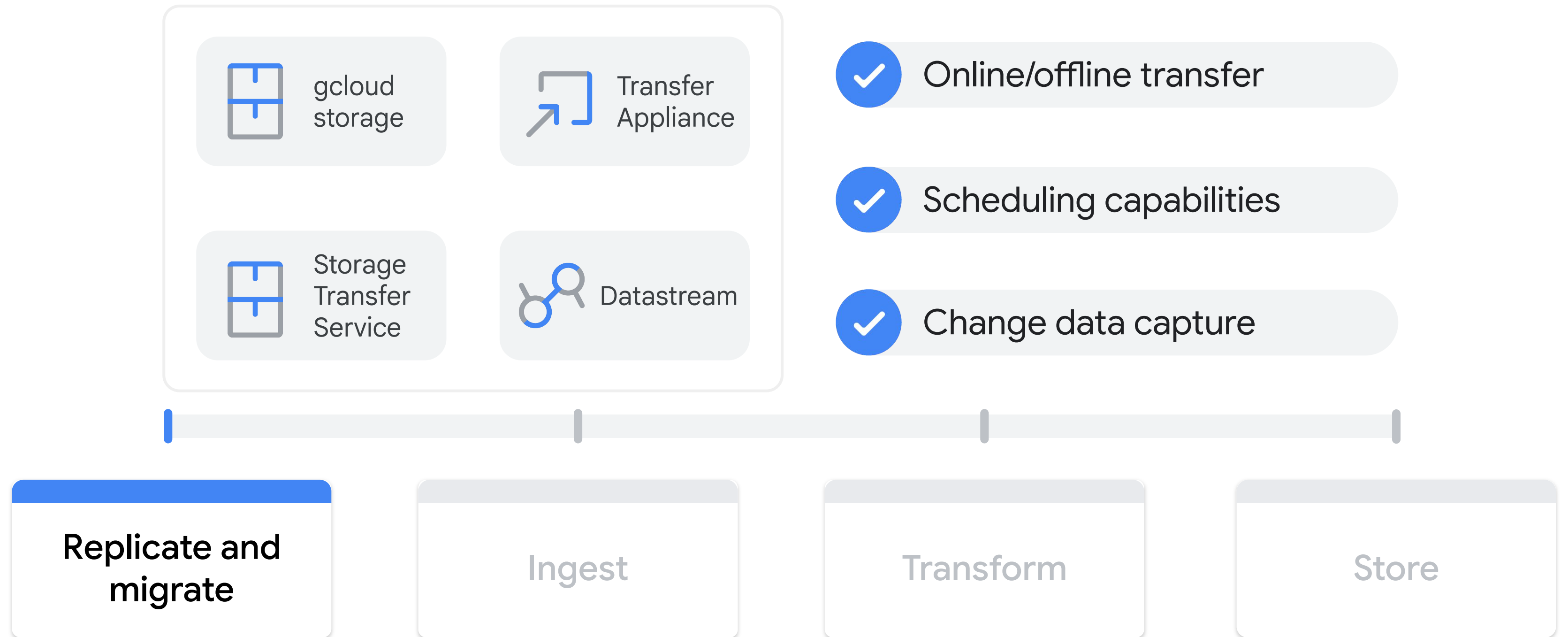
This block, across the lunch break

- | | | |
|-------------|---|-----|
| 11:10–11:50 | M2 — Data replication & migration: killing the nightly extracts | |
| 11:50–12:00 | Lab: Datastream PostgreSQL → BigQuery, part 1 — set up and start the stream | LAB |
| 12:00–13:00 | Lunch — the stream replicates while we eat | |
| 13:00–13:15 | Lab part 2 + CDC demo — validate backfill, watch a live change land | LAB |
| 13:15–13:45 | M3 — Extract & Load: the pattern for data that still arrives as files | |

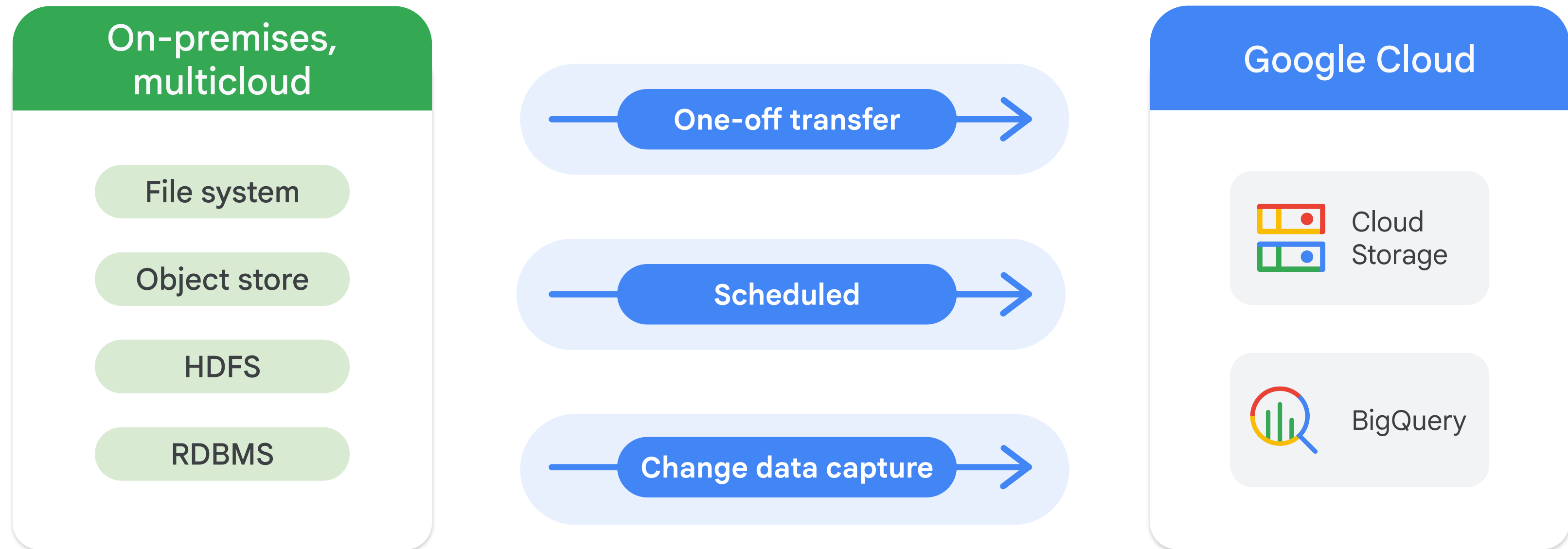


Data replication and migration

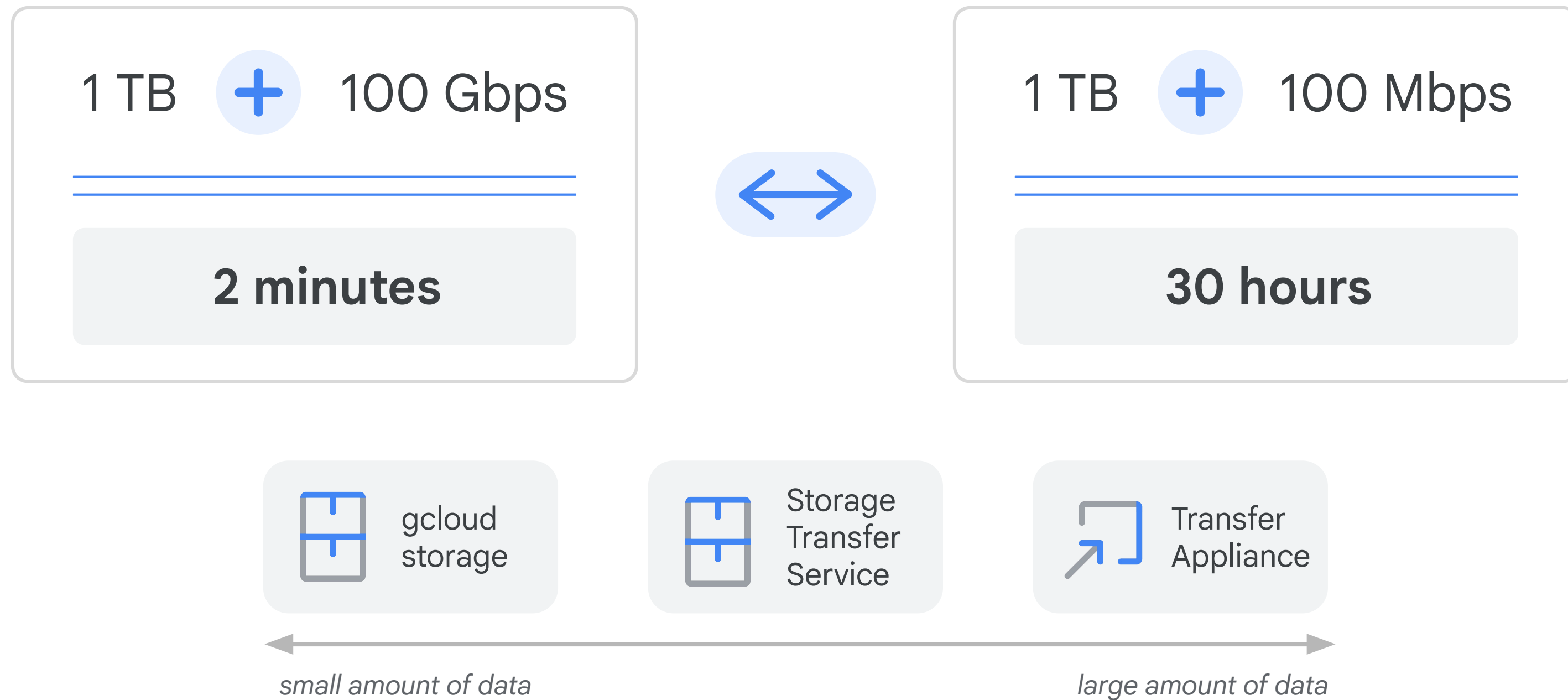
Replication and migration services onboard your data into Google Cloud



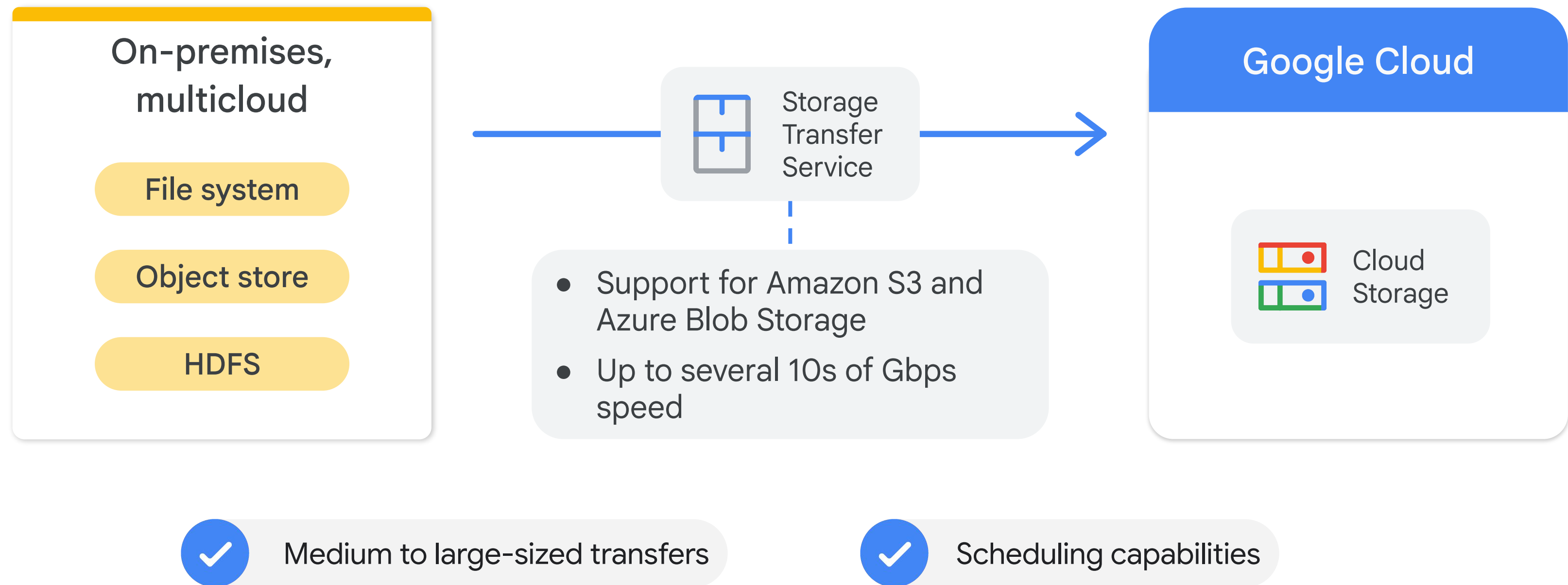
Common replication and migration scenarios for ingesting your data into Google Cloud



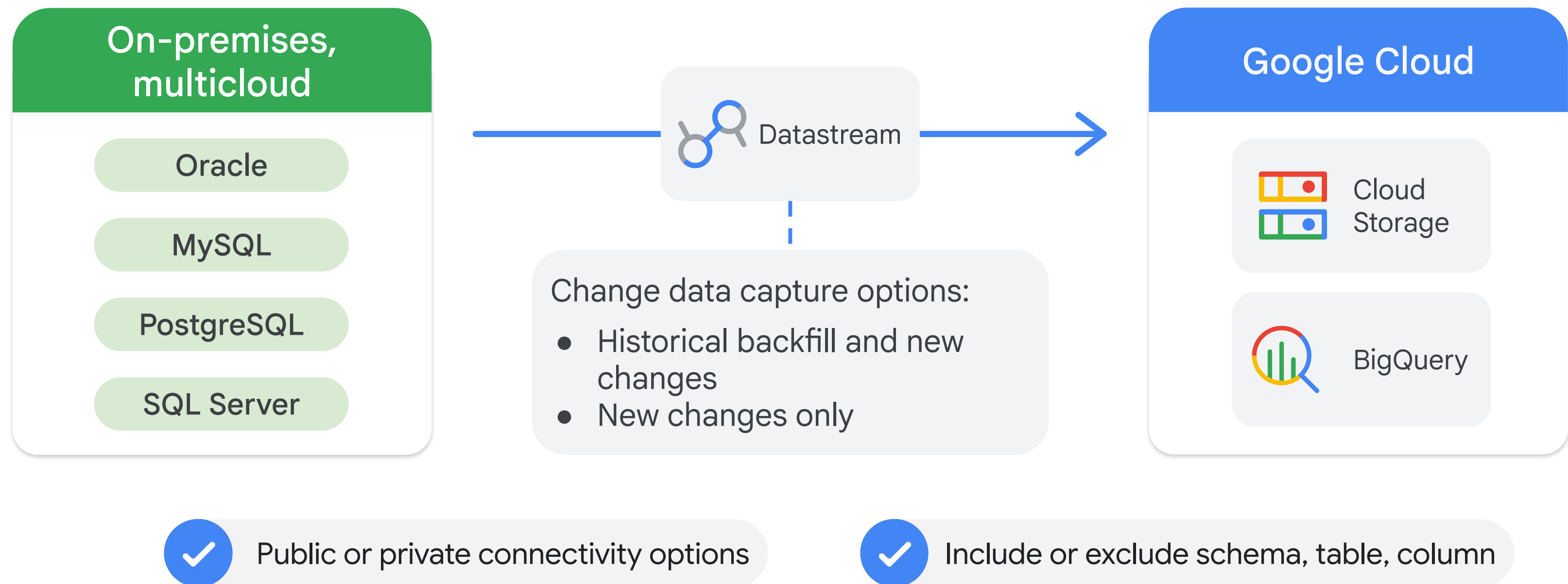
Choosing between migration options depends on your amount of data and network bandwidth



Move large amounts of data with Storage Transfer Service



Datastream continuously replicates your RDBMS into Google Cloud for analytics



SQL Server IS a supported Datastream source

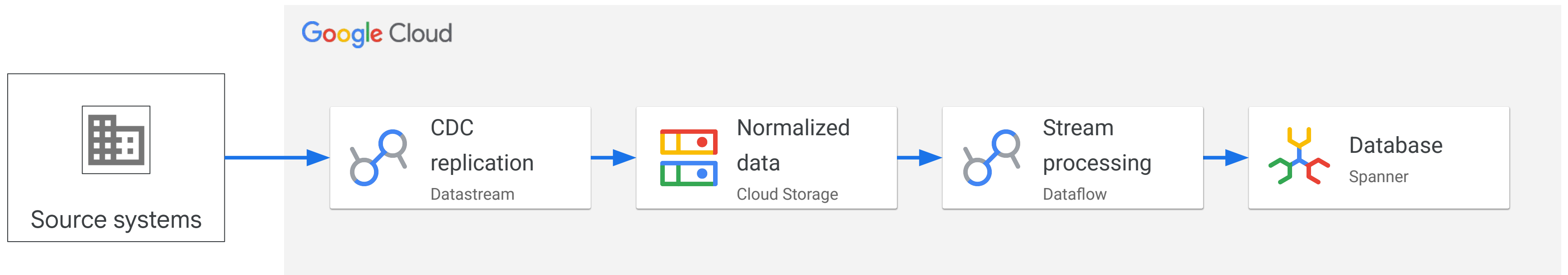
The deck's source graphic is the short list — SQL Server is fully supported.

- Self-managed or cloud-hosted SQL Server— supported alongside Oracle, MySQL and PostgreSQL
- Captured from the SQL Server transaction logs— the same log your databases already write
- Your SQL Server → BigQuery replication is a first-class Datastream scenario, not a workaround
- Today's lab uses PostgreSQL only because it is free-tier friendly— the mechanics are identical
- This lab is your project's pilot: source database → continuous changes → BigQuery

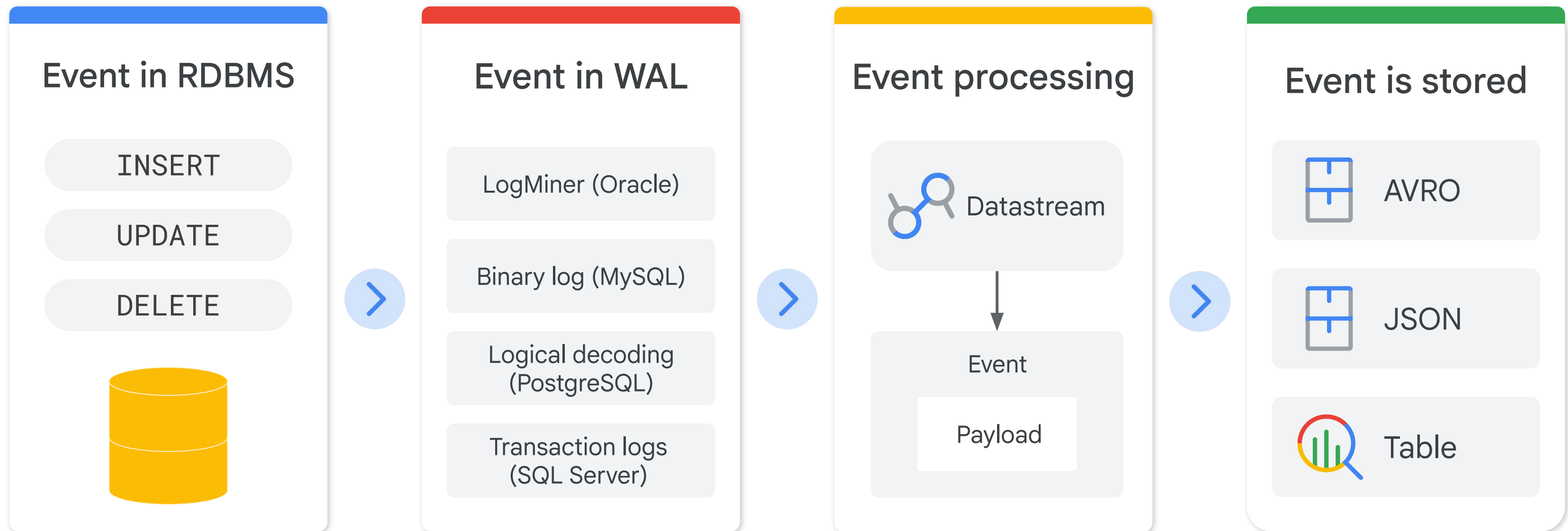
If the room takes one sentence from this module: Datastream supports SQL Server, and that is your path.

Datastream use cases

- Analytics with database replication into BigQuery
- Analytics with custom data processing in Dataflow into BigQuery
- Data processing using an event-driven architecture
- **Example:** database replication and migration using Dataflow templates



Datastream uses the database's write-ahead log (WAL) to process change events



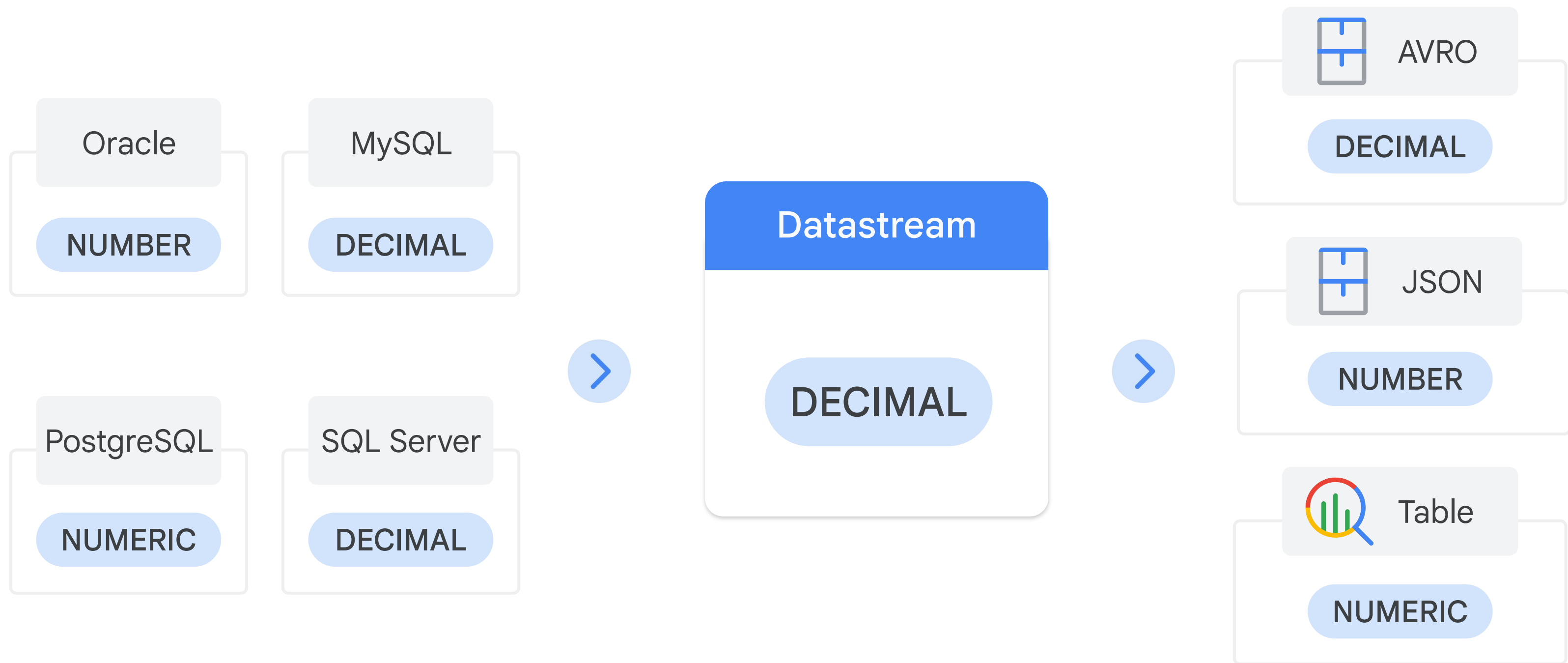
Log-based CDC — the answer for your M&O team

Datastream reads the write-ahead log / transaction log the database is already writing.

- No triggers— nothing is added to your tables or to the write path
- No polling— Datastream never runs change-detection queries against production
- No batch window— there is no 1 AM–4 AM slot where the source gets hammered
- The source keeps serving its workload— capture reads a log, not the tables
- Deletes arrive too— `change_type` and `is_deleted` metadata on every replicated row

Pre-empt the objection before M&O raises it: replication adds no triggers, no polling, no production load.

Datastream uses unified types to map source to destination data types



Compare migration and replication options

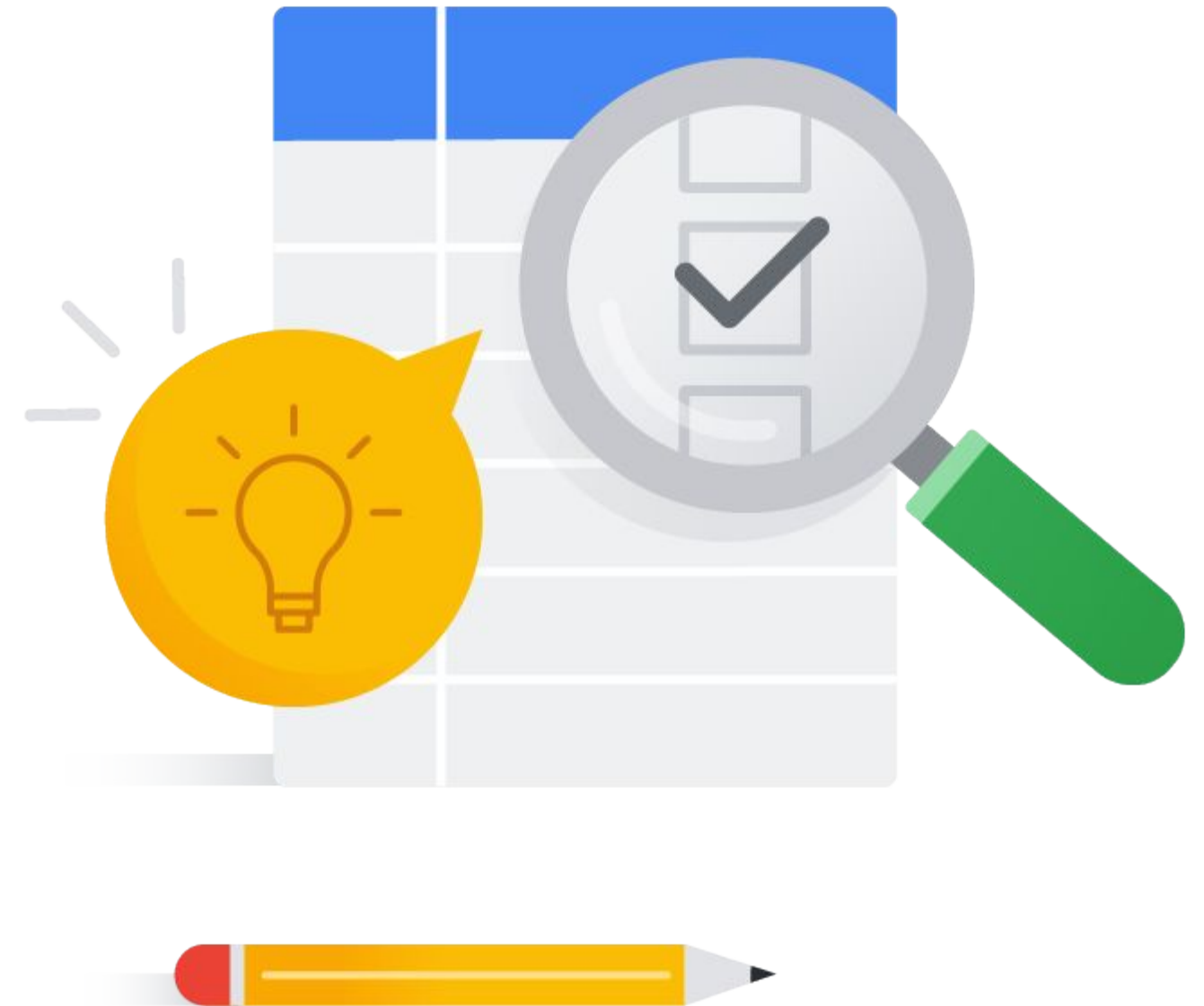
	gcloud storage	Storage Transfer Service	Transfer Appliance	Datastream
Transfer type	online	online	offline	online
Source locations	On-premises, Google Cloud	On-premises, Google Cloud, multicloud	On-premises	On-premises, Google Cloud, multicloud
Data range	less than 1 TB recommended	more than 1 TB recommended	7 TB, 40 TB, 300 TB	10,000 tables per stream
Velocity	Batch	Batch (hourly at minimum)	Batch	Batch and Stream
Data format	any	any	any	structured

Lab (optional): Datastream: PostgreSQL Replication to BigQuery

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Lab part 1 — Datastream: PostgreSQL to BigQuery (setup & start)

10 minutes — 11:50–12:00, then lunch while it runs · Google Skills classroom — your pre-warmed lab project

1. Open the module 2 lab — the Cloud SQL for PostgreSQL instance is pre-warmed
2. Follow the instructor click-for-click: create the connection profile for the PostgreSQL source
3. Create the connection profile for the BigQuery destination — regions must match
4. Create the stream and start replication — hands up when stream status shows Running
5. Leave everything running and break for lunch — do not pause or clean up

Why it matters — the stream backfills and replicates while we eat — at 13:00 you validate what landed. PostgreSQL is here because it is free-tier friendly; the mechanics are identical for SQL Server.

Lab part 2 — validate backfill and changes (after lunch)

15 minutes — 13:00–13:15 · BigQuery console + your lab's Cloud SQL instance

1. Open the BigQuery tables the stream created — destination tables appear as the stream runs
2. Validate the backfill: row counts vs the source, eyeball the data
3. Inspect the Datastream metadata columns — `change_type` and `is_deleted` on every row
4. Apply the lab's source change (or your own UPDATE/INSERT) against Cloud SQL
5. Poll the BigQuery table until the change lands — expect roughly a minute

Why it matters — one-time backfill, then incremental changes forever — this is the end of nightly truncate-and-reload for your largest tables.

Demo — CDC validation: a change lands while we watch

Inside lab part 2 — 13:00–13:15 · Instructor screen — Cloud SQL source to BigQuery destination

1. Run an UPDATE and an INSERT against the source Cloud SQL instance
2. Watch the Datastream stream status and throughput tick in the console
3. Poll the BigQuery destination table until the changed rows appear
4. Read the metadata on the new rows — `change_type` tells insert from update
5. Note the latency: source commit to BigQuery in about a minute, no job ran

Why it matters — your data changed while we ate lunch — no nightly job, no extract window, no SSIS package ran. That is the whole reason you are migrating.

M2 — What you now have

- ✓ The distinction that frames your project— one-time backfill, then a continuous change stream, in one operation
- ✓ SQL Server confirmed as a supported Datastream source— your migration path is first-class
- ✓ The M&O answer in your pocket— log-based CDC: no triggers, no polling, no production load
- ✓ A stream you started before lunch and validated after— backfill plus a live change, seen end to end
- ✓ Next: data that still arrives as files— the Extract & Load pattern (M3, 13:15)

Lab — Datastream: PostgreSQL → BigQuery (split across lunch)

10 min setup 11:50–12:00 + 15 min validation 13:00–13:15 · Google Skills classroom — Course 1, Module 2 lab

1. Part 1 (11:50): prepare the Cloud SQL for PostgreSQL source and import the seed data
2. Create two connection profiles — PostgreSQL source, BigQuery destination (same region!)
3. Create the stream and start it — then go to lunch; it backfills and replicates while you eat
4. Part 2 (13:00): validate the backfill — row counts, and the `change_type` / `is_deleted` columns
5. UPDATE/INSERT a row in the source and watch it land in BigQuery in about a minute — no job ran

Why it matters — your data changed while you ate lunch — no nightly job, no extract window. PostgreSQL is free-tier friendly here; the mechanics are identical for SQL Server, a supported Datastream source.

Lunch

60 minutes · 12:00–13:00

the Datastream stream keeps replicating while we eat

WATCH FIRST · 3:24 · GOOGLE CLOUD TECH

What is Storage Transfer Service?

- ✓ The managed mover for bulk files — bucket-to-bucket, from S3, or from your on-prem file shares — no scripts, no transfer VM to babysit.
- ✓ This afternoon you load files by hand; this is what you hand the historical archive to when 'by hand' means 40 TB.



Scan to open on YouTube

youtu.be/1olw2mT_4-M

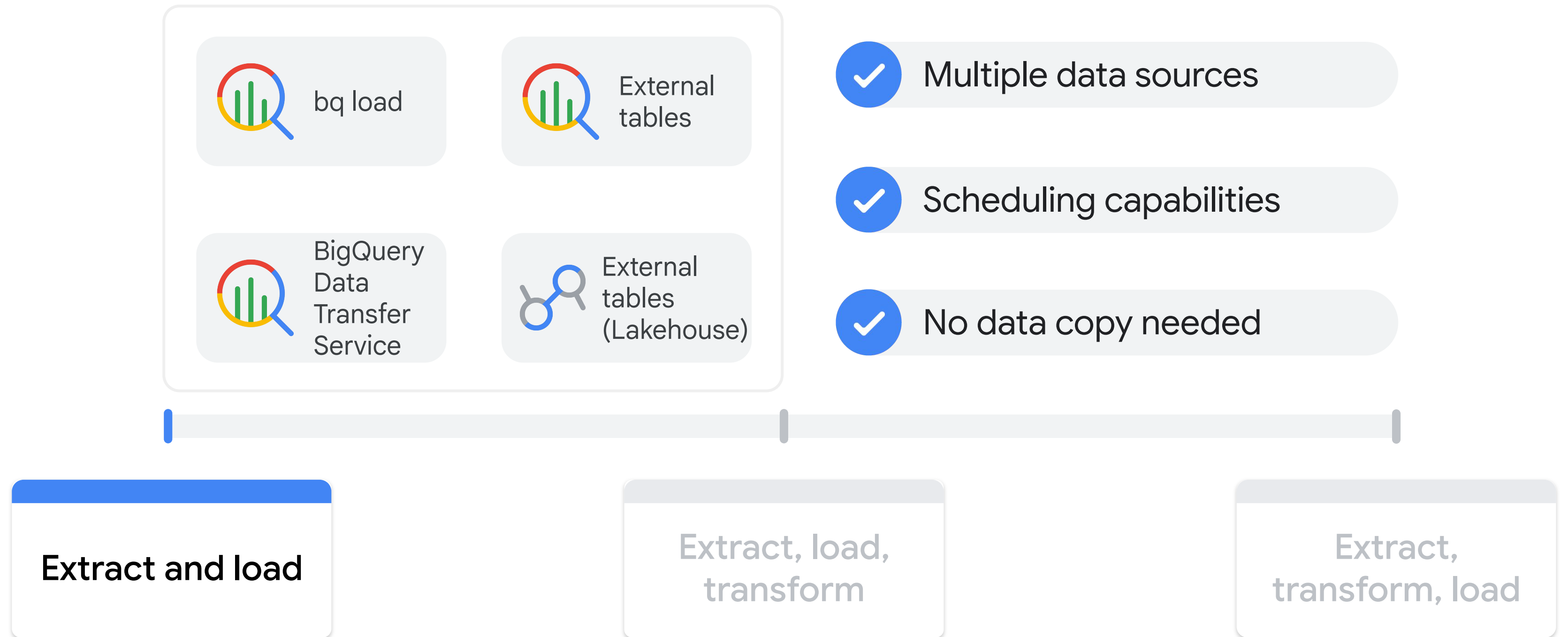
M3 — What this block gets you

- ✓ Extract & Load = your Bronze-landing pattern for files— land the file in BigQuery as-is, transform later
- ✓ One load job, three front doors— console form, bq load, and LOAD DATA SQL; batch loads are all free
- ✓ Autodetect is a first look, not a pipeline— pinned schemas in source control for anything recurring
- ✓ Data Transfer Service = scheduled ingest— the managed answer to your SQL Agent import jobs
- ✓ BigLake / external tables = query files where they live— the bridge to Day 2's lakehouse design

After lunch: land the files, 13:15–14:45

- 13:15–13:45** M3 Extract & Load — EL as Bronze landing; bq CLI; Data Transfer Service; BigLake / external tables
- 13:45–14:30** Loading block, four free DINs — console & CLI load → autodetect pitfall → CSV vs Parquet race → LOAD DATA SQL **DIN**
- 14:30–14:45** Break

Extract and load ingests data into BigQuery without the need for transformation



BigQuery supports importing from and exporting to multiple formats

Loading data

- Avro
- Parquet
- ORC
- CSV
- JSON
- Firestore export



Query results

- CSV
- JSON
- Google Sheets

Table export

- CSV
- JSON
- Avro
- Parquet

Load data into BigQuery by using the UI or the **LOAD DATA** statement

Create table from

Upload ▼

Select file **BROWSE**

File format

CSV ▼

Schema

☒ Auto detect

```
# create a new table by loading data from
# CSV files using schema auto-detect
LOAD DATA INTO dataset_name.table_name
FROM FILES(
  format='CSV',
  uris = ['gs://mybucket/*.csv']
)
```



Use **LOAD DATA INTO** to append to an existing table.

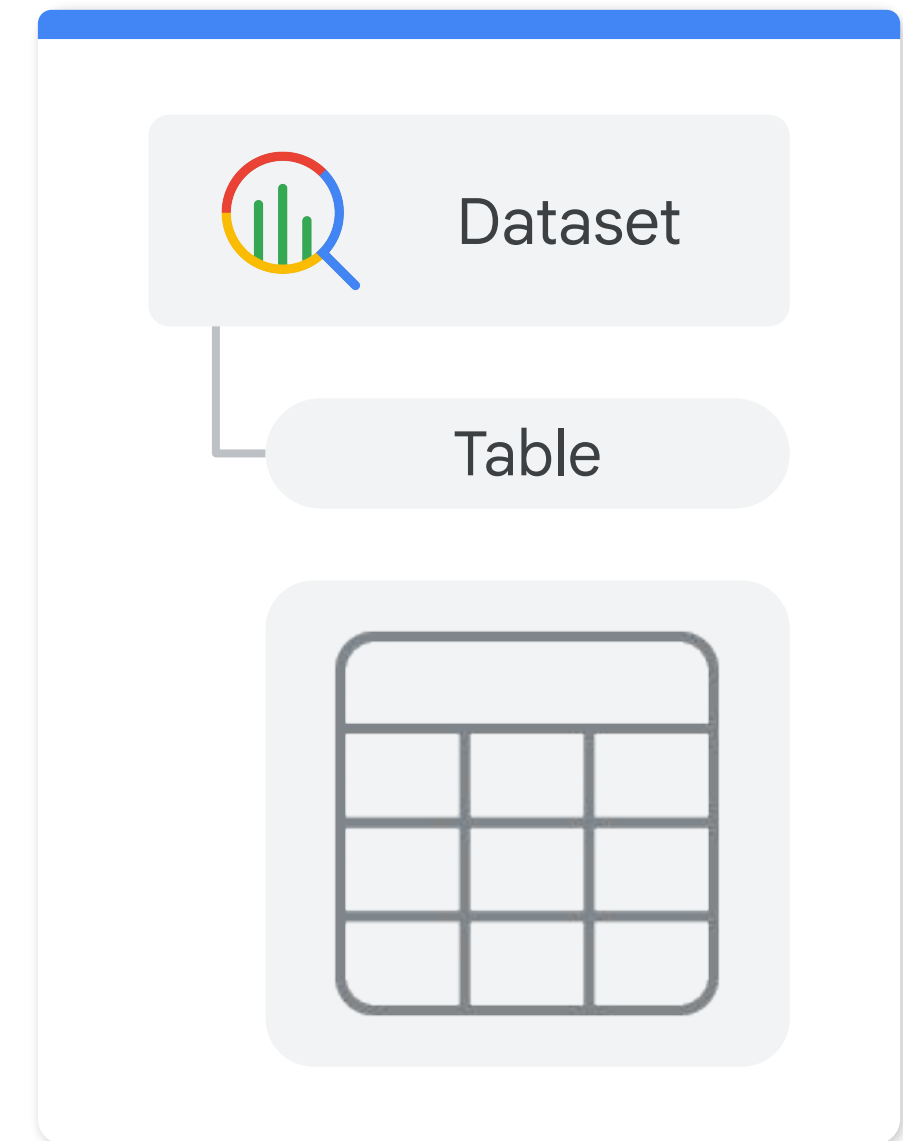
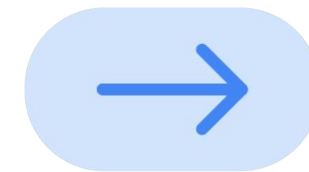


Use **LOAD DATA OVERWRITE** to overwrite the existing table.

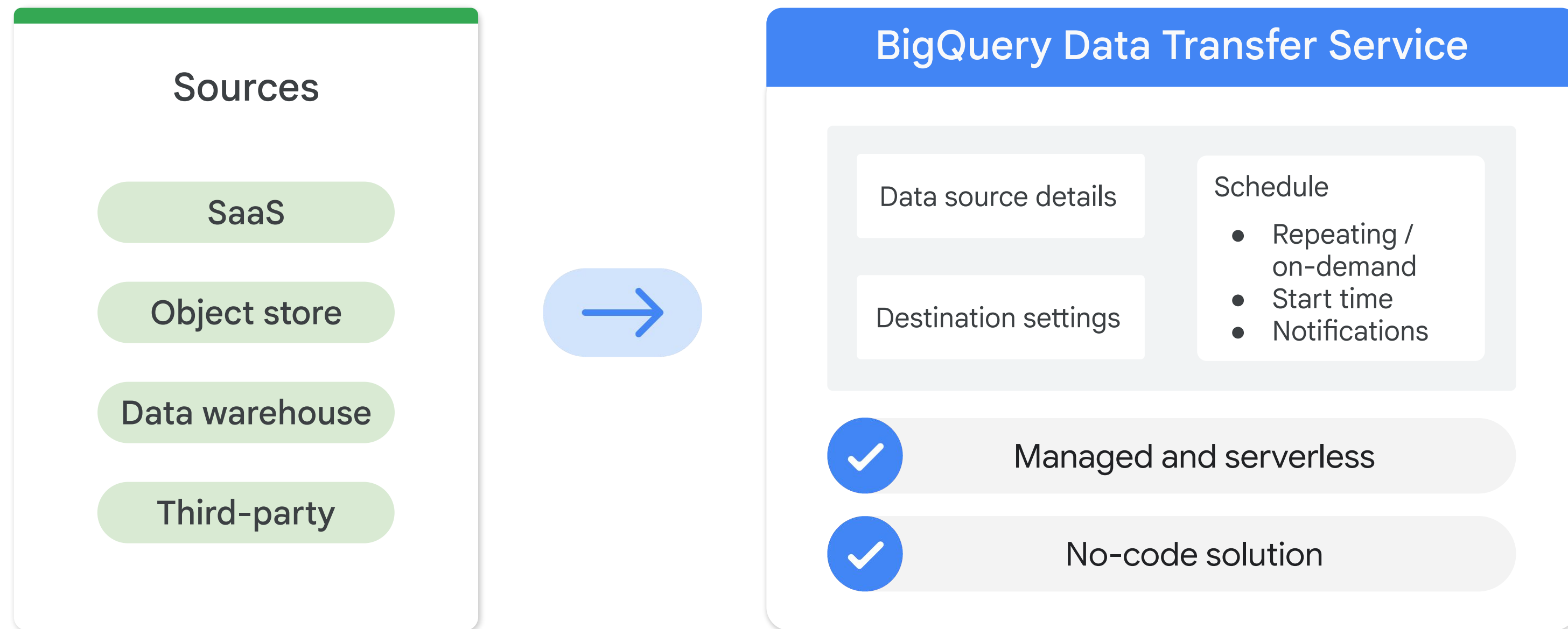
Interact with BigQuery by using the `bq` command

```
> bq mk --location=US -dataset dataset-name
```

```
> bq load \  
  --source_format=CSV \  
  --skip_leading_rows=2  
  dataset-name.table_name \  
  "gs://mybucket/00/*.csv", "gs://mybucket/01/*.csv" \  
  ./table_schema.json
```

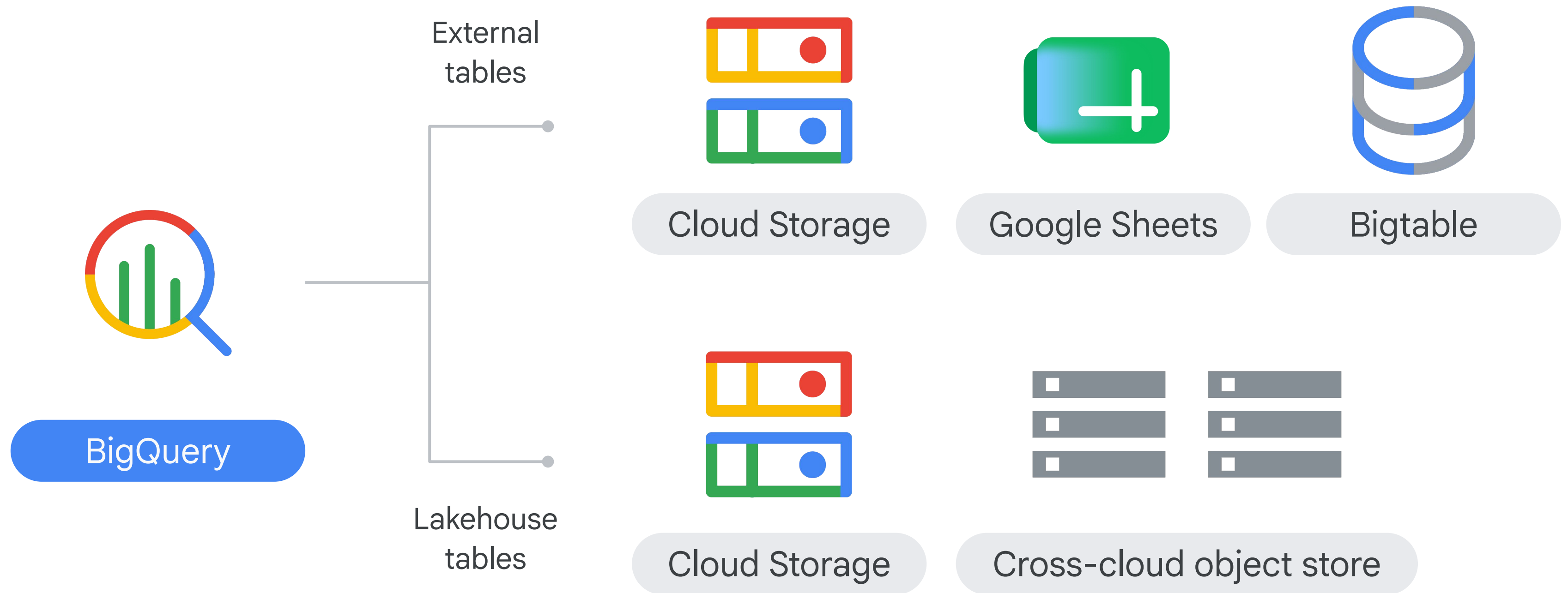


Use the BigQuery Data Transfer Service to load data from other structured data sources

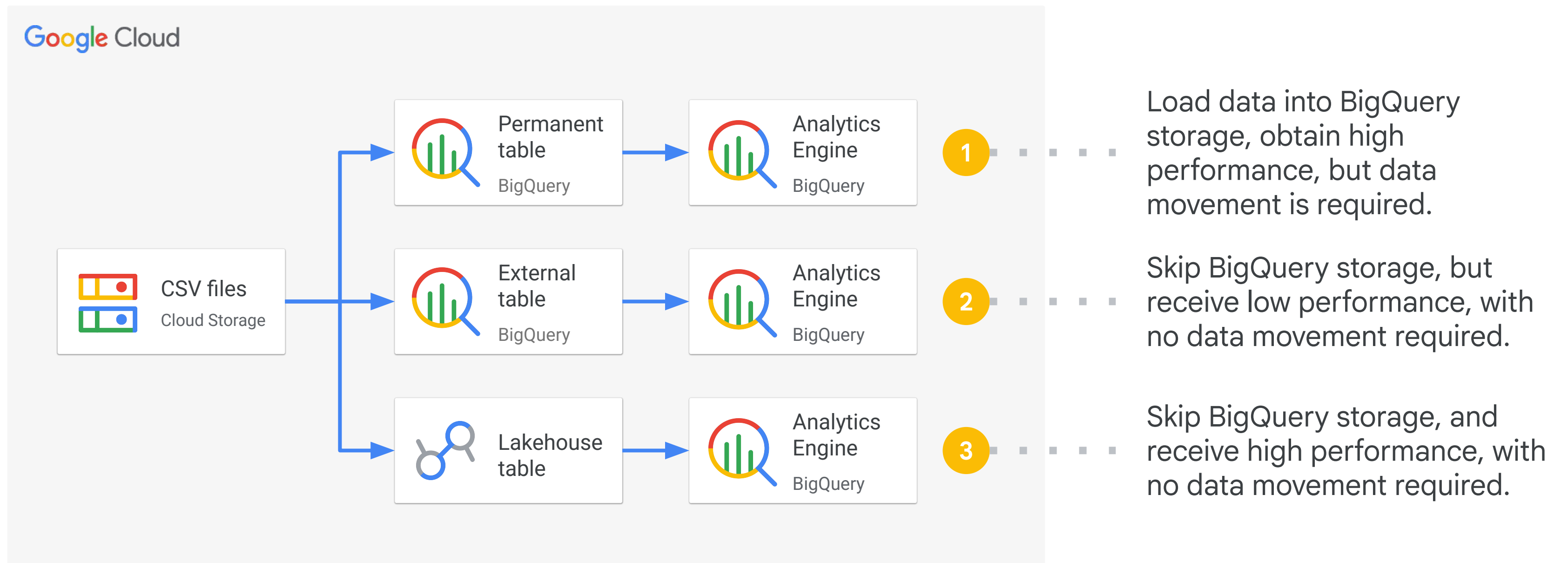


Your data isn't stored in BigQuery?

You can still query it!



Three ways of analyzing structured data in BigQuery



External tables let you query Google Sheets data directly in BigQuery

SCHEMADETAILSLINEAGE...

Table information

Table ID	your-project-id.sheets.sheets_table
...	

External data configuration

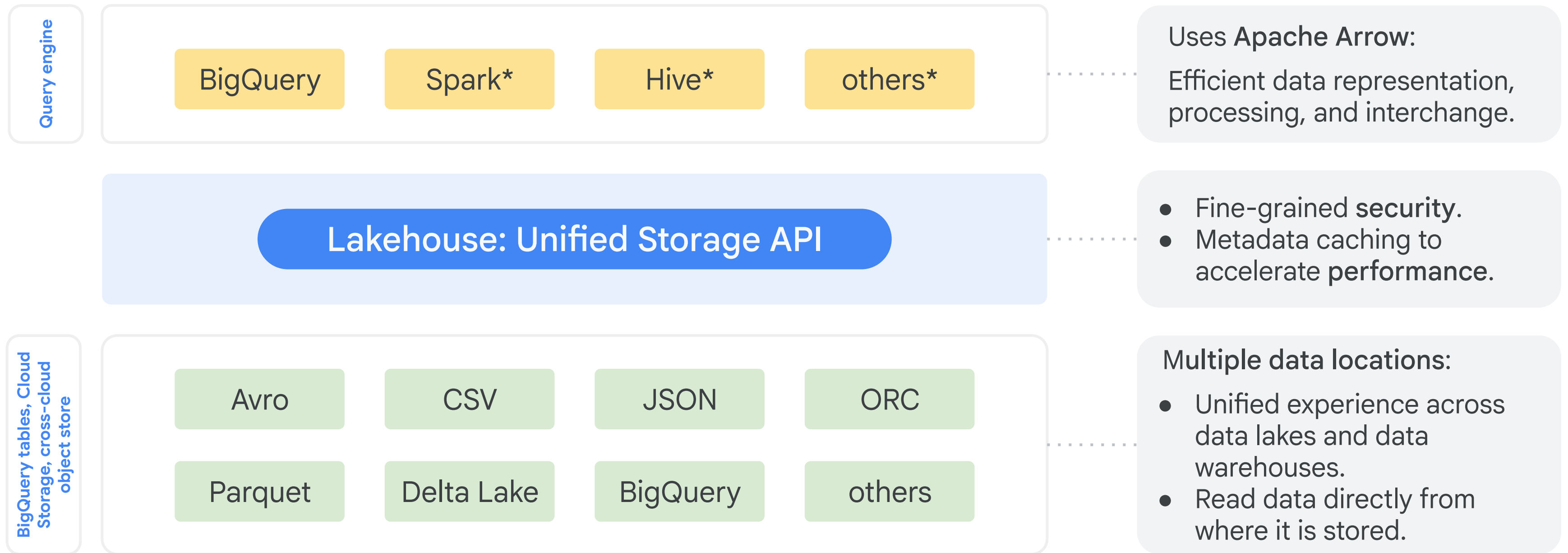
Source URI(s)	https://docs.google.com/spreadsheets/...
Source format	GOOGLE_SHEETS



```
# Query the Google Sheets table
SELECT ...
FROM
  `your-project-id.sheets.sheets_table`;
```

-  No query cost estimation, table preview, or query caching available.
-  Performance is slower than querying a permanent table in BigQuery.

Lakehouse: BigQuery on your data lake... and much more!



* using BigQuery connectors

A Lakehouse table behaves the same way as a permanent table

SCHEMADETAILSLINEAGE...

Table information

Table ID	your-project-id.biglake.csv_table
...	

External data configuration

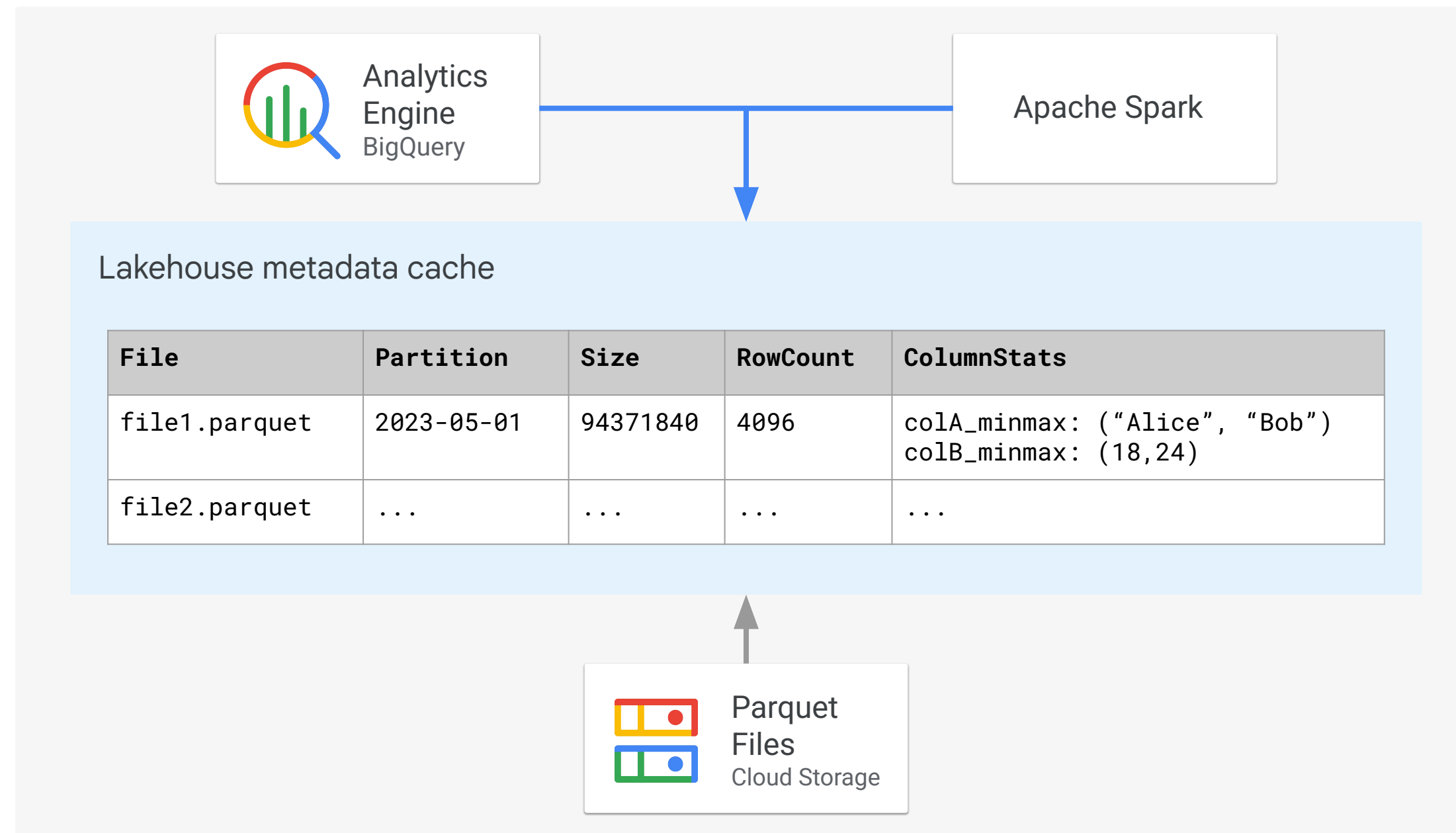
Source URI(s)	gs://your-bucket/your_file.csv
...	



```
# Query the Lakehouse table
SELECT ...
FROM
  `your-project-id.biglake.csv_table`;
```

- ✓ Use Lakehouse if loading data into BigQuery isn't an option for your use case.
- ✓ Queries are performant due to metadata caching.
- ✗ No query cost estimation or table preview.

Lakehouse uses metadata caching to accelerate performance



Advantages of BigQuery

- Skip object list
- Faster file and partition pruning
- Dynamic predicate pruning

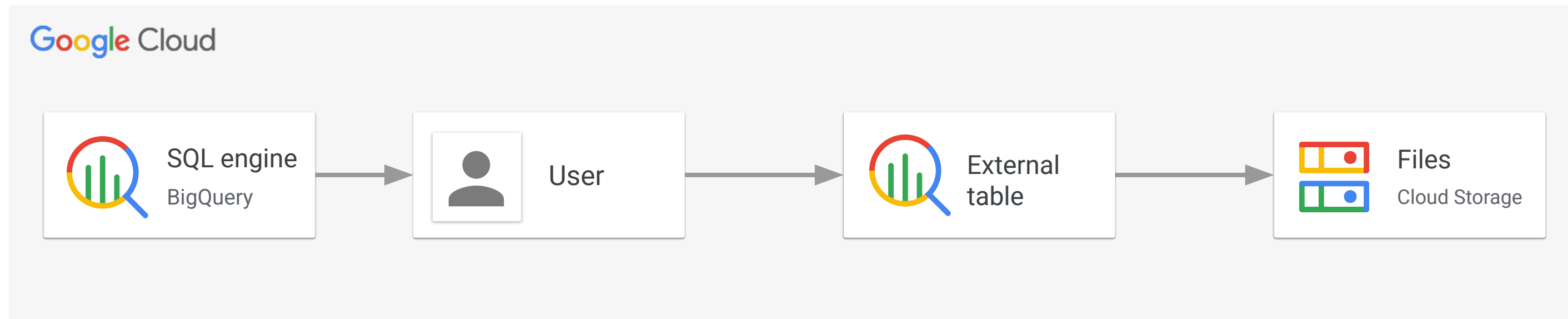
Advantages of Spark

- Can read metadata statistics using the spark-bigquery connector.

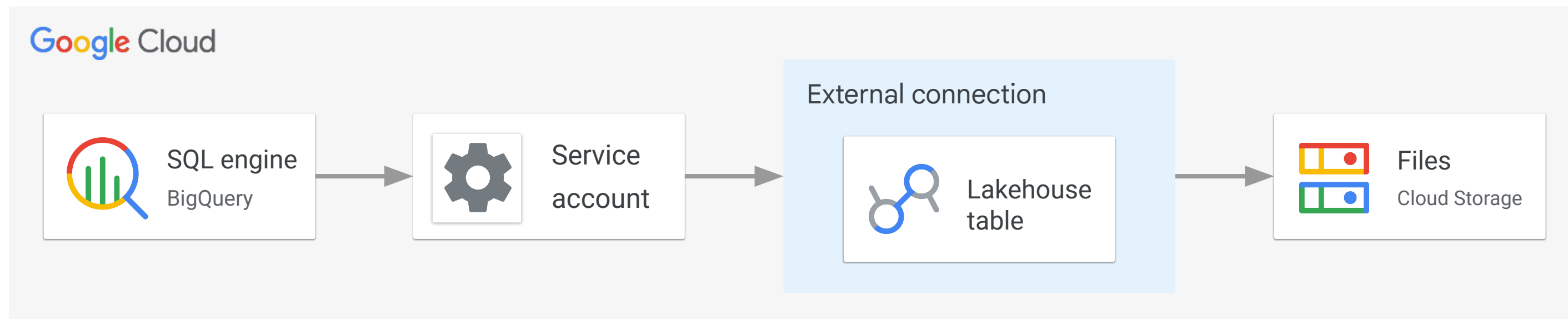
Cache properties

- Staleness between 30 min and 7 days
- Automatic or manual refresh

Compare security approaches: Non-native stored data

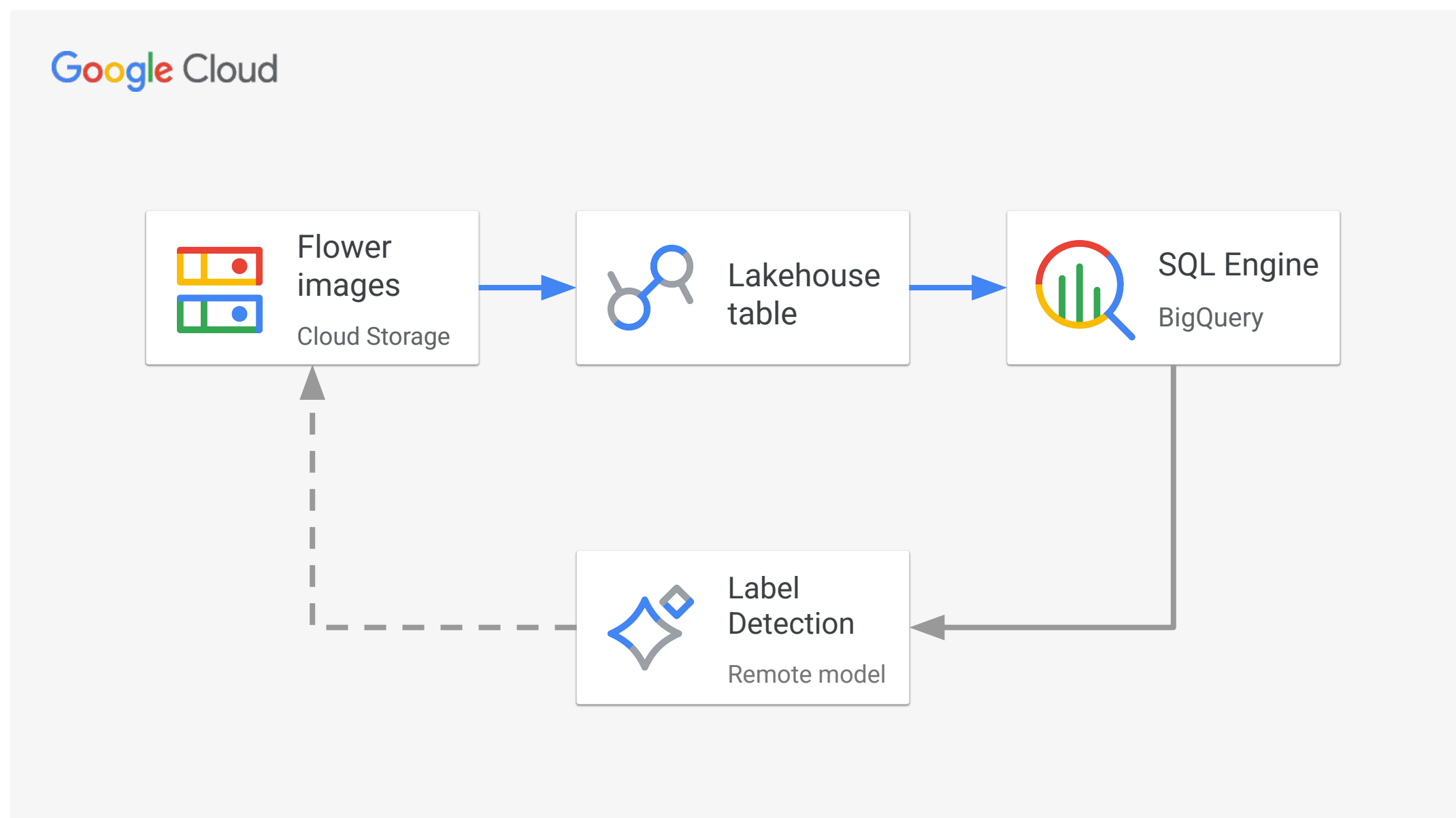


External tables: the user needs separate permissions to access the table and the data source.



Lakehouse: access is delegated using a service account. It decouples access to the table from the underlying data source.

[Demo] Analyze an object table by using a remote model



```
gs://biglake-flowers/flower_photos/  
daisy/pic.jpg
```

```
"label_annotations": [{  
  "description": "Flower",  
  "score": 0.98062  
}]
```

Compare external and Lakehouse tables

	External tables	Lakehouse tables
Query Engine	BigQuery	BigQuery, others using BigQuery connector
Data types	• Parquet, Avro, JSON, ORC, CSV • Datastore export, Firestore export • Google Sheets	• Parquet, Avro, JSON, ORC, CSV • Apache Iceberg, Delta, Hudi • Hive partitioned • Any unstructured (via object tables)
Locations	• Cloud Storage • Bigtable • Google Drive	• Cloud Storage • Amazon S3 • Azure Blob Storage
Access pattern	Direct link to the source. The user needs to have access to the source	External connection with access delegation. The service account has access to the source.
Security	None	• Column-level security, data masking • Row-level security

DIN: Loading data — console and CLI

~10 minutes · BigQuery console + Cloud Shell — into your class dataset

1. Console: Create table on your class dataset from `jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv`
2. Details tab: 1,000,000 rows; autodetect typed minutes as INTEGER, event_date as DATE
3. Append Tuesday's file: `bq load --autodetect --source_format=CSV class.usage gs://.../usage_2026-07-21.csv`
4. Refresh: 2,000,000 rows — loads append by default; `--replace` truncates first
5. View all jobs: console and bq load are two front ends for the same free load job

Why it matters — your Bronze-landing pattern in miniature — vendor file → bucket → BigQuery table, the same shape as the extract jobs you are replacing.

DIN: The autodetect pitfall — N/A in a numeric column

~10 minutes · Cloud Shell + BigQuery console

1. Paste `autodetect_demo.sh` into Cloud Shell — two vendor drops with N/A in the numeric minutes column
2. Load Monday (100k rows, N/A at row 90,000) with `--autodetect`: fails loudly — Unable to parse
3. Load Tuesday (1,000 rows, N/A at row 700) the same way: succeeds, but minutes lands as `STRING`
4. `SUM(minutes)` fails — loud failure vs silent wrong schema, decided by the sample window
5. Fix: pinned schema JSON + `--null_marker='N/A'` — 101,000 rows, 2 NULLs, `SUM` works

Why it matters — autodetect is the SSIS "Suggest Types" button — fine for a first look, never for a recurring feed. The silent failure is the one that costs you days.

DIN: CSV vs Parquet race — 60 million rows, two ways

~10 minutes · Cloud Shell — `gs://jwd-gcp-demos/ingest_demo`

1. Same 60M rows staged two ways: one 317 MiB gzipped CSV vs 100 Parquet files (~829 MiB total)
2. Load the gzip: `bq load --source_format=CSV` with an inline schema — watch the counter, about two minutes
3. While it runs: a gzip cannot be split — one worker decompresses and parses the whole file alone
4. Load the Parquet copy: `bq load --source_format=PARQUET` — done in seconds; Parquet carries its own schema
5. `bq show` both tables: 60,000,000 rows each — more than twice the bytes, roughly 20x faster

Why it matters — your Bronze-layer format decision: land Parquet wherever you control the format; for vendor CSVs you don't, prefer many small files over one large gzip.

DIN: LOAD DATA — the load job goes SQL

~5–8 minutes · BigQuery query editor — `load_data_example.sql`

1. Run the load-monday query: `LOAD DATA INTO class.usage_sql` with a column list pinning the schema, `FROM FILES CSV` options
2. Run the load-tuesday query: no column list — the table exists; `LOAD DATA INTO` appends, `OVERWRITE` would replace
3. Run the verify query: each date holds 1,000,000 meetings and `SUM(minutes)` works
4. Check Job information on either load: Bytes billed = 0 B — SQL syntax, same free load machinery

Why it matters — the load is now a statement that lives in a script, procedure, or scheduled query — one scheduled `LOAD DATA` with a date-built URI replaces a nightly extract job.

M3 — What you now have

- ✓ The EL pattern mapped: console, bq load, and LOAD DATA all submit the same free load job into BigQuery
- ✓ Four loads with your own hands— console & CLI, the autodetect N/A trap, the 60M-row format race, LOAD DATA SQL
- ✓ Formats and pinned schemas are Bronze-layer design decisions, not conveniences
- ✓ Scheduled ingest covered two ways: Data Transfer Service, or a scheduled query running LOAD DATA
- ✓ Next: break, then M4 ELT— transform where the data already lives, and your first look at Dataform

Console & CLI load (DIN §24)

~10 minutes — within the 13:45–14:30 loading block · BigQuery console + Cloud Shell — into your class dataset

1. Create table from Cloud Storage on your class dataset:
jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv
2. Format CSV, table usage, check Auto detect → Create; the Details tab shows 1,000,000 rows
3. Append Tuesday from Cloud Shell: `bq load --autodetect --source_format=CSV class.usage gs://.../usage_2026-07-21.csv`
4. Refresh Details: 2,000,000 rows — loads append; View all jobs shows two identical Load jobs

Why it matters — the console form and bq are two front ends over one job API — what you click today is what you script in the migration.

Autodetect pitfall (DIN §25)

~10 minutes — within the 13:45–14:30 loading block · Cloud Shell + BigQuery editor — into your class dataset

1. Paste `autodetect_demo.sh` into Cloud Shell — writes Monday, Tuesday and a pinned-schema JSON
2. Load Monday with autodetect → loud failure: its N/A at row 90,000 sat outside the sample
3. Load Tuesday → silent failure: N/A inside the sample → minutes typed STRING; SUM fails
4. Fix: reload both days with `--schema`, `--skip_leading_rows=1`, `--null_marker='N/A'` (`--replace` first)
5. Verify: 101,000 rows, exactly 2 NULL minutes, SUM works

Why it matters — same bad value — loud once, silent once; the silent one costs days. Pin schemas in source control; autodetect is a first look, never a pipeline.

CSV vs Parquet load race (DIN §26)

~15 minutes — within the 13:45–14:30 loading block · Cloud Shell — into your class dataset

1. 60M rows staged twice: one gzipped CSV (~317 MiB) vs 100 Parquet files (~829 MiB) — place your bet
2. Load the CSV with an explicit schema — about two minutes: gzip can't split, one worker alone
3. Load the Parquet with no schema (the files carry their types) — seconds
4. bq show both tables: 60,000,000 rows each; Parquet got INTEGER/DATE from the files

Why it matters — twice the bytes, ~20x faster — columnar + splittable beats one big gzip. Wherever you control the format, land Parquet.

LOAD DATA SQL (DIN §27)

~10 minutes — within the 13:45–14:30 loading block · BigQuery query editor — into your class dataset

1. Run the Monday statement in `load_data_example.sql` — the column list pins the schema
2. Run the Tuesday statement — `LOAD DATA INTO` appends (`LOAD DATA OVERWRITE` would replace)
3. Run the verify query: 1,000,000 meetings per date — Job information shows Bytes billed: 0 B

Why it matters — a load expressed as SQL lives in source control, procedures and schedules — the nightly-extract replacement in a single statement.

Afternoon Break

15 minutes · 14:30–14:45

M4 — What this block gets you

- ✓ ELT reframed— land raw in BigQuery, transform with SQL inside the warehouse, not in an SSIS data flow
- ✓ Scripting, UDFs, stored procedures— your T-SQL instincts transfer; watch dialect, not concepts
- ✓ Scheduled queries— the SQL Agent instinct in one click; the closing demo makes it concrete
- ✓ Dataform— your architecture doc's Silver→Gold tool: sqlx models, the ref() graph, assertions, releases
- ✓ Hands-on: watch a real repo end to end, then build your own SQL workflow in the anchor lab

This afternoon, block by block

14:45–15:10 M4 — ELT in BigQuery and the Dataform intro; fast through scripting, UDFs, sprocs

15:10–15:25 Jeff's dataform-demo repo, live — a real Silver→Gold pipeline in git

DEMO

15:25–16:05 Lab: Create and Execute a SQL Workflow in Dataform — the anchor lab, full slot

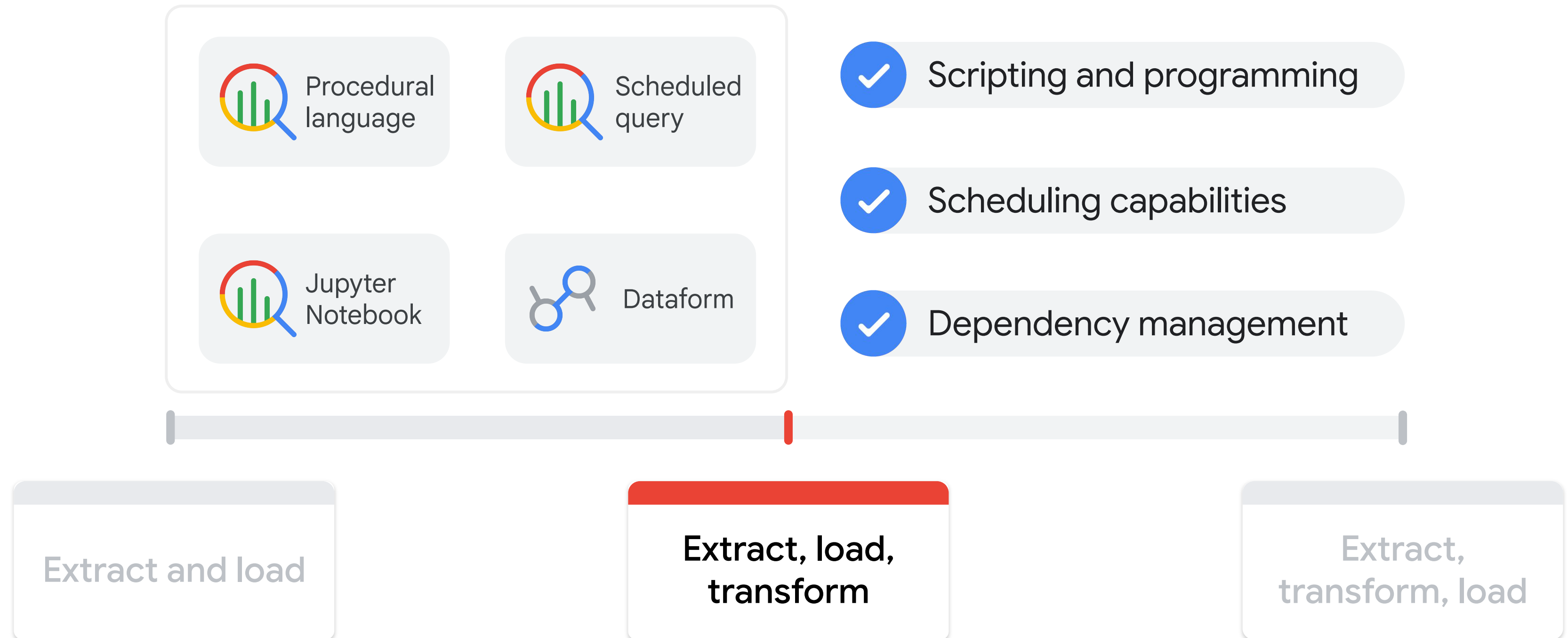
LAB

16:05–16:20 M5 + M6 compressed — ETL options and the automation ladder

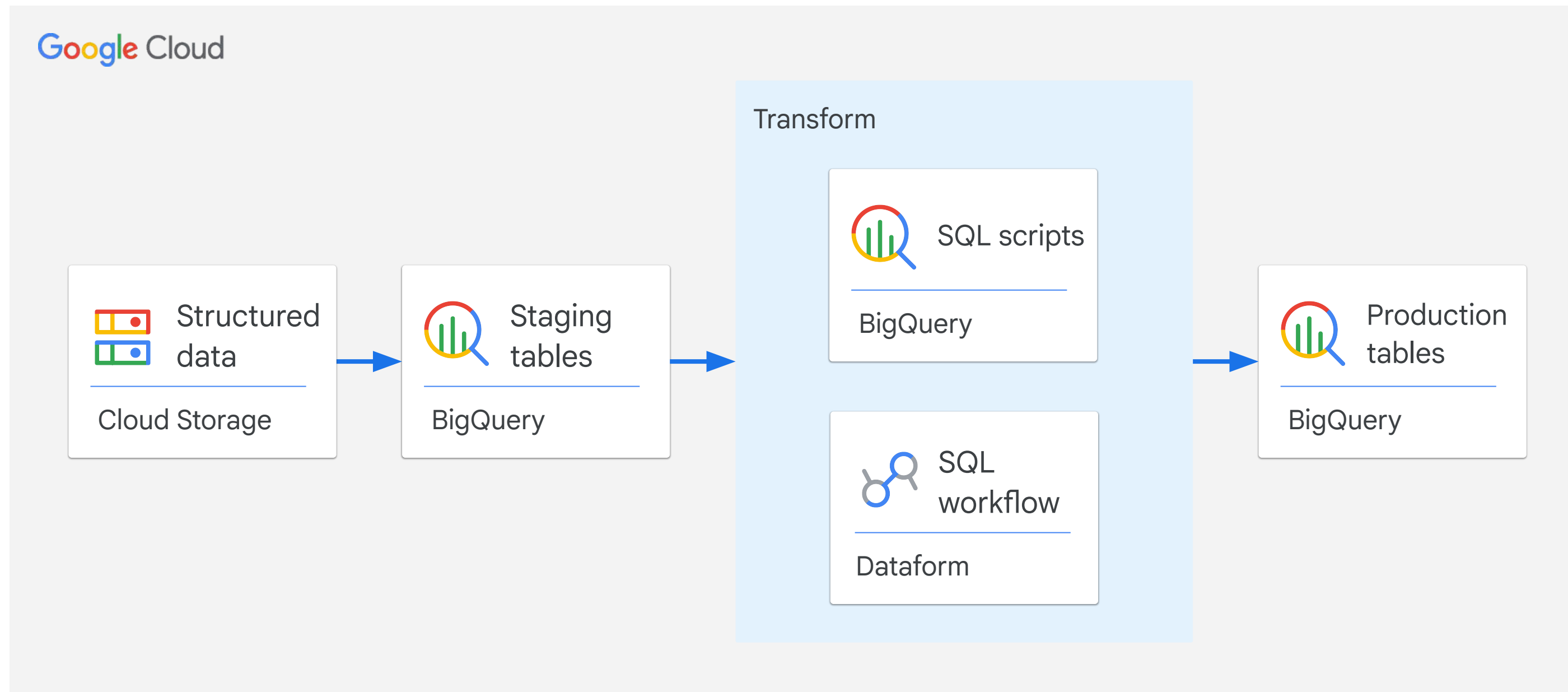
16:20–16:30 Close: schedule the derived-table query — one click, your nightly SQL Agent job

DEMO

Once data is loaded into BigQuery, there are multiple ways to transform it



Use an ELT pipeline to apply transformations after data is staged in BigQuery



BigQuery supports procedural language queries to perform multiple statements in a sequence

```
-- create variables
DECLARE book_name STRING DEFAULT 'Ulysses';
DECLARE book_year INT64 DEFAULT 1922;

-- Create a temporary table called Books.
EXECUTE IMMEDIATE
  "CREATE TEMP TABLE Books
    (title STRING, publish_date INT64)";

-- add a row for Ulysses, using the variables
-- declared and the ? placeholder
EXECUTE IMMEDIATE
  "INSERT INTO Books (title, publish_date) VALUES(?, ?)"
  USING book_name, book_year;
```

Procedural language in BigQuery:

- Runs multiple statements in a sequence with shared state.
- Automates management tasks, such as creating or dropping tables.
- Implements complex logic using programming constructs like `IF` and `WHILE`.
- Can declare user-created variables or reference BigQuery's system variables.
- Can use transactions (`COMMIT`, `ROLLBACK`).

User-defined functions (UDFs) transform data by using SQL or JavaScript

```
# create a permanent user-defined function
CREATE FUNCTION dataset_name.AddFourAndDivide
(x INT64, y INT64) RETURNS FLOAT64 AS (
  (x + 4) / y
);
```

```
# use the function in your SQL script
SELECT
  val, dataset_name.AddFourAndDivide(val, 2)
FROM ...;
```

Usage notes:

- Can be persistent (CREATE FUNCTION) or temporary (CREATE TEMPORARY FUNCTION).
- Use SQL where possible. Use JavaScript if the same cannot be accomplished with SQL.
- JavaScript UDFs can use additional input libraries.
- The `bigquery-utils` project and the `bigquery-public-data.persistent_udfs` public dataset contain community-contributed UDFs.

Stored procedures execute a collection of SQL statements

```
# create a stored procedure
CREATE OR REPLACE PROCEDURE
  dataset_name.create_customer()
BEGIN
  DECLARE id STRING;
  SET id = GENERATE_UUID();
  INSERT INTO dataset_name.customers (customer_id)
    VALUES(id);
  SELECT FORMAT("Created customer %s", id);
END
```

```
# call the procedure in your SQL script
CALL dataset_name.create_customer();
```

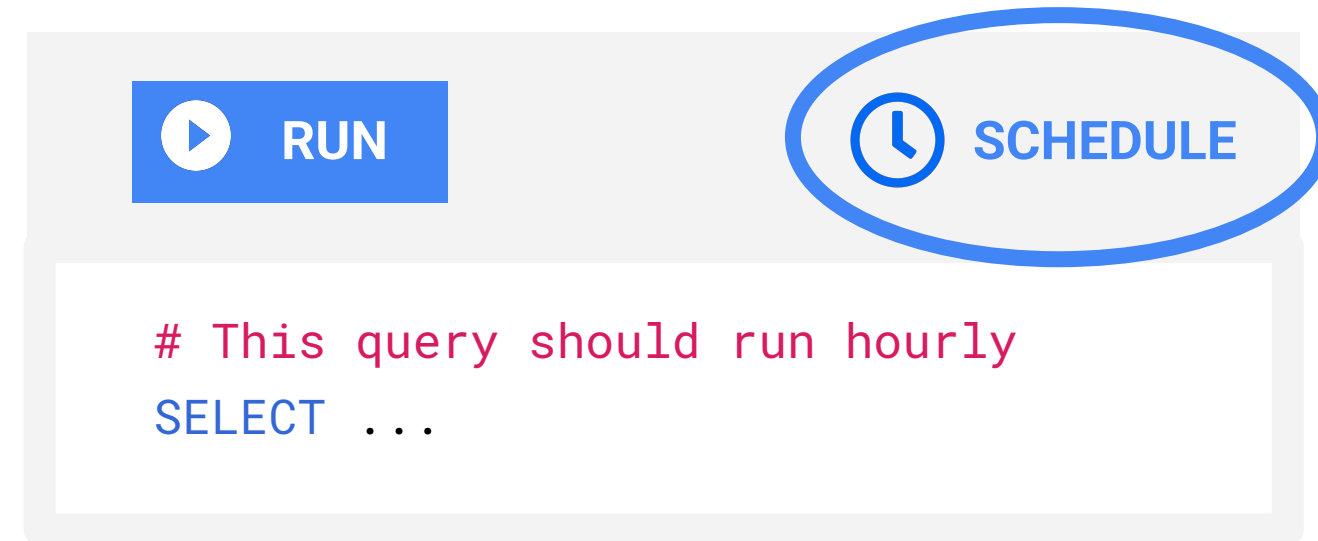
Usage notes:

- Can be called from queries or other stored procedures.
- Accept input values and return values as output.
- Can access or modify data across multiple datasets.
- Can contain multi-statement queries and support procedural language statements.

Save your queries and schedule them for repeating use cases

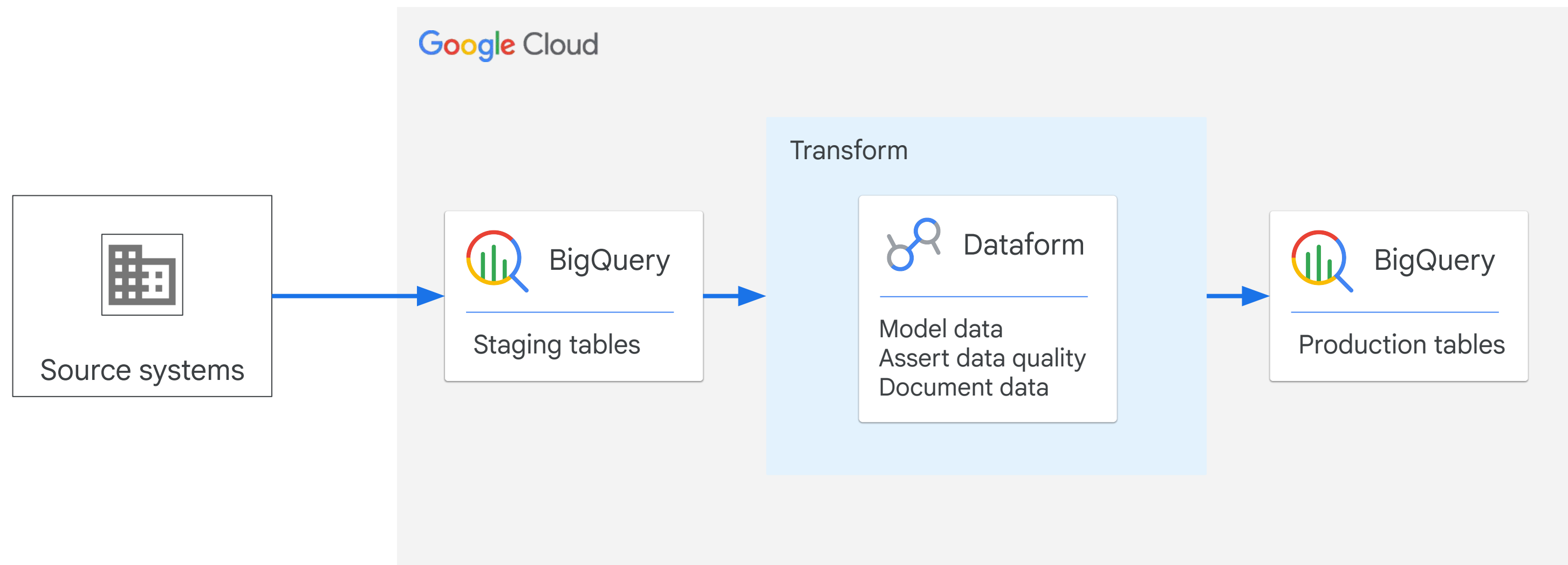


- Is visible in Explorer > Queries.
- Supports version control.
- Share with users or groups.
- Download as `.sql` file.
- Upload other queries from `.sql` files.



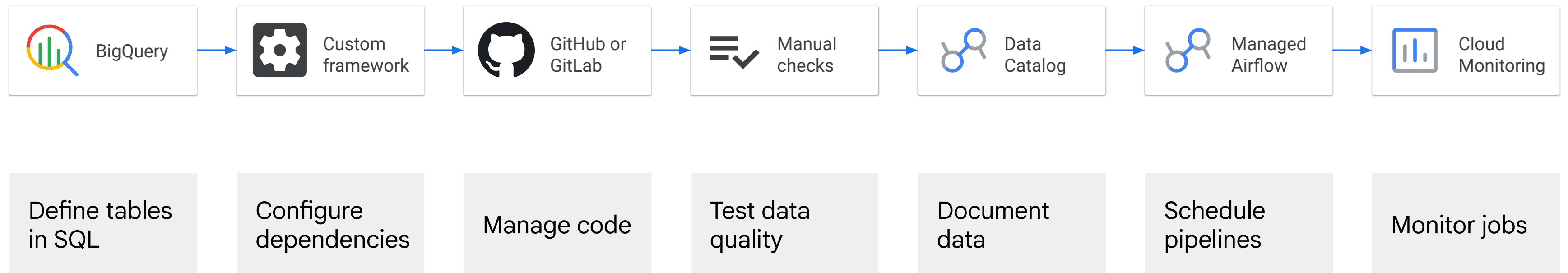
- Set repeat frequency.
- Set start and end times.
- Specify destination for query results.
- Can use runtime parameters.
- **Tip:** use Dataform for SQL workflows.

Dataform is a serverless framework to develop and operationalize ELT pipelines in SQL

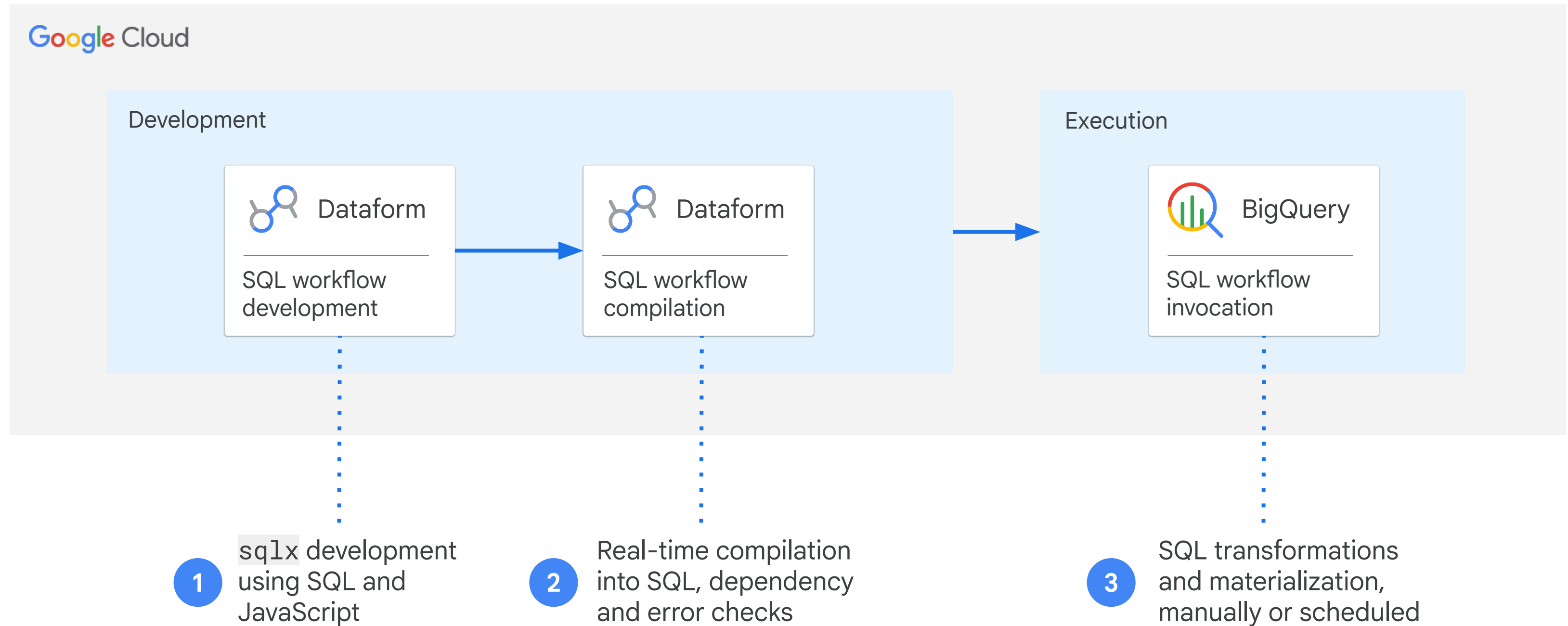


Dataform unifies data transformation, assertion, and automation in BigQuery

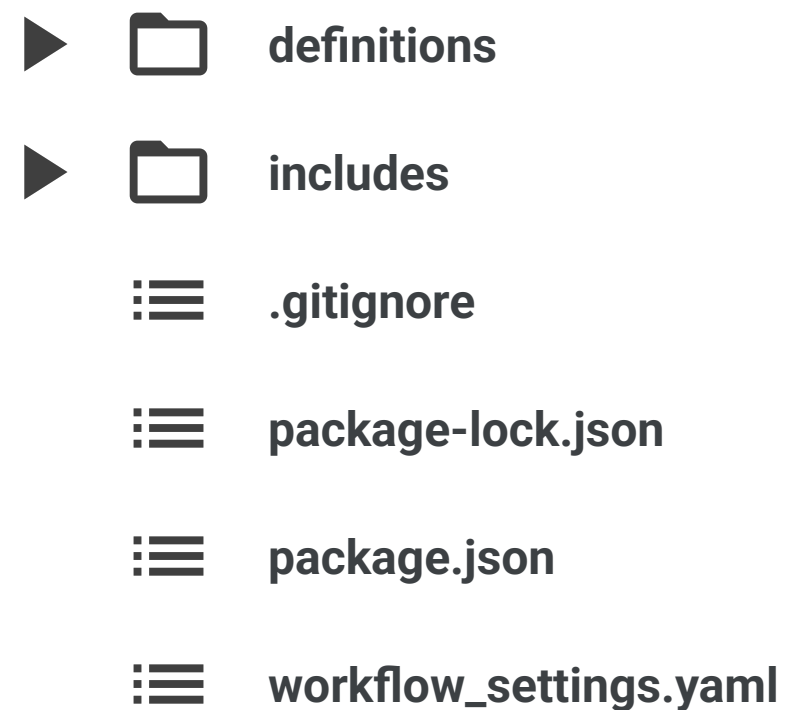
Without Dataform, this would be a time-consuming and error-prone process



SQL workflow development and compilation run in Dataform, workflow execution runs in BigQuery



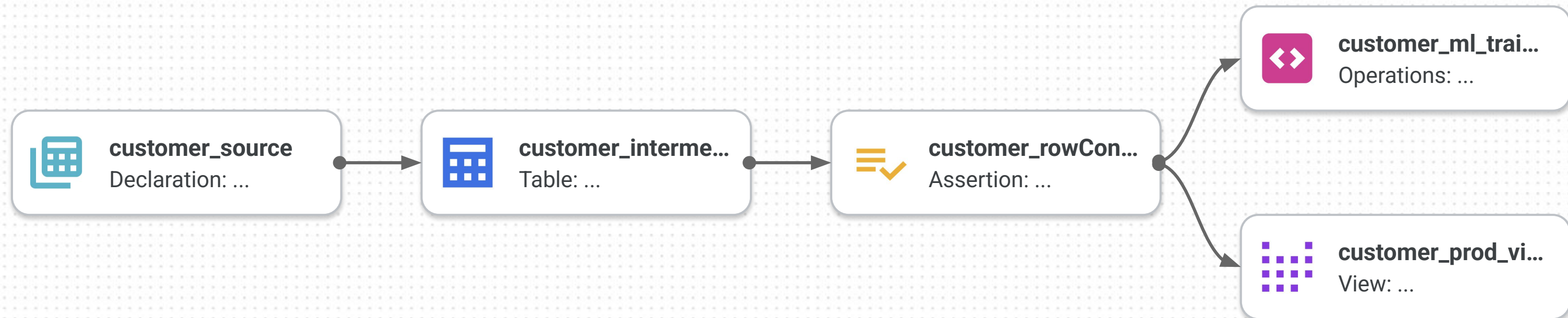
Development happens in workspaces with default files and folders



You can add custom files like `README.md` as well.

- **definitions** folder:
contains `sqlx` files.
- **includes** folder:
contains JavaScript files.
- **.gitignore** file:
lists files and directories to ignore during Git commits.
- **package.json** file (optional):
defines JavaScript packages and required versions.
- **package-lock.json** file (optional):
contains installed packages with exact versions.
- **workflow_settings.yaml** file:
contains basic settings required to compile the project.

The compiled graph visualizes the SQL workflow with its definitions and dependencies



Demo: a real Dataform repo, end to end

15 minutes · github.com/jwdavis/dataform-demo, plus the Google Cloud console

1. Walk the repo tree: dataform.json, one constants file, definitions/ — declarations, staging, marts
2. Read the sqlx: a declaration (Bronze), staging cleanup (Silver), an aggregate mart (Gold)
3. Console: repository, GitHub connection, development workspace — your feature branch
4. Compiled graph: ref() calls become the dependency graph — your Silver→Gold pipeline as code
5. Add an assertion live, then release config + scheduled invocation — your CI/CD, translated

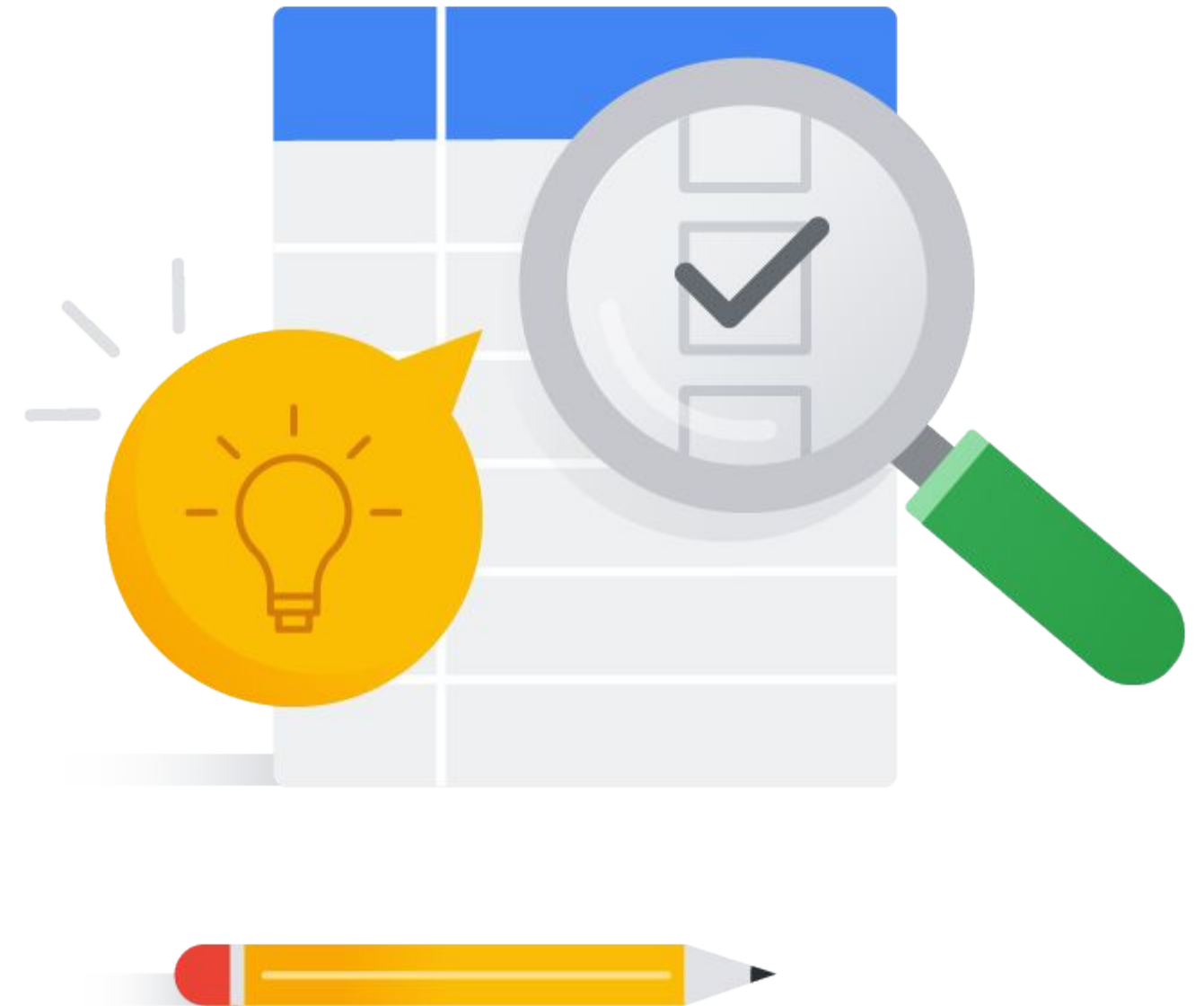
Why it matters — your architecture doc names Dataform for Silver→Gold — here is one running: raw → staging → marts, in git, deployable. The discipline you already run for SQL Server.

Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Lab: Create and Execute a SQL Workflow in Dataform

40 minutes · Google Skills classroom — lab project, BigQuery Studio → Dataform

1. Create a Dataform repository — watch the region; a mismatch is the classic stall
2. Create and initialize a development workspace — files appear only after Initialize
3. Define a staging view and a mart table in sqlx, wired together with `ref()`
4. Execute the workflow and read the logs — actions run in dependency order, compiled SQL shown
5. Early finishers: add a `uniqueKey` or `rowConditions` assertion and watch it enter the graph

Why it matters — the one solid lab in the course — these 40 minutes are exactly what your migration does at work: SQL models in git, compiled into a graph, executed in order, tested in-pipeline.

Close: schedule the derived-table query

5 minutes · BigQuery Studio — Schedule → Create new scheduled query

1. Run the sales-by-zip query once into class.march_zip_sales
2. Schedule → Create new scheduled query: repeat daily at 12:01 AM
3. Destination: your class dataset; write preference Overwrite (WRITE_TRUNCATE)
4. Save, find it under Scheduled queries, trigger one manual run to prove it fires

Why it matters — a SQL statement, on a schedule, overwriting a table — your nightly SQL Agent job minus the server. Day 2 replaces hand-rolled derived tables with Dataform-managed marts.

M4 — What you now have

- ✓ ELT on BigQuery— land Bronze, let SQL build Silver and Gold inside the warehouse
- ✓ Dataform, hands-on— repo, workspace, sqlx models, the ref() graph, executed and logged
- ✓ The git discipline you already run— workspace = feature branch, release config = deployment
- ✓ A scheduled query overwriting a derived table nightly— SQL Agent translated, first automation
- ✓ Tomorrow: the medallion and governance— Gold marts feeding the Power BI you already have

Lab — Create & Execute a SQL Workflow in Dataform

40 minutes — 15:25–16:05 · Google Skills classroom — Course 1, Module 4 lab

1. Launch the lab; work in the lab project, in the region the lab specifies — check the picker
2. Create a Dataform repository (BigQuery Studio → Dataform)
3. Create a development workspace and Initialize it — the #1 stall: no files until you do
4. Define a staging view and a mart table in sqlx, wired with ref() — Silver→Gold in miniature
5. Execute the workflow; read the execution logs — each action's status and compiled SQL
6. Early? Add an assertion (uniqueKey / rowConditions) and watch it join the compiled graph

Why it matters — your architecture names Dataform as the Silver→Gold ELT tool — these 40 minutes are the thing you will actually do at work.

WATCH FIRST · 3:20 · GOOGLE CLOUD TECH

Moving Cron Jobs to the Cloud

- ✓ Cloud Scheduler = managed cron that calls HTTP, Pub/Sub, or App Engine — the SQL Agent analogue with no scheduler VM to patch.
- ✓ Fifteen minutes from now you'll watch a scheduled BigQuery query fire — this is the general-purpose version of that same idea.



Scan to open on YouTube

youtu.be/7Z1mgOxWTs8

M5 + M6 — What this 15-minute block gets you

- ✓ M5 in one line— the escape hatch when SQL is not enough: custom code, streaming, non-BigQuery targets
- ✓ Batch vs streaming as a latency decision— Pub/Sub, Dataflow and templates, named not drilled
- ✓ M6 = your CI/CD analogue— Cloud Scheduler $\approx$ SQL Agent, Composer $\approx$ the orchestrator
- ✓ Aimed at the M&O team— who keeps jobs running: scheduling, retries, failure visibility
- ✓ No lab in this block— the Spark lab is optional homework; the close schedules one live query

Last block of Day 1 — no lab, then we close

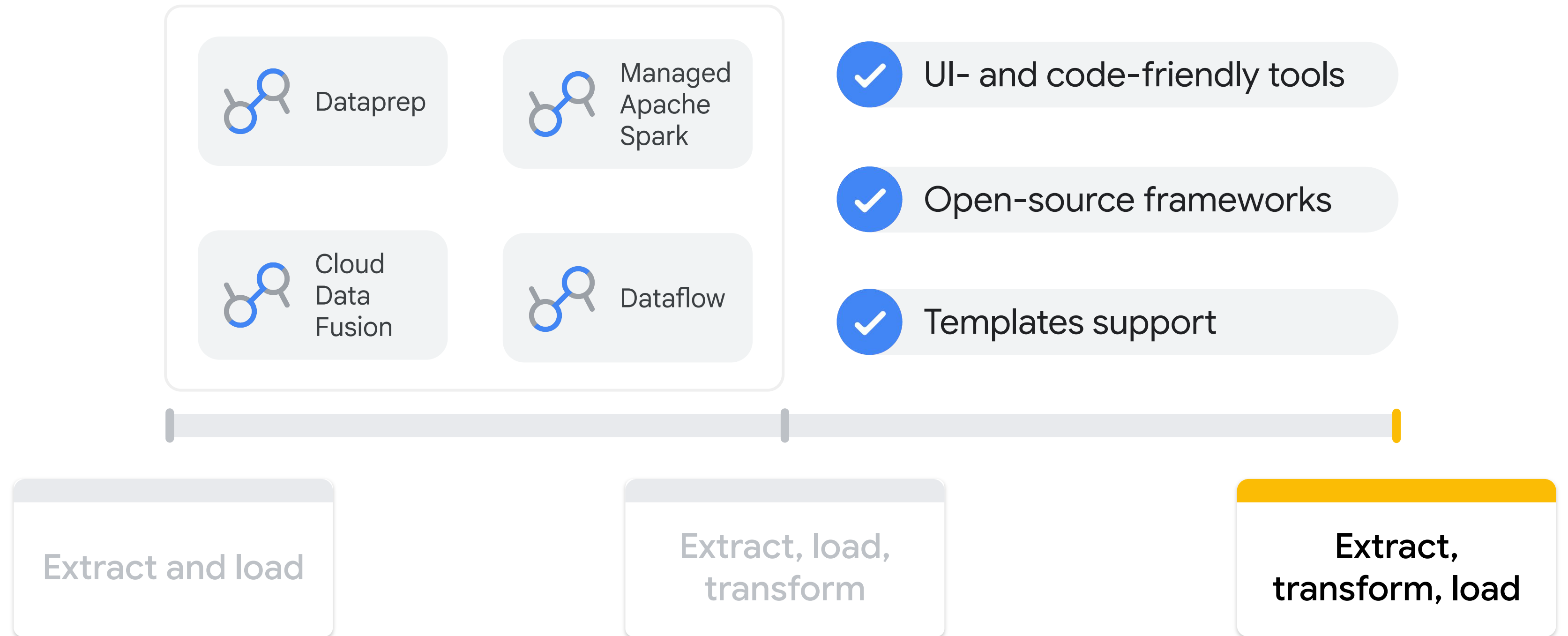
16:05–16:15 M5 compressed — ETL options: batch vs streaming, Pub/Sub, Dataflow, templates

16:15–16:20 M6 compressed — automation: Scheduler, Composer, Dataform release configs

16:20–16:30 Close: schedule the derived-table query live, Day 1 recap, Day 2 preview

DEMO

Google Cloud provides multiple services for distributed data processing

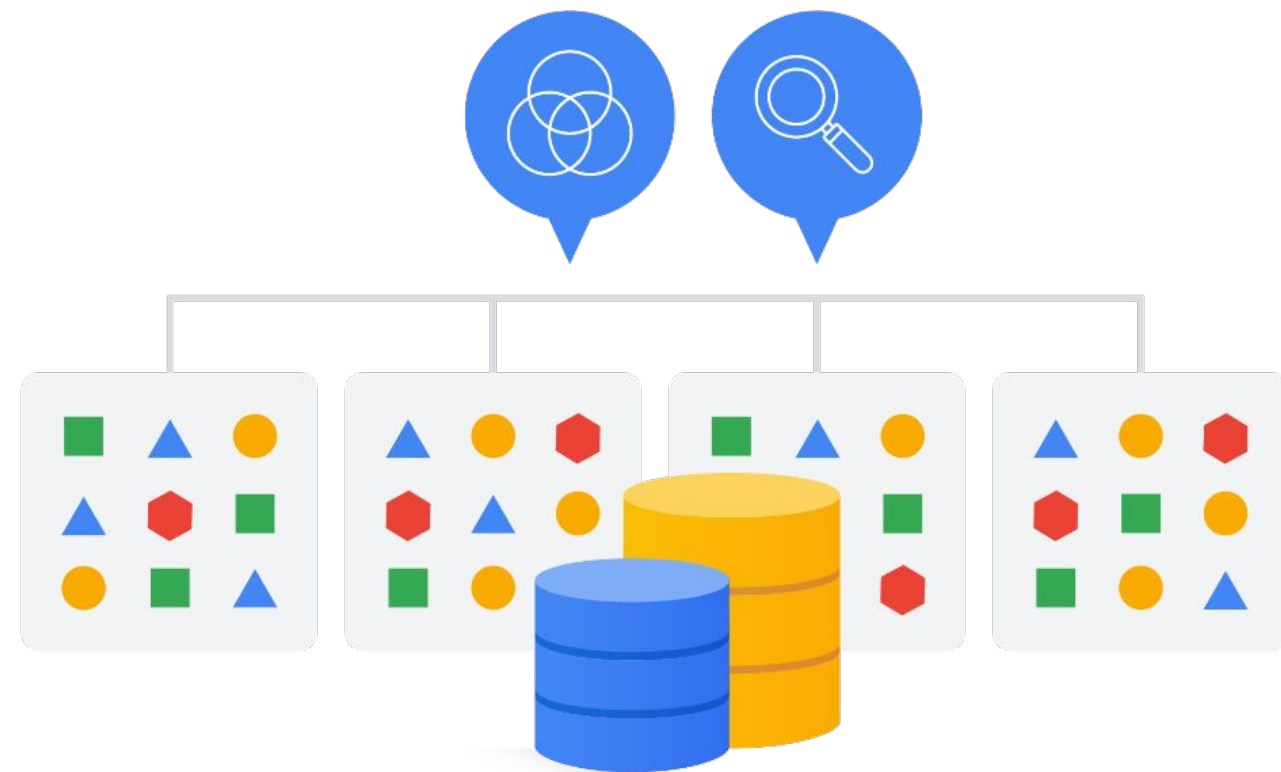


M5 — the escape hatch, not the destination

- Default stays ELT in BigQuery— reach beyond SQL only for custom code, streaming, non-BQ targets
- Batch vs streaming— a latency decision, not a religion; nightly covers most warehouse work
- Pub/Sub— the managed messaging backbone; your Service Broker / queue analogue
- Dataflow— managed Apache Beam runner for batch and stream; templates = configure, not code
- Depth lives elsewhere— the Spark lab is optional homework; streaming is Course 4

Conceptual pass only — know the names and when to reach for each.

Batch processing versus streaming data processing

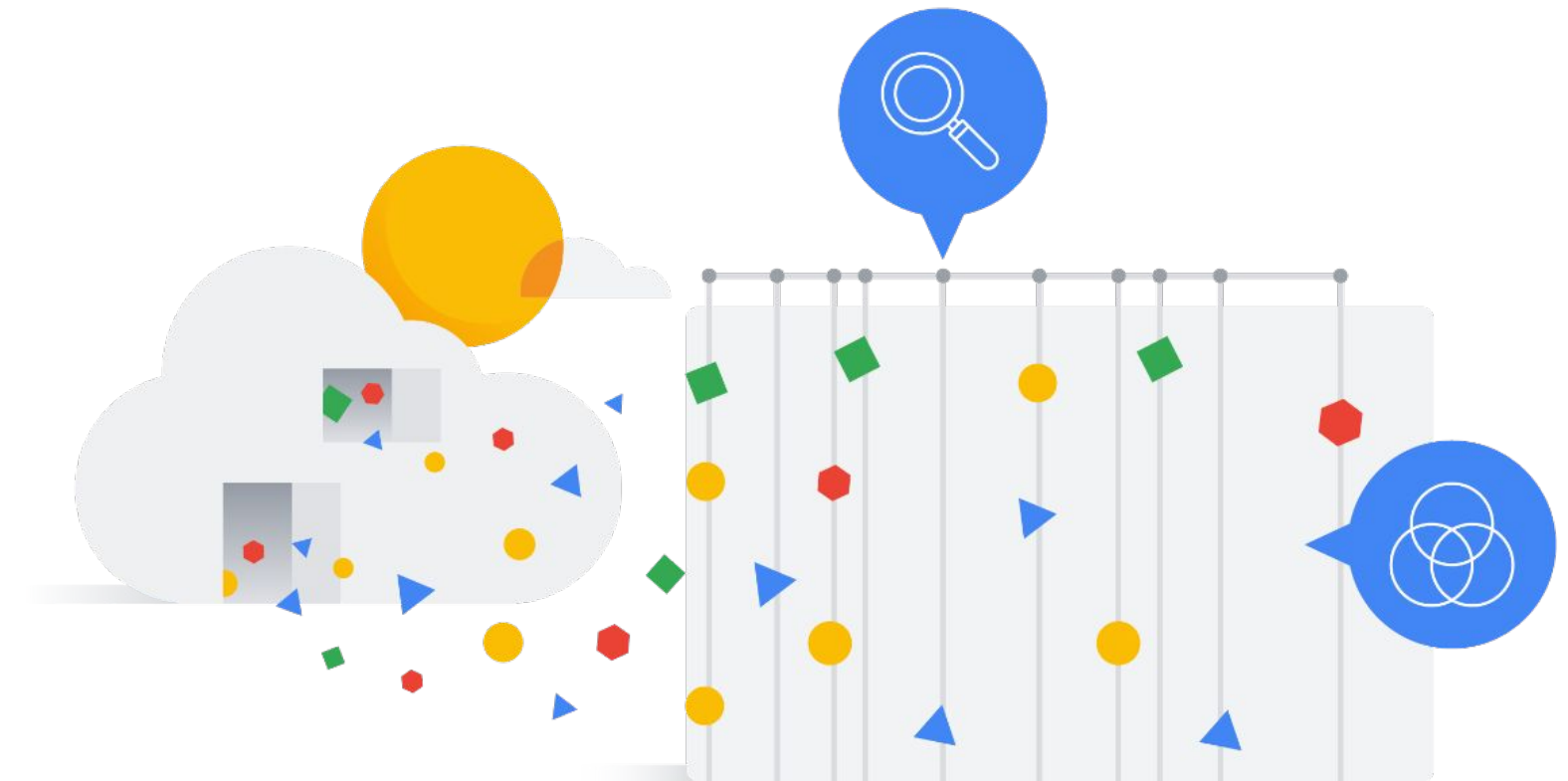


Batch processing

Processing and analysis happens on a set of stored data.

Payroll system

Billing system



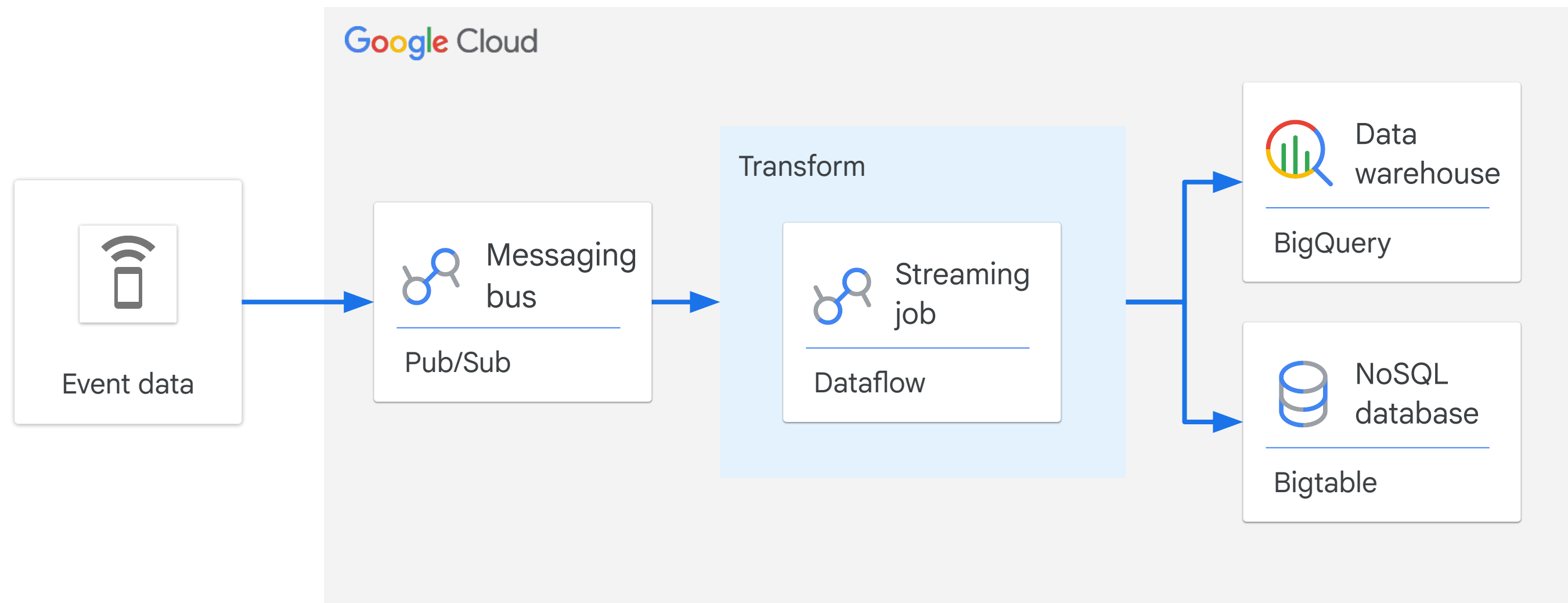
Streaming data processing

A flow of data records generated by various data sources.

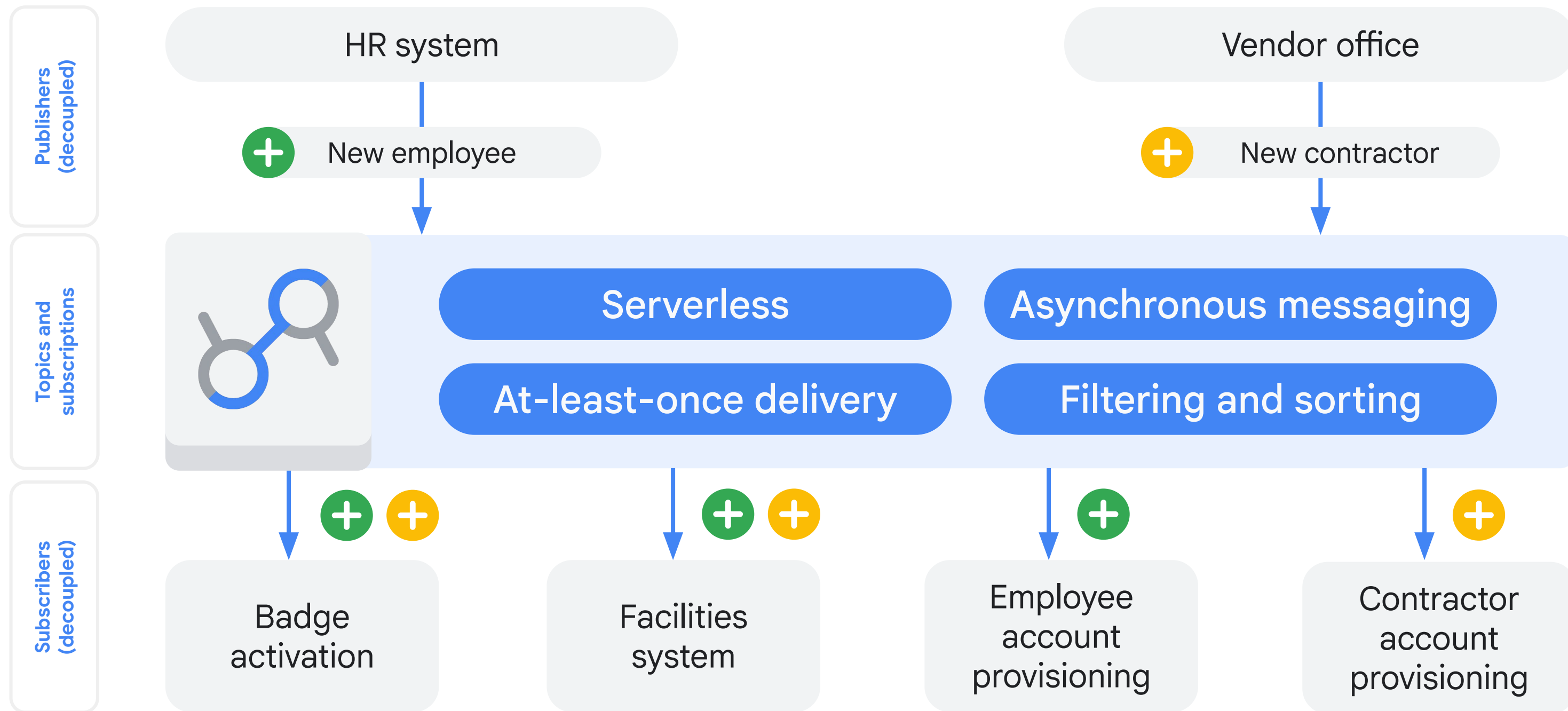
Fraud detection

Intrusion detection

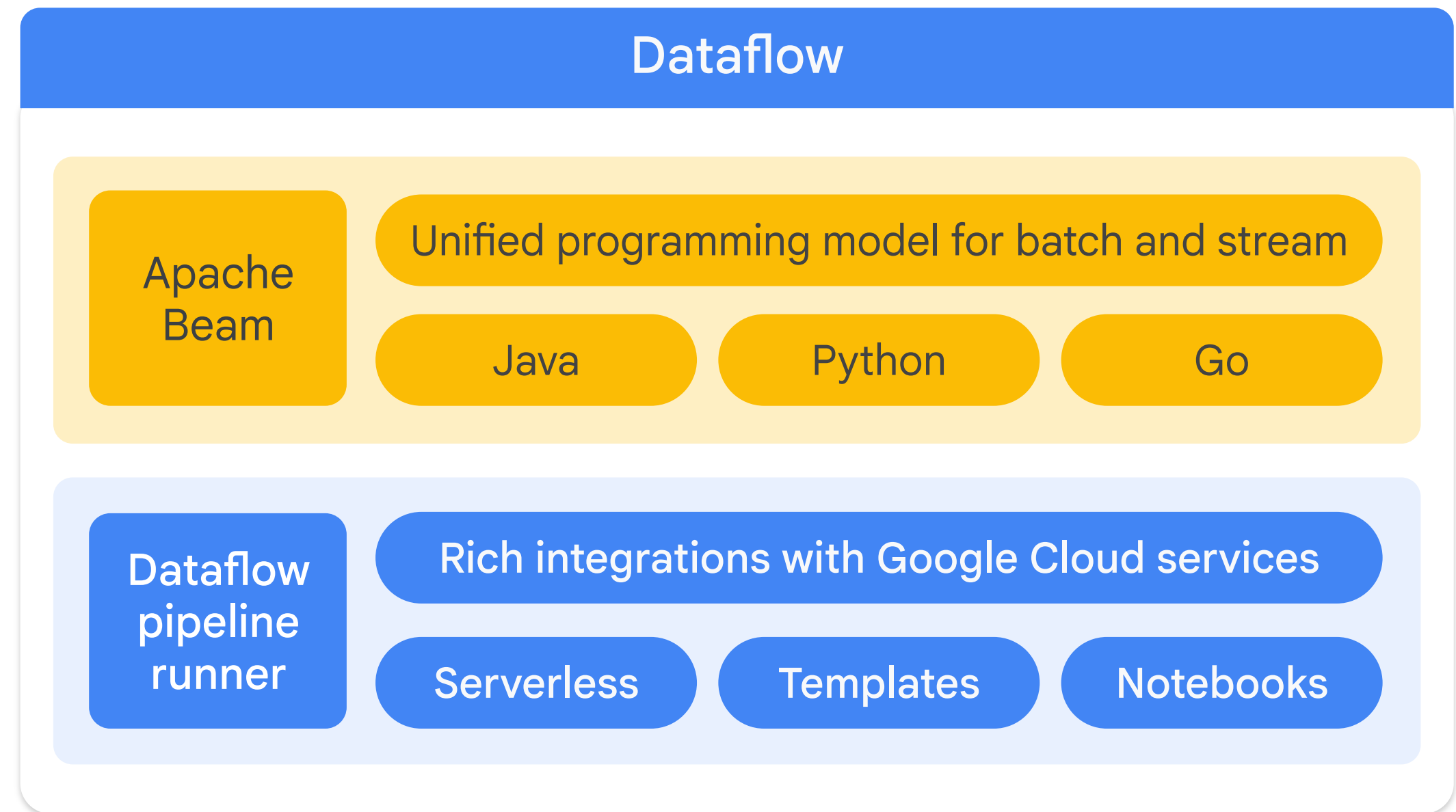
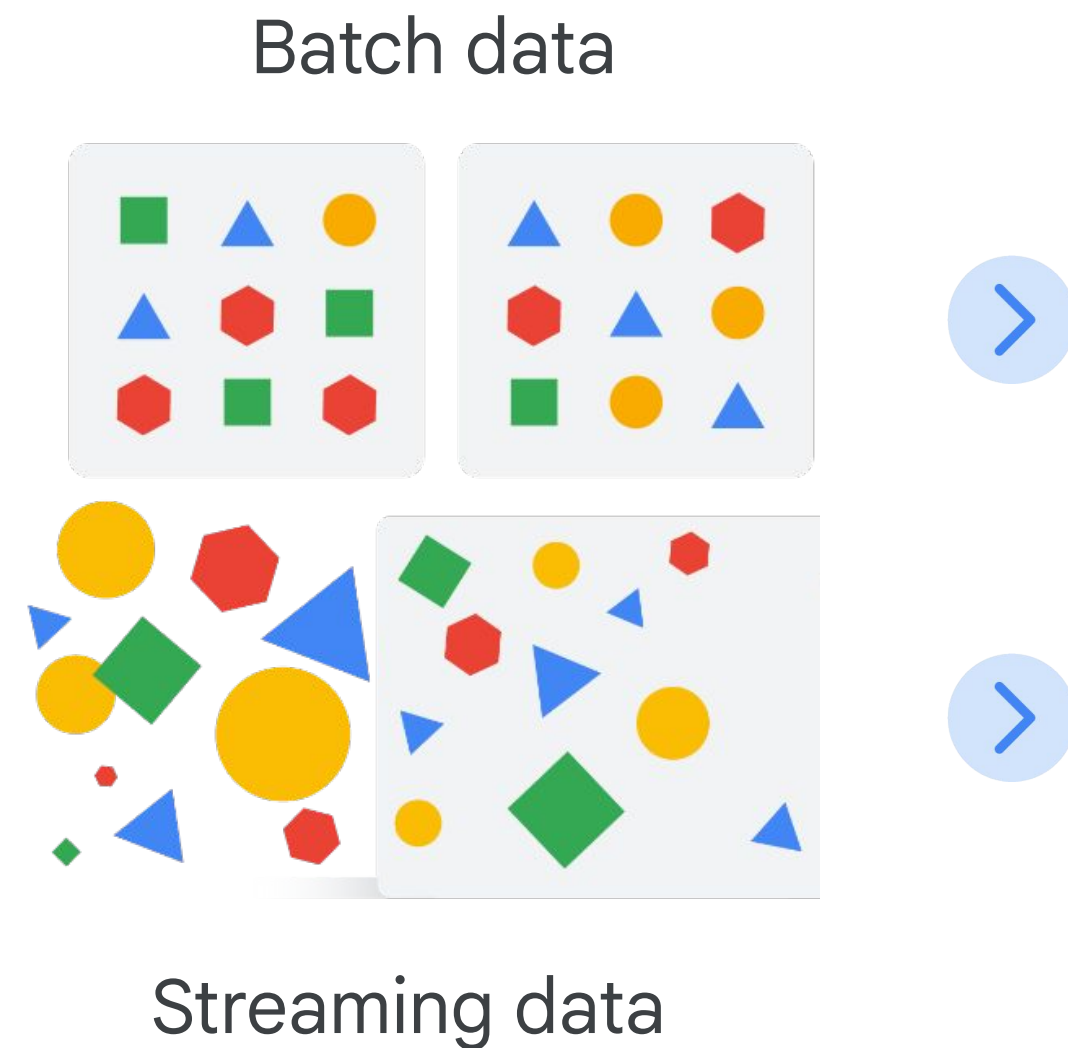
Streaming ETL workloads require continuous data ingestion, processing, and near real-time analytics



Use Pub/Sub to ingest high volumes of event data and distribute to different consuming systems



Dataflow can process both batch and streaming data using the Apache Beam programming model



Example: Streaming and transforming messages from Pub/Sub to BigQuery

```
with beam.Pipeline(options=pipeline_options) as pipeline:

    # Read messages from Pub/Sub
    messages = pipeline | 'read' >> ReadFromPubSub(topic='<PUBSUB_TOPIC>')

    # Parse messages
    def parse_message(message):
        return json.loads(message.decode('utf-8'))

    parsed_messages = messages | 'parse' >> beam.Map(parse_message)

    # Write parsed messages to BigQuery
    parsed_messages | 'write' >> WriteToBigQuery(
        table='<BIGQUERY_TABLE>',
        schema=table_schema,
        create_disposition=BigQueryDisposition.CREATE_IF_NEEDED,
        write_disposition=BigQueryDisposition.WRITE_APPEND
    )

    # Run the pipeline
    pipeline.run()
```

ReadFromPubSub():

- Retrieves messages from a Pub/Sub topic or subscription.

beam.Map():

- Applies a specified transformation to the message.

WriteToBigQuery():

- Writes the transformed message into a BigQuery table.

Use Dataflow templates for recurring pipeline executions with different parameters

 **CREATE JOB FROM TEMPLATE**

Dataflow template
Cloud Storage Text to BigQuery (Stream) ▼

Source
gs:// Cloud Storage Input File(s) [BROWSE](#)

Target
BigQuery output table [BROWSE](#)

Required Parameters ▼

Benefits of using templates

- Pipeline design is separate from deployment.
- Parameters customize the pipeline and make it reusable with different inputs.
- Templated pipelines can be deployed from the Google Cloud console, command-line interface, or API.
- Use Google-provided pre-built templates for common scenarios.



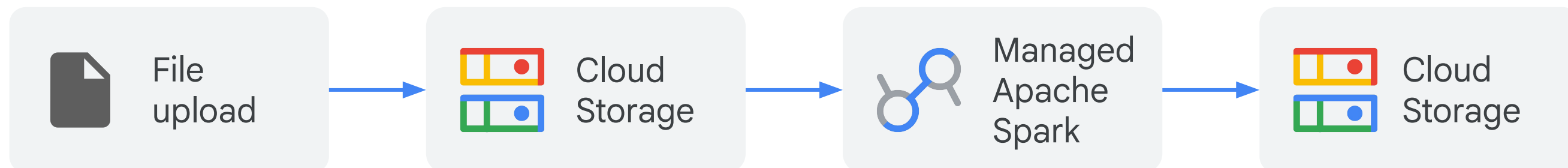
You can create your own custom templates.

ELT and ETL workloads can be automated to run on a recurring basis

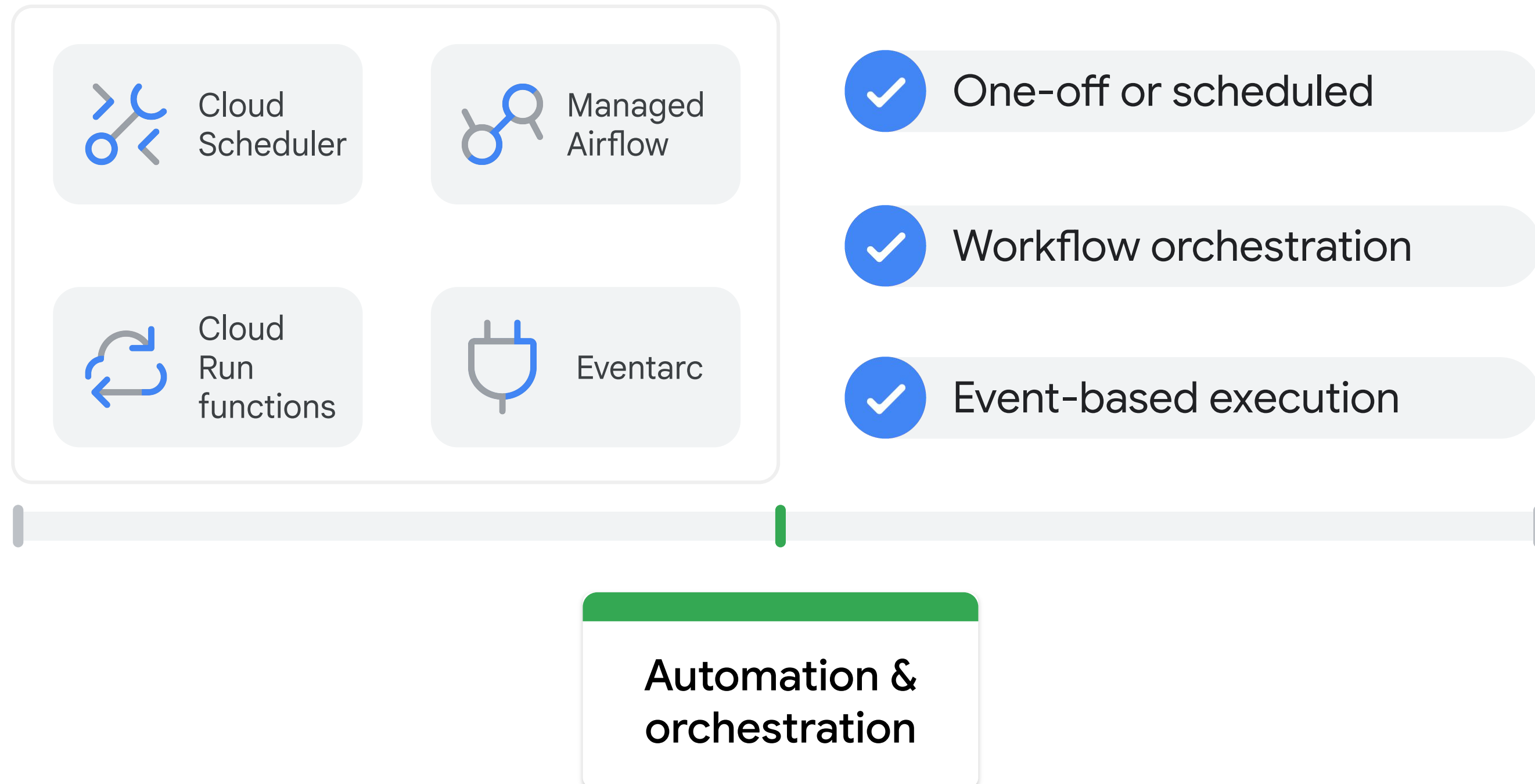
Scheduled ELT example



Event-driven ETL batch example



Google Cloud provides multiple services for automating and orchestrating your workloads

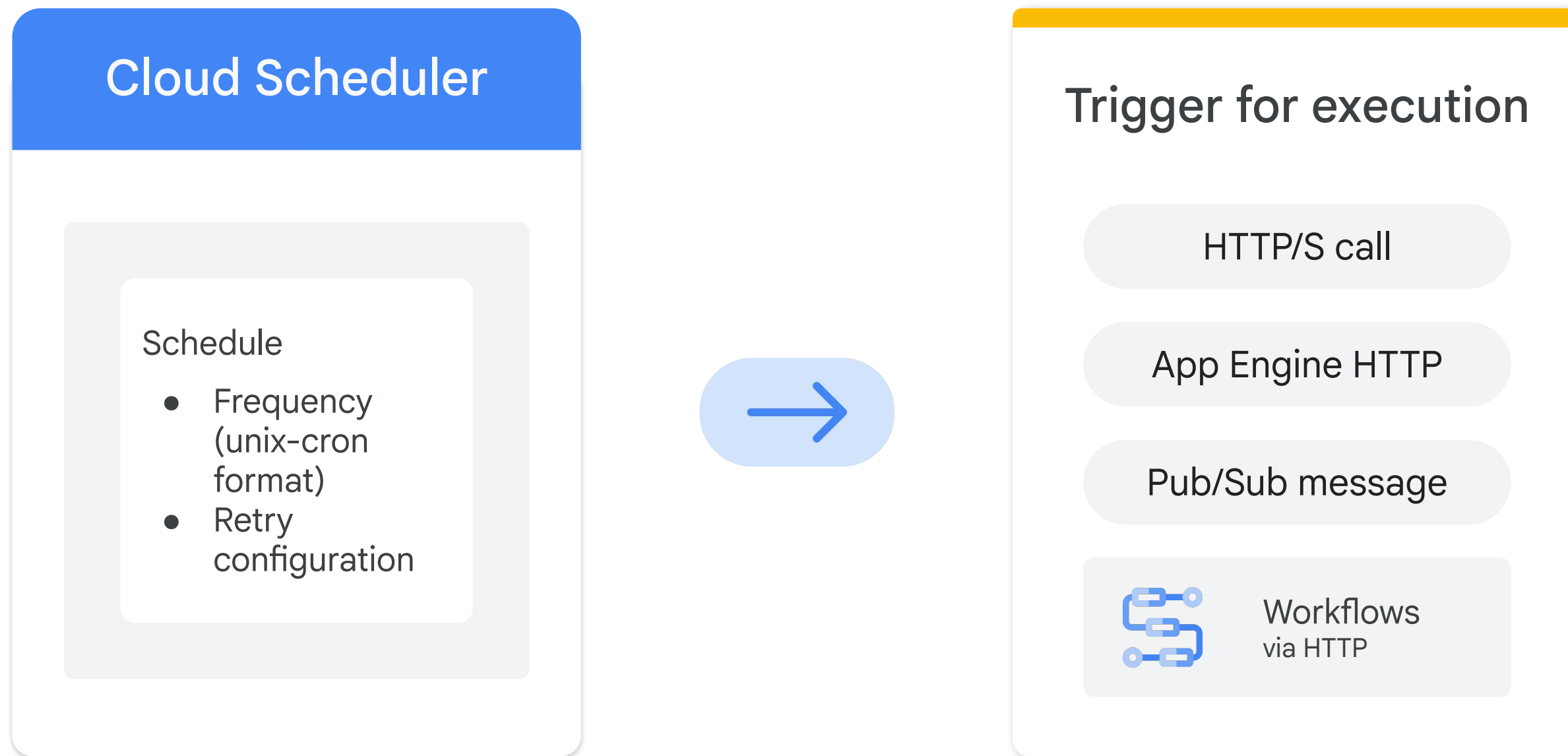


M6 — for the M&O team: who keeps the jobs running

- Your instincts translate— SQL Agent schedules and master packages, one for one
- Cloud Scheduler— managed cron triggering HTTP or Pub/Sub; no scheduler VM to patch
- Scheduled queries— one SQL statement on a timer; the closing demo puts one live
- Dataform release configs— your Silver→Gold runs on a cadence, compiled from a git commit
- Composer (managed Airflow)— the orchestrator when many pipelines depend on each other
- Eventarc + Cloud Run functions— event triggers and glue code, your file-watcher analogue

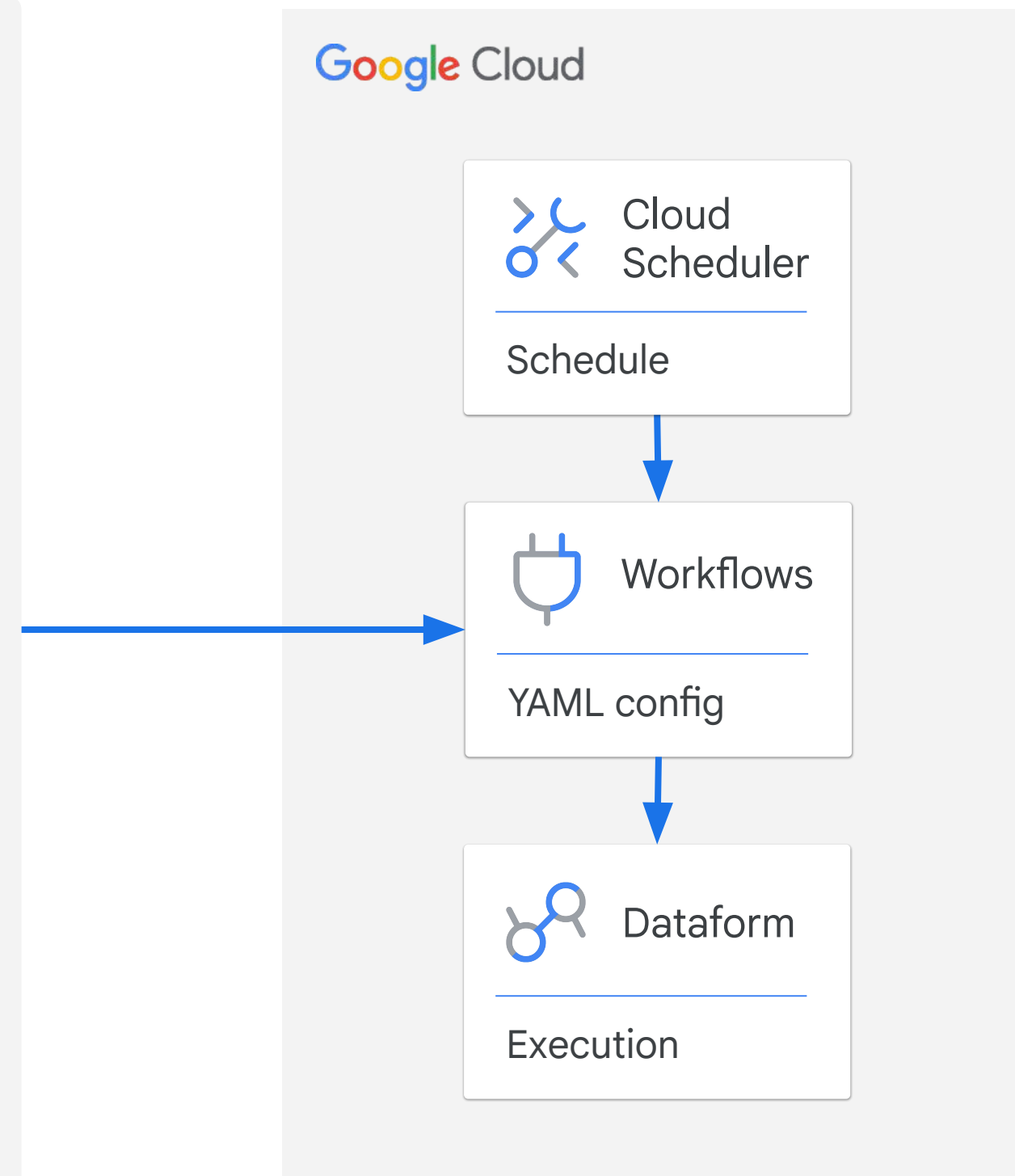
Your discipline is unchanged — source control, review, staged deployment, monitoring. Only the buttons move.

Cloud Scheduler invokes your workloads at recurring intervals



Example: Trigger a Dataform SQL workflow

```
- createCompilationResult:
  call: http.post
  args:
    url: ${"https://dataform.googleapis.com/[...]"}
    auth:
      type: OAuth2
    body:
      gitCommitish: <your_branch>
  result: compilationResult
- createWorkflowInvocation:
  call: http.post
  args:
    url: ${"https://dataform.googleapis.com/[...]"}
    auth:
      type: OAuth2
    body:
      compilationResult: ${compilationResult.body.name}
      invocationConfig:
        includedTags:
          - <your_tag>
  result: workflowInvocation
```



Compare automation options

	Cloud Scheduler	Managed Airflow	Cloud Run functions	Eventarc
Trigger type	schedule, manual	schedule, manual	event	event
Serverless	yes	no	yes	yes
Coding effort	low	medium	high	high
Programming languages	YAML (with Workflows)	Python	Python, Java, Go, Node.js, Ruby, PHP, .NET core	any

Day 1 done — what you now have

- ✓ Your data lands in BigQuery— Datastream CDC killed the nightly extract window before lunch
- ✓ The loading toolbox— console, CLI, autodetect pitfalls, Parquet vs CSV, LOAD DATA in SQL
- ✓ The Silver→Gold transform— a working Dataform SQL workflow from this afternoon's lab
- ✓ The automation map— Scheduler $\approx$ SQL Agent, Composer $\approx$ orchestrator, releases compiled from git
- ✓ Tomorrow— lakehouse and the medallion architecture: Bronze/Silver/Gold made formal, then Power BI

Closing demo — schedule the derived-table query

10 minutes — 16:20–16:30 · BigQuery Studio — instructor-led; replicate it after class

1. Run the sales-by-zip query once on the roi-bq-demos nested table — note bytes processed
2. Above the editor: Schedule → Create new scheduled query
3. Name it; repeat daily at 12:01 AM; destination class.march_zip_sales
4. Write preference: Overwrite table (WRITE_TRUNCATE) — rebuilt nightly, idempotent re-runs
5. Save, open Scheduled queries, and use the manual-run option once to prove it fires

Why it matters — your nightly SQL Agent job, translated: SQL on a schedule overwriting a table — no scheduler box, no agent service, no SSIS package.